



Tanta University

Faculty of Engineering

Department of Computer and Automatic Control



StructuraNet AI

AI-Powered Platform for Generating Validated, Simulation-Ready GNS3 Projects

A graduation project submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer and Control Engineering



Team Members

Seif El-Eslam Mohamed Ahmed

Samir Mohamed Samir

Farah Bassam El-Shenawy

Zyad Wael Attia

Seif El-Deen Abd El-Fatah Mahmoud

Nourhan Sameh Fayed

Ibrahim Hisham Al-Khouly

Ahmed Hussein Yousef

Supervise By

Prof. Assoc. Hany El-Ghaish

2025-2026

Approval and Certification

This graduation project report, entitled "StructuraNet AI: AI-Powered Platform for Generating Validated, Simulation-Ready GNS3 Projects," has been submitted to the Department of Computer Science and Control, Faculty of Engineering, University of Tanta, in partial fulfillment of the requirements for the graduation project.

Supervisor: Assoc. Prof. Hany Elgaish

Signature: _____ Date: _____

Department approval: _____

Signature: _____ Date: _____

Declaration

We hereby declare that the graduation project entitled "StructuraNet AI: Web Platform for Turning Network Vision into Production-Ready Infrastructure in Seconds" is our original work and has not been submitted elsewhere for any academic degree, diploma, or professional qualification. All sources of information, data, and materials used in this project have been duly acknowledged and referenced according to the IEEE citation standard.

To the best of our knowledge and belief, this project contains no material previously published or written by another person, except where due reference has been made in the text. No part of this project may be reproduced without the prior written permission of the authors and/or the Faculty of Engineering, University of Tanta.

We confirm that the development, design, implementation, and documentation of StructuraNet AI represent the collective effort of all team members listed on the cover page. Each member has contributed substantially to the research, coding, testing, and writing processes described in this document. The system architecture, source code, and experimental results presented herein are the product of our combined work under the supervision of Dr. Hany Elgaish.

Name: Seif El-Eslam Mohamed

Signature:

Date: / / 2026

Name: Seif El-Deen Abd El-Fatah

Signature:

Date: / / 2026

Name: Samir Mohamed

Signature:

Date: / / 2026

Name: Zyad Wael

Signature:

Date: / / 2026

Name: Ahmed Yousef

Signature:

Date: / / 2026

Name: Ibrahim El-Khouly

Signature:

Date: / / 2026

Name: Farah Bassam

Signature:

Date: / / 2026

Name: Nourhan Sameh

Signature:

Date: / / 2026

Supervisor: Assoc. Prof. Hany El-Gaish

Signature:

Date: / / 2026

Abstract

Network topology design is a foundational task in computer networking that has traditionally required significant manual effort, specialized expertise, and extensive familiarity with network simulation tools such as GNS3, Cisco Packet Tracer, and EVE-NG. Network engineers and students must manually place devices, configure interfaces, assign IP addressing schemes, and establish routing protocols — a process that is both time-consuming and error-prone, particularly for complex multi-node topologies. The growing demand for rapid network prototyping in educational institutions, enterprise environments, and DevOps workflows has highlighted the need for automated solutions that can reduce the gap between a network concept and its operational implementation.

Despite the availability of network simulation platforms, the initial design phase remains largely unautomated. Users must conceptualize the topology mentally and then translate that vision into a series of tool-specific configurations, a workflow that introduces cognitive overhead and opportunities for misconfiguration. Existing automation tools focus primarily on configuration management (e.g., NAPALM, Netmiko) or infrastructure-as-code (e.g., Terraform) but do not address the creative, conversational process of designing a topology from a natural-language description. This gap motivated the development of StructuraNet AI — a web-based platform that leverages Large Language Models

(LLMs) to transform natural-language network descriptions into visually editable, export-ready network topologies.

StructuraNet AI employs a conversational interface powered by an LLM equipped with custom tool-calling capabilities. Users describe their desired network in plain English, and the system interprets the request, generates a structured topology representation (nodes, links, and device roles), and renders it as an interactive visual canvas. The platform supports iterative refinement: users can request modifications through follow-up messages, and the LLM applies edits to the existing topology while preserving prior design decisions. Once the design is finalized, users can export the topology directly to GNS3-compatible format and auto-generate device configurations. The system architecture follows a full-stack design with a dedicated backend handling LLM communication via streaming (Server-Sent Events), a persistent database for session and topology storage, and a modern React-based frontend providing real-time topology visualization and user interaction.

The platform successfully demonstrates the feasibility of LLM-driven network topology generation. In initial testing, StructuraNet AI was able to interpret varied natural-language prompts — ranging from simple two-router topologies to more complex multi-site enterprise networks — and produce valid, GNS3-importable topology files. The interactive editing workflow proved effective for refining designs iteratively, and the auto-generated configurations provided a functional starting point for deployment. Key performance indicators include the reduction in

time from network concept to exportable topology, the accuracy of LLM-generated device assignments and interconnections, and the usability of the conversational interface as evaluated through structured test scenarios.

StructuraNet AI contributes to the intersection of artificial intelligence and network engineering by demonstrating that LLMs, when augmented with domain-specific tool definitions, can serve as effective co-pilots for network design tasks. The system lowers the barrier to entry for network prototyping, making it accessible to students, educators, and professionals who may lack deep familiarity with simulation tool interfaces. By automating the translation from natural language to structured network artifacts, StructuraNet AI represents a practical step toward intelligent network infrastructure automation, with potential applications in education, training, and rapid DevOps network provisioning.

Keywords: Network Topology Design, Large Language Model, GNS3, Network Automation, Conversational AI, Full-Stack Web Application, Server-Sent Events, Topology Visualization

Acknowledgement

First and foremost, gratitude and praises go to Almighty Allah, in whom we have put our faith and trust. We would like to extend our deep gratitude to our fathers and our lovely mothers for their honorable and courteous support all the way through our lives to date.

It was a great opportunity to gain experience on this project and learn how to design and analyze real projects. We would like to thank all the people who made this possible for us. Special thanks to the supervisor of the graduation project, Assoc. Prof. Hany El-Ghaish, for his efforts in providing us with all the useful information. His dynamism, vision, sincerity, and motivation have deeply inspired us.

He has taught us the methodology to carry out the project and to present the project works as clearly as possible. It was a great privilege and honor to work and study under his guidance. We are extremely grateful for what he has offered us.

We are extremely grateful to our parents for their love, prayers, caring, and sacrifices for educating and preparing us for our future.

Table of Contents

Approval and Certification	II
Table of Contents	1
List of Figures	11
List of Tables	14
List of Abbreviations	16
Chapter 1: Introduction	19
1.1 Preamble	20
1.2 Background	21
1.3 Problem Statement	23
1.4 Aim of the Project	24
1.5 Objectives	25
1.6 Scope	27
1.7 Significance	28
1.8 Limitations	29
1.9 Thesis Organization	31
Chapter 2: Literature Review and Background Technologies	33
2.1 Introduction	34
2.2 Large Language Models and Function Calling	34

2.2.1	Evolution of Large Language Models	34
2.2.2	Function Calling and Tool Use.....	36
2.2.3	LLM Providers: OpenAI and OpenRouter	37
2.3	Network Simulation Platforms	38
2.3.1	GNS3 (Graphical Network Simulator-3).....	39
2.3.2	Cisco Packet Tracer	40
2.3.3	EVE-NG (Emulated Virtual Environment - Next Generation)	41
2.4	Network Automation and Configuration Tools	42
2.4.1	NAPALM (Network Automation and Programmability Abstraction Layer with Multivendor support)	42
2.4.2	Netmiko	43
2.5	Web Application Technologies.....	44
2.5.1	React and Component-Based Architecture	44
2.5.2	Node.js and Express.js.....	45
2.5.3	MongoDB and NoSQL Databases.....	47
2.5.4	Real-Time Communication: Server-Sent Events	48
2.6	Python in Network Automation	49
2.6.1	Pydantic for Data Validation	49
2.6.2	OpenAI Python SDK	50

2.7 Frontend Visualization Technologies.....	51
2.7.1 D3.js for Network Topology Rendering.....	51
2.7.2 Tailwind CSS and Utility-First Design	52
2.7.3 Zustand for State Management.....	53
2.8 Comparative Analysis	54
2.8.1 Cross-Category Synthesis and Research Gap.....	56
2.9 Chapter Summary	61
Chapter 3: Requirements Analysis and User Experience Design.....	63
3.1 Introduction	64
3.2 Stakeholders and System Context.....	64
3.3 Functional Requirements.....	65
3.4 Non-Functional Requirements	66
3.5 Constraints and Assumptions	67
3.6 Requirements Traceability	68
3.7 User Interface Design System	69
3.7.1 Color Palette	69
3.7.2 Typography.....	72
3.7.3 Spacing, Shadows, and Animation.....	73
3.8 Landing Page Design	74

3.8.1 Header and Navigation	74
3.8.2 Hero Section	75
3.8.3 How It Works, Stats Strip, and Supporting Sections	76
3.9 Authentication Interface	77
3.10 Chat Interface	78
3.10.1 Layout Structure and Sidebar	78
3.10.2 Message Rendering and Streaming	79
3.10.3 Input Area and Action Chips	80
3.10.4 Action Trace and Progress Indication	81
3.11 Topology Visualization	81
3.11.1 Hierarchical Layout Algorithm	82
3.11.2 Full-Canvas Viewer	82
3.12 Export and Download Interface	83
3.13 Onboarding Flow	84
3.14 UI/UX Principles	85
3.15 Chapter Summary	88
Chapter 4: Backend Services, AI Engine, and Topology Generation	90
4.1 Introduction	91
4.2 System Architecture Overview	93

4.2.1 Three-Tier Architecture	93
4.2.2 Technology Stack and Dependencies	94
4.2.3 Request Lifecycle	97
4.3 Backend Framework and Middleware Stack	98
4.3.1 Express Application Configuration	98
4.3.2 Security and Performance Middleware	99
4.3.3 Error Handling Architecture	100
4.4 Authentication and Authorization	102
4.4.1 Dual-Token JWT Strategy	102
4.4.2 Middleware Variants	103
4.4.3 Input Validation with Zod	104
4.5 Database Design	105
4.5.1 Data Models Overview	105
4.5.2 User Model	106
4.5.3 Session Model.....	107
4.5.4 Topology and Export Models	108
4.5.5 Persistence Boundaries and Lifecycle Integrity	109
4.6 RESTful API Design	111
4.6.1 Authentication Endpoints	111

4.6.2 Session and Message Endpoints	113
4.6.3 Topology, Export, and Supporting Endpoints	114
4.6.4 API Contracts, Ownership Checks, and Error Semantics	116
4.7 Real-Time Communication via Server-Sent Events	119
4.7.1 SSE Service Architecture	119
4.7.2 Event Types and Data Flow.....	120
4.8 Chat Orchestration and LLM Integration.....	122
4.8.1 Hybrid Intent Routing.....	122
4.8.2 Tool-Use Loop.....	123
4.8.3 Security Profile Gating	126
4.9 Node.js–Python Bridge	127
4.9.1 Persistent Worker Architecture	127
4.9.2 JSONL Protocol.....	128
4.9.3 Error Recovery and Lifecycle Management.....	129
4.10 AI Engine: Topology Generation Pipeline.....	130
4.10.1 Phase 1: LLM-Driven Topology Synthesis.....	130
4.10.2 Auto-Repair Pipeline	132
4.10.3 Phase 2: Software Configuration Generation	133
4.10.4 Three-Gate Safe Merge	134

4.11 GNS3 Export Pipeline	137
4.11.1 Export Stages	137
4.11.2 Structural Validation (11 Checks)	138
4.12 Implementation Verification Boundaries	140
4.12.1 Deterministic Validation Hooks	140
4.12.2 Cross-Service Observability	141
4.12.3 Evidence Handoff to Chapter 6	141
4.13 Chapter Summary	142
Chapter 5: Development Methodology and Deployment	144
5.1 Introduction	144
5.2 Development Methodology	144
5.2.1 Iteration and Integration Cycle	145
5.2.2 Version-Control and Review Workflow	147
5.2.3 Repository-Evidenced Development Timeline	149
5.3 Team Contribution Breakdown	150
5.4 Deployment Architecture and Procedure	151
5.4.1 Prerequisites and Configuration	151
5.4.2 Operational Verification	152
5.4.3 Deployment Limitations	153

5.5 Risk Management and Deployment Readiness.....	154
5.6 Chapter Summary.....	161
Chapter 6: Results, Testing, and Evaluation.....	162
6.1 Introduction	163
6.2 Unit Testing.....	166
6.2.1 Node.js Bridge Argument Validation Tests	166
6.2.2 Node.js Bridge Export and Internal Tests	167
6.2.3 Pydantic Schema Validation Tests	168
6.2.4 Python Golden Export Tests.....	169
6.3 Integration Testing	171
6.3.1 Persistent Worker Bridge Tests	172
6.3.2 LLM-Dependent End-to-End Tests	173
6.4 Structural Validation Testing	176
6.4.1 Eleven-Check Validator Architecture	177
6.4.2 Validation Test Results.....	180
6.5 AI Pipeline Effectiveness Evaluation.....	181
6.5.1 Auto-Repair Pipeline Effectiveness	181
6.5.2 LLM Retry Loop Distribution	183
6.5.3 Three-Gate Safe Merge Verification	183

6.6 Performance Benchmarking	184
6.6.1 Worker Architecture Latency Comparison	184
6.6.2 SSE Streaming Latency	185
6.7 Interaction Case Testing	187
6.7.1 Core Intent Classification	187
6.7.2 Edge Case and Error Handling	188
6.8 Security Testing	190
6.8.1 Three-Gate Safe Merge Under Adversarial Input	190
6.8.2 JWT Authentication and Token Rotation	190
6.8.3 Security Profile Gating	191
6.8.4 Repository-Verified Cases and Failure-Mode Analysis	191
6.9 Usability Evaluation	198
6.9.1 Participant Study Protocol	199
6.9.2 Proposed Analysis and Reporting Rules	199
6.10 Chapter Summary	203
Chapter 7: Conclusion and Future Work	206
7.1 Conclusion	206
7.2 Summary of Key Contributions	206
7.3 Achievements Relative to Project Objectives	208

7.4 Challenges Encountered and Lessons Learned	208
7.5 Future Work	210
7.5.1 Multi-Provider LLM Support	211
7.5.2 Worker Pool for Concurrent Processing	212
7.5.3 Additional Simulation Platform Support	212
7.5.4 Automated Configuration Verification	213
7.5.5 Advanced Topology Patterns	213
7.5.6 Collaboration and Version Control Features	214
7.5.7 Frontend Enhancements and Mobile Support	214
7.5.8 Comprehensive Testing and Continuous Integration	215
References	217
Appendices	221
Appendix A: Complete GNS3 Device Catalog	222
Appendix B: AI Engine Worker Command Protocol	226
B.1 Request and Response Format	228
Appendix C: Eleven-Check Structural Validation Specifications	229
Appendix D: Pydantic Schema Definitions	232
Appendix E: Dynamips Hardware Compatibility Matrix	234
E.1 Module Interface Reference	236

Appendix F: Sample GNS3 Export Format (Golden Topology)	238
F.1 Portable Project ZIP Structure.....	240
Appendix G: Complete User Interface Screenshots	242
Appendix H: End-to-End Topology Generation Evidence.....	267
Appendix I: Security Analyzer Interface Evidence	283
Appendix J: Supplementary Strategic Analysis.....	289
J.1 SWOT Analysis	289
J.2 Business Model Canvas	290

List of Figures

Figure 1.1: StructuraNet AI system workflow	25
Figure 3.1: StructuraNet AI Use Case Diagram.	69
Figure 4.1: StructuraNet AI system architecture	93
Figure 4.2: StructuraNet AI technology stack	95
Figure 4.3: StructuraNet AI Database Entity Relationships.	109
Figure 4.4: Chat Orchestration and Topology Generation Sequence.	125
Figure 4.5: Three-Gate Safe Merge Flow.	135
Figure 5.1: Repository-Evidenced Iterative Development Timeline.	149

Figure 6.1: StructuraNet AI verification pyramid	163
Figure 7.1: StructuraNet AI proposed roadmap	211
Figure G.1: Chat Interface — Account Properties.	243
Figure G.2: Chat Interface — Main Workspace.	244
Figure G.3: Chat Interface — History Properties.	245
Figure G.4: Chat Interface — Image Specifications.	246
Figure G.5: Chat Interface — Sidebar.	247
Figure G.6: Landing Interface — Capabilities.	248
Figure G.7: Landing Interface — Documentation.	249
Figure G.8: Landing Interface — Footer.	250
Figure G.9: Landing Interface — Home Page.	251
Figure G.10: Landing Interface — How It Works.	252
Figure G.11: Landing Interface — Privacy Policy.	253
Figure G.12: Landing Interface — Security Information.	254
Figure G.13: Landing Interface — Use Cases.	255
Figure G.14: Authentication Interface — Log In.	256
Figure G.15: Authentication Interface — Sign In.	257
Figure G.16: Payment Interface — Secondary Payment View.	258

Figure G.17: Payment Interface — Payment Details.	259
Figure G.18: Payment Interface — Subscription Upgrade.	260
Figure G.19: Privacy Policy Interface — Page 1.	261
Figure G.20: Privacy Policy Interface — Page 2.	262
Figure G.21: Privacy Policy Interface — Page 3.	263
Figure G.22: Privacy Policy Interface — Page 4.	264
Figure G.23: Privacy Policy Interface — Page 5.	265
Figure G.24: Security Information Interface.	266
Figure H.1: Test Scenario 1 - Design.	268
Figure H.2: Test Scenario 1 - Download.	269
Figure H.3: Test Scenario 1 - Prompt.	270
Figure H.4: Test Scenario 1 — Topology Preview.	271
Figure H.5: Test Scenario 1 — Generated Topology.	272
Figure H.6: Test Scenario 2 - Design.	273
Figure H.7: Test Scenario 2 - Download.	274
Figure H.8: Test Scenario 2 - Prompt.	275
Figure H.9: Test Scenario 2 — Generated Topology.	276
Figure H.10: Test Scenario 2 — Topology View.	277

Figure H.11: Test Scenario 3 - Design.	278
Figure H.12: Test Scenario 3 - Download.	279
Figure H.13: Test Scenario 3 - Prompt.	280
Figure H.14: Test Scenario 3 — Topology Preview.	281
Figure H.15: Test Scenario 3 — Generated Topology.	282
Figure I.1: Security analyzer interface view 1.	284
Figure I.2: Security analyzer interface view 2.	285
Figure I.3: Security analyzer interface view 3.	286
Figure I.4: Security analyzer interface view 4.	287
Figure I.5: Security analyzer interface view 5.	288

List of Tables

Table 2.1: Feature comparison of network design and simulation platforms. (* Packet Tracer provides limited auto-configuration for specific lab scenarios. † GNS3 offers a web interface via the GNS3 Web UI project. ‡ EVE-NG Community is free and open-source; EVE-NG Professional requires a paid license.)	55
Table 2.2: Cross-Category Related-Work Synthesis	59

Table 3.1: StructuraNet AI color palette with semantic roles and usage contexts.

71

Table 3.2: Typography system — font families and their roles. 73

Table 4.1: Backend Production Dependencies 96

Table 4.2: Error Class Hierarchy 101

Table 4.3: Mongoose Data Models Overview 105

Table 4.4: Authentication API Endpoints 112

Table 4.5: Session and Message API Endpoints 114

Table 4.6: Supporting API Endpoints 115

Table 4.7: Server-Sent Events (SSE) Event Types 121

Table 4.8: Eleven-Check Structural Validator 139

Table 5.1: Repository-Evidenced Risk Register 155

Table 5.2: Deployment Readiness Gate 159

Table 6.1: Testing Strategy Overview 164

Table 6.2: Unit Test Summary 170

Table 6.3: Integration Test Summary 174

Table 6.4: Eleven-Check Structural Validator 179

Table 6.5: Auto-Repair Pipeline Effectiveness 182

Table 6.6: Performance Benchmark Summary	185
Table 6.7: Repository-Verified Test Cases and Evidence Status	192
Table 6.8: Proposed Usability Measures and Reporting Rules	200
Table A.1 — Complete StructuraNet AI GNS3 Device Catalog	223
Table B.1 — AI Engine Worker Command Protocol Specification	227
Table C.1 — Eleven-Check Structural Validation Specification	230
Table D.1 — Pydantic Schema Model Definitions	233
Table E.1 — Dynamips Platform Hardware Compatibility Matrix	235
Table E.2 — Module Interface Name and Port Count Reference	236
Table F.1 — GNS3 Portable Project ZIP Internal Structure	240
Table J.1: SWOT Analysis of StructuraNet AI.	289
Table J.2: Business Model Canvas for StructuraNet AI.	291

List of Abbreviations

AI → Artificial Intelligence

API → Application Programming Interface

CI → Continuous Integration

D3 → Data-Driven Documents

GNS3 → Graphical Network Simulator-3

HTTP → Hypertext Transfer Protocol

IOU/IOL → IOS on Unix / IOS on Linux

JSONL → JSON Lines

JWT → JSON Web Token

LLM → Large Language Model

ODM → Object Document Mapper

QEMU → Quick Emulator

REST → Representational State Transfer

SSE → Server-Sent Events

UI/UX → User Interface / User Experience

UUID → Universally Unique Identifier

VLAN → Virtual Local Area Network

REST → Representational State Transfer

CORS → Cross-Origin Resource Sharing

FSM → Finite-State Machine

ODM → Object Document Mapper

CI → Continuous Integration

CD → Continuous Delivery

UI → User Interface

UX → User Experience

UUID → Universally Unique Identifier

ZBF → Zone-Based Firewall



CHAPTER

1

Introduction

Project background, problem definition, motivation, objectives, scope, contributions, limitations, and thesis organization.

Chapter 1: Introduction

1.1 Preamble

The design and deployment of computer networks have long relied on the expertise of network engineers who translate business requirements and technical specifications into physical and logical topologies. This process involves selecting appropriate networking devices, determining interconnection patterns, assigning addressing schemes, and configuring protocols — all of which demand both theoretical knowledge and practical experience. As networks grow in scale and complexity, the manual approach to topology design becomes increasingly labor-intensive, creating a significant bottleneck in educational laboratories, enterprise provisioning workflows, and rapid prototyping environments.

StructuraNet AI is a web-based platform that addresses this challenge by enabling users to describe their desired network topology in natural language and receive a fully structured, GNS3-importable network project in return. The system integrates a Large Language Model (LLM) as its core intelligence, augmented with domain-specific tool definitions that allow it to generate, edit, and export network topologies through a conversational interface. By combining modern AI capabilities with established network engineering principles, StructuraNet AI bridges the gap between a conceptual network vision and a validated, simulation-ready GNS3 artifact.

1.2 Background

The field of network simulation has evolved considerably over the past two decades. Tools such as **GNS3 (Graphical Network Simulator-3)** have become industry-standard platforms for network emulation, allowing engineers and students to create virtual network topologies using real firmware images from vendors like Cisco, Juniper, and Arista. GNS3 operates by managing hypervisor processes (Dynamips for legacy Cisco hardware emulation, QEMU for x86-based appliances, and IOU for Cisco IOS on Unix) that run actual network operating systems, providing a high-fidelity simulation environment. However, the topology creation process in GNS3 remains fundamentally manual: users must drag devices onto a canvas, connect interfaces, configure each device individually, and ensure that the resulting topology is both logically sound and physically realizable within the constraints of the chosen emulation platform. [8], [35], [36]

Alternative simulation platforms such as **Cisco Packet Tracer** and **EVE-NG** offer similar capabilities with varying levels of fidelity and ease of use. Packet Tracer provides a simplified, educational environment with a visual drag-and-drop interface, while EVE-NG supports a broader range of enterprise network images and multi-user collaboration. Despite these differences, all share a common limitation: the initial design phase — deciding what devices to use, how many, and how they should be interconnected — is left entirely to the user’s expertise and imagination. No existing mainstream tool automates the translation from a natural-language description of a network to a complete, simulation-ready topology file.

On the automation side, tools such as **NAPALM** (Network Automation and Programmability Abstraction Layer with Multivendor support) and **Netmiko** provide programmatic interfaces for configuring network devices that are already deployed. **Terraform** and Ansible offer infrastructure-as-code frameworks that can provision and configure network resources at scale. However, these tools operate at the configuration and provisioning layer; they assume that the network topology has already been designed and that devices are already accessible. They do not assist with the creative, conceptual phase of deciding what the network should look like in the first place.

The emergence of **Large Language Models** (LLMs) such as GPT-4, Claude, and their open-source counterparts has opened new possibilities for automating tasks that previously required human reasoning. LLMs have demonstrated remarkable capabilities in understanding natural language, following complex instructions, and generating structured output — qualities that make them well-suited for interpreting a user's network description and producing a machine-readable topology specification. The function-calling (tool-use) paradigm, in which an LLM is provided with named tools and autonomously decides when to invoke them, provides a natural mechanism for integrating AI reasoning with deterministic computational steps such as port assignment, hardware validation, and file generation.

1.3 Problem Statement

Despite the maturity of network simulation platforms and the growing sophistication of network automation tools, a critical gap persists in the network design workflow. When a network engineer, student, or instructor needs to create a new network topology — whether for a classroom exercise, a proof-of-concept demonstration, or a pre-deployment validation — the process begins with a manual, error-prone sequence of steps. The user must mentally conceive the topology, translate that conception into a series of drag-and-drop operations within a simulation tool, manually configure each device's interfaces and protocols, and iteratively verify that the resulting setup matches the original intent. For a topology involving ten or more devices, this process can take thirty minutes to several hours, depending on the complexity of the design and the user's familiarity with the tool.

The problem is compounded in educational settings, where students who are still learning networking fundamentals must simultaneously master the mechanics of a simulation tool. The cognitive load of operating GNS3 — understanding its project structure, managing appliance images, configuring hypervisor connections — often detracts from the learning objectives of understanding network design principles themselves. Similarly, in enterprise environments, network architects who need to rapidly prototype multiple design alternatives for evaluation find the manual creation process to be a significant productivity bottleneck.

Existing automation solutions do not address this gap. Configuration management tools (NAPALM, Netmiko) assume the network already exists.

Infrastructure-as-code platforms (Terraform, Ansible) focus on provisioning and configuration at scale. None of these tools can take a natural-language description such as “Design a three-site enterprise network with OSPF routing, a DMZ, and VLAN segmentation” and produce a ready-to-import GNS3 project file. StructuraNet AI was developed to fill this specific gap in the network design toolchain.

1.4 Aim of the Project

The primary aim of this project is to design and implement a web-based platform, StructuraNet AI, that leverages Large Language Models to transform natural-language network descriptions into complete, validated, GNS3-importable network topologies with auto-generated device configurations. The system is intended to serve as an intelligent co-pilot for network design, reducing the time and expertise required to move from a conceptual network idea to a functional simulation project.

StructuraNet-AI Workflow



Figure 1.1: StructuraNet AI system workflow

1.5 Objectives

The specific objectives of this project are as follows:

1. **Develop a conversational interface** that accepts natural-language descriptions of network topologies and interprets user intent accurately, supporting both initial design requests and iterative modification requests through a multi-turn dialogue.

2. **Implement an LLM-powered topology generation engine** that translates natural-language requests into structured topology definitions comprising nodes, links, device types, and roles, using the OpenAI function-calling paradigm with custom tool definitions.

3. Build an automated validation and repair pipeline that checks generated topologies against GNS3 structural requirements (port limits, connectivity constraints, node-type legality) and automatically repairs common issues such as disconnected subgraphs and single-port violations before presenting results to the user.

4. Create an interactive topology visualization using a hierarchical 6-tier layout algorithm (NAT, Firewall, Router, Switch, Server, Endpoint) with drag-and-drop positioning, zoom and pan controls, and PDF export capability.

5. Implement a GNS3 export system that produces a valid .gns3project ZIP archive containing the project file, device startup configurations (with optional security hardening at None, Basic, or Enterprise levels), and an image requirements manifest.

6. Design and implement a full-stack web application with a React-based frontend (Vite, Zustand state management, Tailwind CSS), an Express.js backend (JWT authentication, MongoDB persistence, SSE streaming), and a Python AI Engine communicating via a persistent JSONL worker process.

7. Implement a GNS3 image calibration system that allows users to map appliance templates to their locally installed GNS3 images through an onboarding interface, ensuring that generated projects reference images the user actually has available.

1.6 Scope

In Scope. This project covers the complete design, implementation, and testing of the StructuraNet AI platform. Specifically, the scope includes the following: the development of a three-service architecture consisting of a React-based frontend, an Express.js backend, and a Python-based AI Engine; the implementation of LLM-driven topology generation with support for up to 45 appliance templates from the GNS3 appliance catalog; the creation of an interactive topology viewer with hierarchical layout and PDF export; the development of a GNS3 export pipeline that produces .gns3project files validated against 11 structural checks; the implementation of three security profile tiers (None, Basic, Enterprise) for device configuration hardening; user authentication and session management via JWT tokens and MongoDB; a GNS3 image calibration onboarding flow; and a Cisco IOS knowledge-base question-answering feature.

Out of Scope. The following are explicitly outside the scope of this project: support for non-Cisco device configurations (Juniper, Arista, Huawei); traffic simulation or packet-level analysis within the platform; real network deployment or live device provisioning; multi-user real-time collaboration on a single topology; native mobile applications (the platform is web-based and responsive but not a dedicated mobile app); support for serial, ATM, or Frame Relay links as primary connection types (the system focuses on Ethernet topologies); and integration with cloud provider APIs (AWS VPC, Azure VNet) for hybrid cloud topology generation.

1.7 Significance

Educational Impact. StructuraNet AI has significant implications for networking education. By automating the mechanical aspects of topology creation, the platform allows students to focus on understanding design principles — why a particular device is placed at a certain tier, why a specific routing protocol is chosen, or how VLAN segmentation improves network security — rather than struggling with the mechanics of a simulation tool’s interface. Instructors can use the platform to quickly generate varied lab topologies for exercises and examinations, reducing preparation time and enabling more diverse practical assessments. The conversational interface also serves as a teaching tool: students can ask the system to explain its design decisions, thereby receiving immediate, contextual feedback on network architecture.

Professional and Economic Value. For network engineers and DevOps teams, StructuraNet AI reduces the time required to move from a network concept to a testable simulation from hours to minutes. This acceleration has tangible economic value in environments where rapid prototyping is essential, such as pre-sales engineering, proof-of-concept demonstrations, and disaster recovery planning. The platform’s security profile feature (None, Basic, Enterprise) ensures that generated configurations adhere to recognized hardening practices, reducing the risk of deploying insecure lab environments that might inadvertently serve as templates for production networks.

Research Contribution. From a research perspective, StructuraNet AI demonstrates a practical application of LLM function-calling to a specialized engineering domain. The project introduces several novel techniques, including a three-gate safe-merge strategy for combining LLM-generated configurations with deterministic hardware properties, a pre-validation auto-repair pipeline that corrects common LLM errors before they reach the user, and a heuristic intent classifier that routes user messages to either a fast chat path or a full tool-calling path without requiring a separate LLM call. These techniques contribute to the broader field of AI-assisted engineering design.

1.8 Limitations

Several limitations were identified during the development and testing of StructuraNet AI, and these are acknowledged transparently to provide an accurate assessment of the system's current capabilities:

LLM-Dependent Accuracy. The quality of generated topologies is inherently dependent on the underlying LLM's understanding of networking concepts. While the system prompt includes detailed guardrails and the auto-repair pipeline corrects structural errors, the LLM may occasionally produce suboptimal design choices, such as placing a firewall at an incorrect tier or assigning an inappropriate device template for a described role. The retry mechanism (up to 3 attempts with repair hints) mitigates but does not eliminate this limitation.

Platform Link Constraints. Dynamips-emulated Cisco routers have strict limits on the number of supported interfaces per platform. For example, a Cisco 7200 router is limited to 3 Ethernet links due to PCI bus contention constraints, while a Cisco 3745 supports up to 6. The system enforces these limits at the prompt level, but complex topologies that require high-density connectivity may exceed what a single Dynamips platform can provide, necessitating the use of IOU or QEMU appliances instead.

Sequential Processing. The Python AI Engine processes commands sequentially (one at a time) through its persistent worker process. While this design choice simplifies state management and eliminates concurrency bugs, it means that concurrent requests from different users are queued. Under high load, this could introduce noticeable latency for subsequent users.

Cisco IOS Configuration Only. The auto-generated device configurations are limited to Cisco IOS command syntax. While the system supports multiple GNS3 node types (Dynamips, IOU, QEMU, Docker, VPCS), the configuration generation logic produces Cisco-style startup configs. Users working with non-Cisco appliances would need to manually modify the generated configurations.

Topology Size. The deterministic branch-room topology builder (used for large multi-site office networks) is capped at 180 total endpoint nodes (branches $\times$ rooms $\times$ people). For LLM-generated topologies, the effective size limit is governed by the LLM's token output window, which constrains the complexity of topologies that can be described in a single generation pass.

1.9 Thesis Organization

This document is organized into seven chapters, each addressing a distinct stage of the StructuraNet AI project. Chapter 1 introduces the project, establishes its motivation and problem statement, and defines the objectives, scope, significance, and limitations.

Chapter 2 reviews the technical background, related platforms, enabling web and AI technologies, comparative alternatives, and the technical gap addressed by StructuraNet AI. Supplementary strategic analysis is provided separately in Appendix J.

Chapter 3 formalizes stakeholders, functional and non-functional requirements, constraints, and requirements traceability, then documents the user experience and interface design that satisfy those requirements.

Chapter 4 presents the overall system architecture and technical implementation, including Express services, authentication, MongoDB persistence, APIs, SSE communication, LLM orchestration, the persistent Python worker, topology generation, validation, repair, configuration generation, and GNS3 export.

Chapter 5 documents the iterative development methodology, version-control and integration workflow, team contribution evidence, deployment architecture, operational prerequisites, and current production limitations.

Chapter 6 reports system testing and evaluation, including unit and integration evidence, structural validation, AI-pipeline effectiveness, performance

observations, security testing, usability inspection, and the proposed participant-study protocol.



CHAPTER

2

Literature Review and Background Technologies

Background concepts, related technologies, comparative analysis, and the technical gap addressed by StructuraNet AI.

Chapter 2: Literature Review and Background Technologies

2.1 Introduction

This chapter reviews the background technologies, theoretical concepts, related systems, and comparative technical context that inform the design and implementation of StructuraNet AI.

The chapter begins with an examination of Large Language Models (LLMs) and their function-calling capabilities, which constitute the core intelligence behind StructuraNet AI's topology generation engine. This is followed by a detailed review of the technology stack selected for the project, including the backend framework (Express.js and Node.js), the database layer (MongoDB with Mongoose ODM), the frontend libraries (React, Vite, Tailwind CSS, Zustand, and D3.js), and the Python-based AI Engine that handles topology generation, hardware injection, and GNS3 export.

The chapter then compares existing network simulation and automation platforms, including GNS3, Cisco Packet Tracer, EVE-NG, NAPALM, and Netmiko, and identifies the technical gap addressed by StructuraNet AI. SWOT analysis and the Business Model Canvas are supplementary strategic material and therefore appear in Appendix J rather than as core related work.

2.2 Large Language Models and Function Calling

2.2.1 Evolution of Large Language Models

Large Language Models (LLMs) represent a class of artificial intelligence systems trained on massive text corpora to understand and generate human

language. The evolution of LLMs can be traced through several pivotal milestones. Early transformer-based architectures, introduced by Vaswani et al. in 2017, replaced recurrent neural networks with self-attention mechanisms, enabling parallel processing of input sequences and significantly improving the quality of language understanding and generation tasks. This architectural breakthrough laid the foundation for subsequent models, including Google's BERT (2018), which demonstrated the effectiveness of bidirectional pre-training for natural language understanding, and OpenAI's GPT series (2018–present), which pioneered autoregressive generation for a wide range of language tasks. [1], [2], [4], [5]

The release of GPT-3 in 2020 marked a significant inflection point, demonstrating that sufficiently large models could exhibit emergent capabilities such as few-shot learning, chain-of-thought reasoning, and instruction following, without the need for task-specific fine-tuning. Subsequent models, including GPT-4 (2023), Claude by Anthropic, and open-weight models such as Meta's LLaMA series, have continued to expand the frontier of what LLMs can accomplish. These models have been applied to domains ranging from code generation and mathematical reasoning to medical diagnosis and legal analysis, demonstrating a broad capacity for domain-specific tasks when provided with appropriate prompts and context.

In the context of network engineering, LLMs offer a particularly compelling capability: the ability to translate imprecise, natural-language descriptions of network requirements into structured, machine-readable representations. A network

engineer might describe a topology as “a three-branch enterprise network with a core router, distribution switches, and firewall protection at the perimeter,” and an appropriately instructed LLM can decompose this description into a formal node-and-link structure complete with device roles and interconnection patterns. StructuraNet AI leverages this capability as its core intelligence mechanism, using the LLM not as a simple text generator but as a reasoning agent equipped with domain-specific tools.

2.2.2 Function Calling and Tool Use

Function calling, also referred to as tool use or tool calling, is a capability introduced by OpenAI in mid-2023 that allows LLMs to invoke predefined functions with structured arguments in response to user queries. Rather than generating free-form text in response to every prompt, the model can decide that a particular tool should be invoked, construct the appropriate arguments as a JSON object, and return a structured tool-call instruction that the application code can execute. The results of the tool execution are then fed back to the model, which interprets them and generates a human-readable response or invokes additional tools as needed. [3], [7]

This mechanism transforms the LLM from a passive text generator into an active agent that can interact with external systems, databases, and computation engines. In StructuraNet AI, the function-calling capability is central to the system’s architecture. The LLM is provided with four tool definitions: `generate_new_topology` (for creating new network designs from scratch),

`modify_current_topology` (for editing existing topologies based on user feedback), `apply_security_and_export` (for generating device configurations and producing the final GNS3 export package), and `search_cisco_knowledge` (for answering Cisco IOS configuration questions using a local knowledge base). The LLM autonomously decides which tools to invoke and in what sequence, enabling it to handle compound intents such as “Design a branch network and apply enterprise security profile” in a single conversational turn.

The function-calling approach offers several advantages over alternative architectures. Unlike a finite state machine (FSM), which requires explicit state transitions and intent classifiers, the LLM-based tool-calling approach handles context switching, compound intents, and ambiguous requests naturally through its understanding of the conversation history and the tool definitions. StructuraNet AI’s chat orchestrator further optimizes this process by implementing a hybrid routing strategy: a lightweight heuristic classifier determines whether a user message requires tool invocation or can be answered through a fast chat path (with `tool_choice` set to ‘none’), reducing response latency for non-design queries from several seconds to one to two seconds.

2.2.3 LLM Providers: OpenAI and OpenRouter

StructuraNet AI uses the OpenAI-compatible API as its LLM interface, with OpenRouter as the default provider. OpenRouter is an API gateway that aggregates multiple LLM providers (including OpenAI, Anthropic, Google, and open-weight models) behind a single, unified API endpoint. This design offers two key

advantages. First, it decouples the application from any single LLM provider, allowing the model to be swapped by changing an environment variable (AI_MODEL) without modifying the application code. Second, it provides access to a wide range of models at different price points and capability levels, from cost-effective models suitable for simple chat responses to high-capability models required for complex topology generation tasks. [1], [4]-[7] [6]

The OpenAI Python SDK and the openai npm package are used on the Python and Node.js sides of the system, respectively. Both sides implement retry logic with exponential backoff and jitter to handle transient API errors, including rate-limit (HTTP 429) responses, timeouts, and connection failures. The Python-side implementation in llm_utils.py provides a lazy singleton client that initializes once and is reused across all requests in the persistent worker process, while the Node.js-side implementation in chat.orchestrator.js creates a per-process client instance. The system's LLM call timeout defaults to 120 seconds per call, configurable via the LLM_CALL_TIMEOUT environment variable, providing a balance between allowing sufficient time for complex topology generation and preventing indefinite hangs.

2.3 Network Simulation Platforms

Network simulation platforms are essential tools for network engineers, students, and researchers who need to design, test, and validate network configurations without access to physical hardware. The following subsections

examine the most widely used platforms, identifying their strengths and limitations in relation to the capabilities that StructuraNet AI aims to provide.

2.3.1 GNS3 (Graphical Network Simulator-3)

GNS3 is an open-source network software emulator that allows users to create complex network topologies by emulating, rather than simulating, real network devices. Originally developed by Jeremy Grossmann in 2008, GNS3 has evolved into one of the most widely used network emulation platforms in both academic and professional settings. The platform supports multiple emulation engines, including Dynamips (for Cisco router emulation using real IOS images), IOU (IOS on Unix), QEMU (for generic virtual machines), and Docker containers. GNS3 provides a graphical interface for placing devices, connecting them with links, and configuring their properties, producing .gns3project files that encapsulate the entire topology. [8]

Despite its power and flexibility, GNS3 presents significant usability challenges that motivated the development of StructuraNet AI. Creating a topology in GNS3 requires the user to manually drag devices onto a canvas, configure each device's properties (including image paths, adapter modules, and RAM allocations), and draw connections between ports. The user must possess detailed knowledge of GNS3's device template system, including which adapter modules are compatible with which router platforms (e.g., a Cisco 7200 supports PA-FE-TX and PA-2FE-TX port adapters, while a Cisco 3745 uses NM-1FE-TX and NM-16ESW network modules). Incorrect configuration of these hardware properties

results in silent failures or import errors, making the topology creation process error-prone and time-consuming, particularly for users who are new to the platform.

GNS3 does not provide any form of natural-language interface or AI-assisted design capabilities. Users must have a clear mental model of their desired topology before they begin, and the translation from that mental model to a working GNS3 project is entirely manual. StructuraNet AI addresses this gap by serving as an intelligent frontend to the GNS3 ecosystem: users describe their network in natural language, and the system produces a fully configured `.gns3project` file that can be imported directly into GNS3, complete with correct hardware properties, port assignments, and device configurations.

2.3.2 Cisco Packet Tracer

Cisco Packet Tracer is a network simulation tool developed by Cisco Systems specifically for educational purposes. Unlike GNS3, which emulates real device firmware, Packet Tracer uses a simplified simulation engine that approximates the behavior of Cisco IOS commands without running actual firmware images. This makes Packet Tracer lightweight and accessible, as it does not require users to obtain or configure IOS image files. The tool provides a drag-and-drop interface for creating topologies and supports a subset of Cisco IOS commands for device configuration, making it well-suited for introductory networking courses and Cisco Networking Academy curricula. [9]

Packet Tracer's primary limitation from the perspective of StructuraNet AI is its closed, proprietary nature. The tool does not expose an API for programmatic

topology creation, and its project files use a proprietary format that cannot be easily generated by external tools. Additionally, Packet Tracer's simulation engine, while sufficient for educational use, does not support the full range of Cisco IOS features, particularly advanced security configurations such as Zone-Based Firewall (ZBF), DHCP snooping, or Dynamic ARP Inspection (DAI). StructuraNet AI targets the GNS3 ecosystem precisely because it supports real IOS images and full-featured configurations, making it suitable for both education and professional prototyping. Packet Tracer serves as a valuable reference point for the educational use case but is not a direct competitor in the automation space.

2.3.3 EVE-NG (Emulated Virtual Environment - Next Generation)

EVE-NG is another network emulation platform that, like GNS3, supports the emulation of real network device images. EVE-NG is available in two editions: a community edition (free, with limited features) and a professional edition (commercial, with additional features such as multi-user collaboration, HTML5 console access, and integration with external APIs). EVE-NG supports a wide range of network devices, including Cisco, Juniper, Arista, and Fortinet appliances, and provides both a desktop client and a web-based interface for topology management. [10]

EVE-NG's professional edition includes a REST API that could theoretically be used for programmatic topology creation, but this capability requires a paid license and is not designed for natural-language interaction. Like GNS3, EVE-NG requires users to manually place devices, configure properties, and establish

connections through its graphical interface. The platform does not incorporate any AI or natural-language processing capabilities, and its topology creation workflow remains fundamentally manual. StructuraNet AI differentiates itself by providing an AI-driven design layer that operates independently of any specific simulation platform, with GNS3 as the initial export target and potential future support for EVE-NG's format.

2.4 Network Automation and Configuration Tools

While StructuraNet AI focuses on topology design and initial configuration generation, it is important to situate the project within the broader landscape of network automation tools. The following subsections examine two widely used Python libraries for network device interaction and explain how StructuraNet AI's scope differs from and complements these tools. [12], [14], [15]

2.4.1 NAPALM (Network Automation and Programmability Abstraction Layer with Multivendor support)

NAPALM is a Python library that provides a unified, vendor-neutral API for interacting with network devices from multiple manufacturers, including Cisco, Juniper, Arista, and MikroTik. NAPALM abstracts the differences between vendor-specific CLI interfaces and APIs, allowing network engineers to write automation scripts that work across heterogeneous device environments. The library supports common operations such as retrieving device facts, configuration management (`get_config`, `load_replace_candidate`, `commit_config`), and running RPC calls. NAPALM is widely used in production network automation, particularly in

conjunction with configuration management frameworks such as Ansible and SaltStack. [11]

NAPALM operates at a fundamentally different stage of the network lifecycle compared to StructuraNet AI. NAPALM is designed for managing configurations on existing, deployed devices (whether physical or virtual). It assumes that the network topology already exists and that the engineer knows which devices need to be configured. StructuraNet AI, by contrast, operates at the design phase: it helps users conceive and formalize the topology itself, producing the initial configuration files that would subsequently be deployed using tools like NAPALM. The two tools are complementary rather than competitive, with StructuraNet AI handling the creative design phase and NAPALM handling the operational deployment phase.

2.4.2 Netmiko

Netmiko is a Python library developed by Kirk Byers that simplifies SSH connections to network devices. Built on top of the Paramiko SSH library, Netmiko provides device-specific connection handlers that account for the differences in CLI behavior across vendors (Cisco IOS, Juniper JunOS, Arista EOS, and others). Netmiko is commonly used for tasks such as retrieving device configurations, pushing configuration changes, and collecting command outputs across large numbers of devices. Its simplicity and reliability have made it one of the most popular Python libraries for network automation, particularly for tasks that involve direct CLI interaction. [13]

Like NAPALM, Netmiko is an operational tool that interacts with existing devices. It does not assist with topology design or initial configuration generation. StructuraNet AI's auto-generated Cisco IOS startup configs could serve as input for Netmiko-based deployment scripts, creating a workflow where the AI generates the initial configuration, the engineer reviews and modifies it as needed, and Netmiko deploys it to the target devices. This complementary relationship underscores the fact that StructuraNet AI occupies a unique position in the network automation ecosystem, addressing the design phase that precedes the operational phase served by tools like NAPALM and Netmiko.

2.5 Web Application Technologies

StructuraNet AI is implemented as a full-stack web application, and the selection of appropriate web technologies was critical to the project's success. This section examines the key technologies that comprise the application's technology stack, explaining the rationale for each choice and its role within the system architecture.

2.5.1 React and Component-Based Architecture

React is a JavaScript library developed by Meta (formerly Facebook) for building user interfaces through a component-based architecture. Released in 2013, React introduced the concept of a virtual DOM (Document Object Model), which enables efficient UI updates by computing the minimal set of changes required to synchronize the rendered page with the application's state. React's component model encourages the decomposition of complex interfaces into small, reusable,

and independently testable units, each managing its own state and rendering logic. This architectural paradigm aligns well with StructuraNet AI's frontend requirements, which include a chat interface, a topology viewer, export controls, and authentication flows, each of which can be implemented as a self-contained component tree. [16]

StructuraNet AI uses React 18.3.1, which introduced concurrent rendering features that improve the responsiveness of complex UIs by allowing React to interrupt, pause, and resume rendering work as needed. The application's component architecture includes specialized components for each functional area: ChatLayout, ConversationView, Sidebar, TopologyPreviewCard, DownloadKit, TopologyFullCanvas, and OnboardingModal, among others. React Router v6 provides client-side routing between the landing page, authentication pages, chat interface, and shared chat views. The choice of React over alternatives such as Vue.js or Angular was driven by the team's familiarity with the library, the breadth of its ecosystem (including specialized libraries for markdown rendering, PDF generation, and state management), and the availability of high-quality UI component patterns.

2.5.2 Node.js and Express.js

Node.js is a JavaScript runtime built on Chrome's V8 engine that enables server-side execution of JavaScript code. Its event-driven, non-blocking I/O model makes it particularly well-suited for real-time web applications that handle many concurrent connections, such as chat systems and streaming services. Express.js is

a minimalist web application framework for Node.js that provides a robust set of features for building HTTP APIs, including routing, middleware composition, request parsing, and error handling. Express has been the most widely used Node.js web framework since its initial release in 2010, and its extensive middleware ecosystem allows developers to add functionality such as authentication, rate limiting, CORS handling, and security headers with minimal configuration. [17], [18]

In StructuraNet AI, Express.js serves as the backend API server, handling user authentication (JWT-based login, registration, and token refresh), session management, topology storage and retrieval, file download endpoints, and SSE (Server-Sent Events) streaming for real-time AI progress updates. The backend also acts as the bridge between the React frontend and the Python AI Engine, managing the persistent worker process that communicates via stdin/stdout JSONL (newline-delimited JSON). This three-service architecture (React frontend, Express backend, Python AI Engine) provides clear separation of concerns and allows each service to be developed, tested, and scaled independently.

The Express backend uses several key middleware packages: Helmet for security headers, cors for cross-origin resource sharing, express-rate-limit for API rate limiting, cookie-parser for session cookie management, compression for HTTP response compression, and morgan for HTTP request logging. Authentication is implemented using jsonwebtoken for JWT generation and verification and bcryptjs

for password hashing, following security best practices for web application authentication.

2.5.3 MongoDB and NoSQL Databases

MongoDB is a document-oriented NoSQL database that stores data in flexible, schema-less JSON-like documents organized into collections. Unlike relational databases, which require a predefined schema with tables, rows, and columns, MongoDB allows each document in a collection to have a different structure, providing the flexibility needed for applications with evolving data models. MongoDB supports rich queries, secondary indexes, aggregation pipelines, and horizontal scaling through sharding, making it suitable for a wide range of application types, from content management systems to real-time analytics platforms. [19], [20]

StructuraNet AI uses MongoDB as its primary data store, accessed through the Mongoose ODM (Object Document Mapper) library, which provides schema validation, middleware hooks, and a fluent query API. The application defines four Mongoose models: User (storing user credentials, profile information, and GNS3 calibration settings), Session (storing conversation sessions with embedded messages), Topology (storing topology snapshots linked to sessions), and ExportJob (storing export job metadata and file paths). The choice of MongoDB over a relational database such as PostgreSQL was driven by the document-centric nature of the application's data: conversation messages, topology dictionaries, and export metadata are naturally represented as nested documents, and MongoDB's

flexible schema accommodates the variability in topology structures (different node types, different property sets) without requiring complex table relationships or schema migrations.

2.5.4 Real-Time Communication: Server-Sent Events

Server-Sent Events (SSE) is a web standard (W3C specification) that enables servers to push real-time updates to clients over a persistent HTTP connection. Unlike WebSocket, which provides full-duplex communication, SSE is unidirectional (server-to-client), making it simpler to implement and deploy. SSE connections are established using the standard EventSource API in the browser, which provides automatic reconnection, event type discrimination, and native integration with the browser's networking stack. SSE is particularly well-suited for applications that require real-time server push but do not need client-to-server streaming, such as live feeds, notification systems, and AI progress updates. [21]

StructuraNet AI uses SSE as its primary real-time communication mechanism between the Express backend and the React frontend. When a user sends a message that triggers the AI pipeline, the backend begins streaming events such as `tool_start` (indicating that a tool has been invoked), `tool_progress` (providing intermediate progress updates), `topology_ready` (signaling that the topology has been generated), `token_delta` (streaming individual LLM text tokens for the chat response), and `complete` (signaling that the entire generation process has finished). The frontend's `SSEManager` class (implemented in `client/src/services/sse.js`) wraps the browser's EventSource API with authentication support (JWT token passed as a query

parameter), exponential backoff reconnection logic (up to 5 attempts), and event parsing. The choice of SSE over WebSocket was deliberate: SSE's HTTP compatibility ensures that the streaming connection works through corporate proxies and CDNs without additional configuration, and the unidirectional nature matches the application's communication pattern (the client sends commands via standard HTTP POST requests and receives streaming updates via SSE).

2.6 Python in Network Automation

Python has become the de facto programming language for network automation, owing to its readable syntax, extensive standard library, and rich ecosystem of networking-specific packages. The Python AI Engine in StructuraNet AI leverages several key Python technologies to implement the topology generation pipeline, hardware injection, and GNS3 export functionality. This section examines the two most critical Python dependencies: Pydantic for data validation and the OpenAI Python SDK for LLM interaction.

2.6.1 Pydantic for Data Validation

Pydantic is a Python library for data validation and settings management using Python type annotations. Released in 2017 and rewritten for version 2.0 in 2023, Pydantic provides runtime type checking, data serialization, and schema generation with minimal boilerplate. Models are defined as Python classes with type-annotated fields, and Pydantic automatically validates input data against these type annotations, raising descriptive validation errors when constraints are violated. Pydantic's performance is significantly better than alternative validation libraries

due to its Rust-based core (in v2), making it suitable for performance-sensitive applications. [22]

StructuraNet AI's Python AI Engine uses Pydantic v2 extensively for validating the structure of generated topologies. The GNS3Project model (defined in constants/schema.py) enforces constraints on node types, link endpoints, port assignments, and VLAN configurations, ensuring that the LLM's output conforms to the requirements of the GNS3 project file format. Additional validators check for port collisions, link limits per platform, connectivity completeness, and interface count bounds. This validation layer serves as a critical safety net between the LLM's probabilistic output and the deterministic GNS3 export process, catching structural errors before they propagate to the export stage.

2.6.2 OpenAI Python SDK

The OpenAI Python SDK is the official client library for interacting with the OpenAI API and any compatible API endpoint (including OpenRouter). The SDK provides a high-level interface for chat completions, function calling, and embeddings, handling low-level concerns such as HTTP connection management, request serialization, response parsing, and error handling. In StructuraNet AI, the OpenAI SDK is used in the Python AI Engine (via llm_utils.py) for two critical operations: Phase 1 topology generation (where the LLM produces a structured node-and-link representation from the user's natural-language description) and Phase 2 configuration generation (where the LLM produces device startup configurations based on a detailed configuration brief). [23]

The SDK's support for function calling is particularly important: it allows the AI Engine to define tool schemas and receive structured tool-call responses from the model, enabling the agent-based architecture described in Section 2.2.2. The `llm_utils.py` module implements a lazy singleton pattern for the OpenAI client, ensuring that the API key and base URL are loaded from environment variables only once, and provides a retry wrapper (`_call_with_retry`) that handles transient errors (timeouts, rate limits, connection failures) with exponential backoff and jitter. This retry logic is essential for maintaining reliability in production, as LLM API calls may occasionally fail due to network issues or provider-side rate limiting.

2.7 Frontend Visualization Technologies

The frontend of StructuraNet AI employs several specialized visualization and UI technologies that are critical to the user experience. This section examines D3.js for network topology rendering, Tailwind CSS for utility-first styling, and Zustand for state management.

2.7.1 D3.js for Network Topology Rendering

D3.js (Data-Driven Documents) is a JavaScript library for producing dynamic, interactive data visualizations in web browsers. Created by Mike Bostock in 2011, D3 provides low-level building blocks for binding data to DOM elements, applying data-driven transformations, and creating animated transitions. Unlike higher-level charting libraries (such as Chart.js or ECharts), D3 does not provide predefined chart types; instead, it offers primitive operations (scales, axes, shapes, forces,

zoom behaviors) that can be composed to create virtually any type of visualization. [24], [25]

StructuraNet AI uses D3.js (specifically the d3-force, d3-selection, and d3-zoom modules) to render the interactive network topology visualization. The topology layout algorithm implements a six-tier hierarchical arrangement that positions nodes according to their role in the network: NAT devices at the top, followed by firewalls, routers, switches, servers, and endpoint devices (VPCS) at the bottom. This tiered layout provides an intuitive visual representation of the network's logical structure. D3's force simulation is used to compute node positions, while d3-zoom enables pan and zoom interactions, and SVG rendering provides the graphical output. The topology viewer supports both light and dark modes, PDF export via jsPDF, and click-to-inspect interactions that display device properties and configurations.

2.7.2 Tailwind CSS and Utility-First Design

Tailwind CSS is a utility-first CSS framework that provides low-level CSS classes for styling HTML elements directly in markup, rather than through predefined component styles or semantic class names. Each Tailwind class corresponds to a single CSS property (for example, flex for display:flex, text-center for text-align:center, p-4 for padding:1rem), allowing developers to compose styles by combining utility classes. This approach eliminates the need for naming CSS classes, reduces the risk of style conflicts, and enables rapid prototyping of user interfaces. [26], [32]

StructuraNet AI uses Tailwind CSS v3.4.10 as its primary styling framework, supplemented by the `@tailwindcss/typography` plugin for styling markdown content rendered in the chat interface. The utility-first approach was chosen for its ability to support a dark-themed, modern UI with consistent spacing and typography scales. The application's design system uses a carefully curated color palette, consistent border-radius values, and responsive breakpoints that adapt the layout for different screen sizes. Vite (v7.3.6) serves as the build tool, providing fast Hot Module Replacement (HMR) during development and optimized production builds.

2.7.3 Zustand for State Management

Zustand is a lightweight state management library for React that provides a simple, hook-based API for creating and consuming global state. Unlike Redux, which requires action types, reducers, and dispatch functions, Zustand allows developers to define stores as plain JavaScript objects with methods that directly mutate state, while still maintaining React's unidirectional data flow through its hook-based subscription model. Zustand's minimal API surface and lack of boilerplate make it well-suited for applications of moderate complexity that do not require the full infrastructure of Redux. [27]

StructuraNet AI defines two Zustand stores: `authStore` (managing user authentication state, tokens, and profile data) and `chatStore` (managing chat sessions, messages, topology state, export kit data, and SSE connection state). The `chatStore` is particularly important, as it serves as the central state container for the

AI interaction flow, tracking the current session's messages, the latest topology data, and the streaming state of the AI response. When SSE events arrive from the backend, the store's action methods update the state, triggering React re-renders that display the topology preview, stream the LLM's text response, and update the download kit with export file URLs. This architecture ensures that the UI remains responsive and consistent during the potentially long-running AI generation process.

2.8 Comparative Analysis

The following table presents a feature-by-feature comparison of StructuraNet AI against the existing platforms and tools discussed in the preceding sections. The comparison highlights the unique capabilities that StructuraNet AI brings to the network design and simulation ecosystem, while also acknowledging the areas where existing tools remain stronger. A check mark (✓) indicates full support, a cross (X) indicates no support, an em dash (—) indicates not applicable, and an asterisk (*) denotes partial or limited support. A dagger (†) indicates the GNS3 Web UI, and a double dagger (‡) indicates that EVE-NG's community edition is open-source while the professional edition is commercial.

Table 2.1: Feature comparison of network design and simulation platforms. (* Packet Tracer provides limited auto-configuration for specific lab scenarios. † GNS3 offers a web interface via the GNS3 Web UI project. ‡ EVE-NG Community is free and open-source; EVE-NG Professional requires a paid license.)

Feature	GNS3	Packet Tracer	EVE-NG	NAPALM	Netmiko	StructuraNet AI
Natural Language Input	✗	✗	✗	✗	✗	✓
AI-Powered Generation	✗	✗	✗	✗	✗	✓
Topology Visualization	✓	✓	✓	✗	✗	✓
GNS3 Export	—	✗	✗	✗	✗	✓
Auto Config Generation	✗	✓*	✗	✗	✗	✓
Iterative Editing	✓	✓	✓	✗	✗	✓
Security Profiles	✗	✗	✗	✗	✗	✓
Web-Based Interface	✓†	✓	✓	✗	✗	✓
Real-Time Streaming	✗	✗	✗	✗	✗	✓
Multi-Device Catalog	✓	✓	✓	✗	✓	✓

Feature	GNS3	Packet Tracer	EVE-NG	NAPALM	Netmiko	StructuraNet AI
Cisco QA Knowledge Base	✗	✗	✗	✗	✗	✓
Open Source	✓	✗	✗‡	✓	✓	✓

The comparison reveals that StructuraNet AI occupies a unique position in the landscape. No existing platform or tool combines natural-language input, AI-powered topology generation, GNS3 export, iterative editing, security profile application, and real-time streaming in a single web-based application. GNS3, Packet Tracer, and EVE-NG provide the simulation and visualization capabilities but lack any form of AI assistance or natural-language interface. NAPALM and Netmiko provide operational automation but do not address the design phase. StructuraNet AI bridges this gap by serving as an intelligent design layer that sits above the simulation layer, transforming the creative, conceptual phase of network design from a manual, expertise-intensive process into a conversational, AI-assisted workflow.

2.8.1 Cross-Category Synthesis and Research Gap

The reviewed literature separates naturally into four technical traditions: network simulation, network automation, infrastructure as code, and language-model-assisted software systems. Network simulators provide an execution environment, but they generally begin after the user has already selected devices and decided how those devices should be connected. GNS3 prioritizes realistic emulation and appliance flexibility [8], while Packet Tracer prioritizes accessibility for education

[9] and EVE-NG emphasizes broad image support and multi-user laboratory operation [10]. These capabilities are valuable, yet none of them converts an imprecise conversational requirement into a complete, checked topology as its primary workflow.

Network automation tools address a different stage of the lifecycle. NAPALM and Netmiko expose programmatic ways to retrieve state and apply configuration to identifiable devices [11], [13]. Terraform and Ansible make provisioning and configuration repeatable through declared state and playbooks [14], [15]. Their common assumption is that an operator already knows the inventory, addressing plan, connectivity, and desired policy. StructuraNet AI therefore does not replace these tools. It occupies an earlier design stage and produces structured artifacts that could later become inputs to configuration verification or infrastructure-as-code workflows.

Large language models supply the missing conversational capability but introduce a reliability problem. GPT-4 demonstrates strong instruction following and structured generation [1], and function calling allows a model to select constrained software operations rather than returning only prose [3]. Nevertheless, a language model has no inherent guarantee that a chosen router has sufficient ports, that two links do not reuse an interface, or that an exported archive satisfies GNS3 schema rules. The literature consequently supports the use of an LLM as an intent interpreter, but not as an unchecked authority over the final engineering artifact.

The research gap is the absence of an integrated workflow that begins with natural-language requirements, retains conversational context during edits, constructs a visual topology, enforces appliance-specific physical constraints, generates configuration in a separate phase, and packages the result for a real emulator. Existing systems provide individual parts of this chain. StructuraNet AI contributes the orchestration and validation boundary between those parts: model reasoning is used where requirements are ambiguous, while schema validation, port assignment, hardware rules, safe merge, and archive validation remain deterministic.

The cross-category comparison in Table 2.2 is functional rather than a claim of universal superiority. For example, a manually authored GNS3 project may provide greater expert control, and an Ansible workflow may be more appropriate for an established production inventory. StructuraNet AI is strongest when the user needs rapid exploration, an editable first design, and a portable laboratory artifact. This positioning also defines the evaluation criteria used later in the thesis: correctness must be assessed at service boundaries and export constraints, not only by judging whether an LLM response sounds plausible.

Table 2.2: Cross-Category Related-Work Synthesis

Technology category	Representative systems	Established strength	Limitation relative to the project	StructuraNet AI response
Network simulation and emulation	GNS3, Packet Tracer, EVE-NG	High-fidelity or education-oriented execution environments	Topology conception, appliance selection, and initial construction remain user-led	Produces a validated design artifact for later execution in GNS3
Device automation	NAPALM and Netmiko	Programmatic configuration and operational access to existing devices	Assumes devices and connectivity are already known	Generates the topology model before configuration is produced

Technology category	Representative systems	Established strength	Limitation relative to the project	StructuraNet AI response
Infrastructure as code	Terraform and Ansible	Repeatable provisioning and configuration workflows	Optimized for declared infrastructure rather than conversational exploration	Uses conversation for requirements discovery, then deterministic export stages
General-purpose LLMs	GPT-family and OpenRouter-accessible models	Natural-language interpretation and structured tool use	Outputs may be inconsistent, incomplete, or hardware-infeasible	Constrains model output with schemas, hardware injection, repair, and validation

Technology category	Representative systems	Established strength	Limitation relative to the project	StructuraNet AI response
Visualization frameworks	React and D3.js	Interactive rendering and component-based interaction	Does not itself create network semantics or exportable projects	Connects an editable visual representation to persisted topology and export state

The synthesis leads to three design implications. First, generated output must remain reviewable because expert control is a strength of existing simulators. Second, topology and configuration generation should be separated because automation tools operate on a known design rather than discovering it at the same time. Third, evaluation must distinguish deterministic evidence from provider-dependent behavior. These implications directly motivate the two-phase pipeline, Three-Gate Safe Merge, persistent worker boundary, and evidence classifications presented in Chapters 4 and 6.

2.9 Chapter Summary

This chapter has presented a comprehensive review of the background technologies and competitive landscape that inform the design and development of StructuraNet AI. The chapter examined Large Language Models and their function-

calling capabilities, which serve as the core intelligence mechanism for topology generation. It reviewed the major network simulation platforms (GNS3, Cisco Packet Tracer, EVE-NG) and network automation tools (NAPALM, Netmiko), identifying the gap in AI-assisted topology design that StructuraNet AI fills. The chapter also analyzed the web application technologies that comprise the project's technology stack, including React, Node.js, Express.js, MongoDB, SSE, D3.js, Tailwind CSS, Zustand, Pydantic, and the OpenAI SDK.

The comparative analysis indicates that the reviewed solutions do not combine natural-language topology generation, GNS3 export, iterative conversational editing, and security hardening in one workflow. This finding establishes the technical gap that motivates the architecture and implementation presented in subsequent chapters. Supplementary SWOT and business-model material is retained in Appendix J.



CHAPTER

3

Requirements Analysis and User Experience Design

Stakeholders, functional and non-functional requirements, constraints, traceability, user flows, and interface design.

Chapter 3: Requirements Analysis and User Experience Design

3.1 Introduction

This chapter documents the graphic design system, user interface architecture, and user experience principles that govern the visual and interactive behavior of StructuraNet AI. The platform's design is a critical component of its value proposition: because the system translates natural language into complex network artifacts, the interface must feel intuitive, trustworthy, and responsive, reducing the cognitive burden on users who may range from experienced network engineers to undergraduate students encountering network design for the first time.

The chapter is organized as follows. Sections 3.2 through 3.6 define stakeholders, functional requirements, non-functional requirements, constraints, and traceability. Section 3.7 presents the design system. Sections 3.8 through 3.14 document the landing page, authentication, chat workspace, topology visualization, export interface, onboarding flow, and UI/UX principles. Section 3.15 summarizes the chapter.

3.2 Stakeholders and System Context

Primary stakeholders include networking students, instructors, laboratory administrators, network engineers, project maintainers, and system administrators. End users require a conversational route from an idea to an editable topology and downloadable GNS3 artifacts. Administrators require secure authentication, controlled credentials, predictable resource consumption, recoverable worker

processing, and auditable failures. Maintainers require modular services, validated interfaces, executable tests, and reproducible installation instructions.

3.3 Functional Requirements

FR-01 — Account management. The system shall provide registration, login, token refresh, logout, password reset, and protected profile access. Acceptance is demonstrated when an authenticated user can access protected resources and revoked or invalid tokens are rejected.

FR-02 — Natural-language topology design. The system shall accept a network requirement through the chat interface and create a structured topology containing devices, links, roles, and hardware properties.

FR-03 — Iterative modification. The system shall apply follow-up design requests while preserving the active session and prior topology context.

FR-04 — Interactive visualization. The client shall render the generated nodes and links in an inspectable topology canvas with device details and layout controls.

FR-05 — Validation and bounded repair. The Python engine shall validate topology structure, reject invalid schema data, and apply bounded repair before returning a result.

FR-06 — Security-profile confirmation. The workflow shall require the applicable security profile before configuration generation and final export.

FR-07 — Separate configuration phase. Phase 1 shall generate the topology.

Phase 2 shall generate software configurations only when the export command is invoked.

FR-08 — GNS3 packaging. The export workflow shall generate a downloadable GNS3 project together with configuration and manifest artifacts.

FR-09 — Progress streaming. The backend shall stream phase, status, thought, validation, and completion events to the active client through Server-Sent Events.

FR-10 — Persistence. The system shall store users, sessions, messages, topologies, export jobs, and relevant metadata in MongoDB.

FR-11 — History retrieval. Authenticated users shall be able to retrieve and continue earlier sessions.

FR-12 — Structured failure reporting. Invalid requests, worker failures, timeouts, validation failures, and export errors shall produce structured, non-sensitive error responses.

3.4 Non-Functional Requirements

NFR-01 — Security. Passwords must be hashed, access and refresh tokens must be separated, refresh tokens must support revocation, and credentials must be supplied through environment variables.

NFR-02 — Reliability. Worker calls must use unique request identifiers, bounded timeouts, structured error envelopes, readiness detection, and restart handling.

NFR-03 — Performance. User-visible streaming should start promptly. Default maximum execution windows are fifteen minutes for generation, editing, and QA and twenty minutes for export, as configured in `backend/src/config/index.js`.

NFR-04 — Usability. Primary design, modification, inspection, and export actions must be accessible from the conversational workspace with visible progress and recovery information.

NFR-05 — Maintainability. React client code, Express services, and the Python AI Engine must remain separated by documented interfaces and validated schemas.

NFR-06 — Portability. Configuration must avoid machine-specific secrets and support environment-dependent MongoDB, model-provider, Python, and GNS3 settings.

NFR-07 — Data integrity. Zod, Pydantic, topology validation, and GNS3 export validation must reject malformed data at component boundaries.

NFR-08 — Traceability. Each critical requirement must map to an implementation component and verification evidence.

3.5 Constraints and Assumptions

The system depends on a reachable MongoDB instance, compatible Node.js and Python runtimes, an available LLM service, and local GNS3 appliance images. Generated network configurations are engineering starting points and require

qualified review before production use. Appliance names and capabilities differ among GNS3 installations; therefore, the onboarding image map is environment-specific input rather than a universal guarantee.

3.6 Requirements Traceability

FR-01 maps to the authentication service, JWT middleware, User model, and authentication security tests. FR-02 through FR-05 map to the chat orchestrator, generate command, Pydantic schemas, topology validator, repair logic, and end-to-end scenario evidence. FR-06 through FR-08 map to the security gate, export command, configuration agent, golden-export test, and GNS3 exporter. FR-09 maps to the SSE service and streaming tests. FR-10 and FR-11 map to the Session, Topology, and Export models and the history interface. FR-12 maps to Express error middleware and the JSONL error envelope used by the worker bridge.

Figure 3.1 summarizes the principal actors and user-visible use cases derived from the functional requirements.

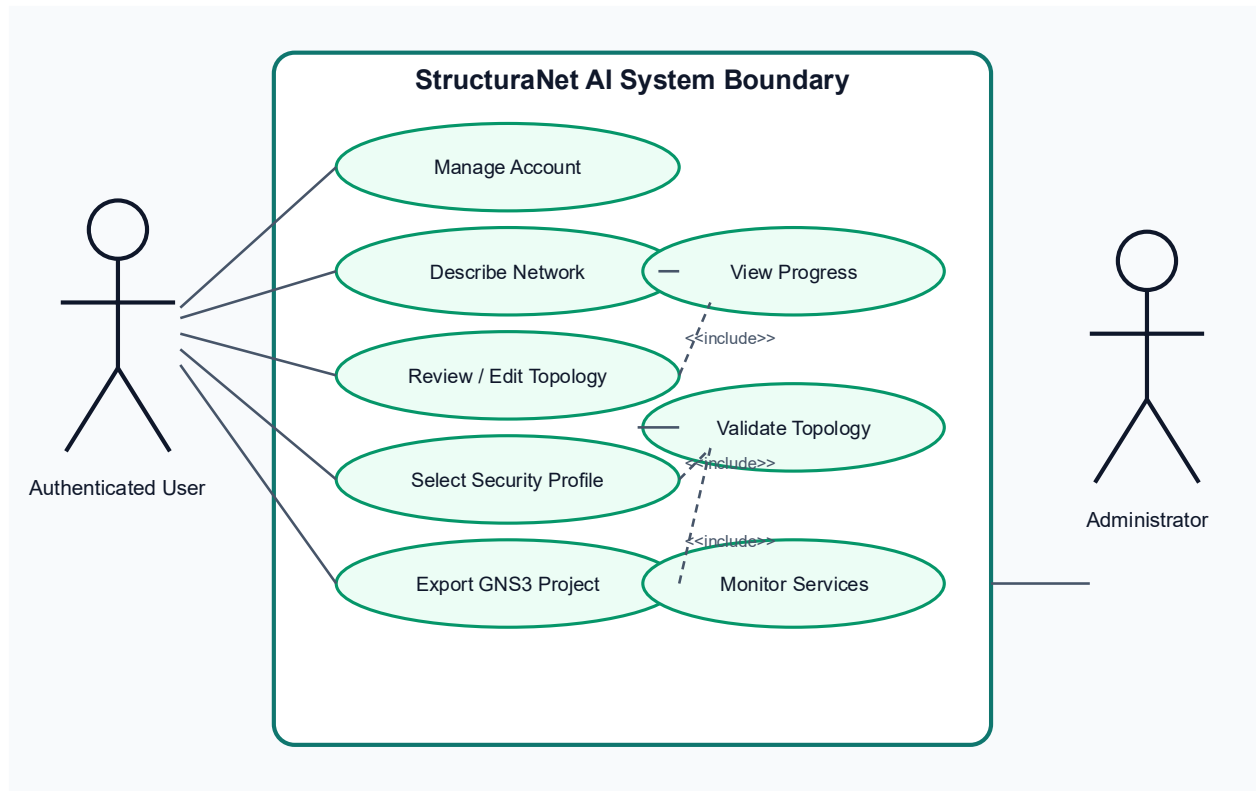


Figure 3.1: StructuraNet AI Use Case Diagram.

3.7 User Interface Design System

The design system of StructuraNet AI is implemented as a centralized configuration within the Tailwind CSS framework, ensuring visual consistency across all pages and components. The system encompasses five primary dimensions: color, typography, spacing, shadows, and animation. Each dimension is defined in the `tailwind.config.js` file and extended with custom values that establish the platform's unique visual identity. [40]-[42]

3.7.1 Color Palette

The color palette is organized into four tiers: brand colors, functional semantic colors, neutral grays, and topology-specific colors. The brand color is an emerald

green (primary hex #079B6B, Tailwind class brand-500) that serves as the visual anchor of the entire application. This color was selected for its association with growth, technology, and reliability, and it provides strong contrast against both the dark chat interface and the dark navy landing page backgrounds. The brand palette spans 11 shades (50 through 950), enabling fine-grained color variations for backgrounds, borders, text, and hover states. The full brand scale ranges from a near-white mint (#E4FFF5 at shade 50) to a near-black forest green (#001F18 at shade 950).

The functional semantic colors follow standard UX conventions: danger (red, based on Tailwind's red scale), warning (amber), and success (mapped to the brand emerald scale, reinforcing brand consistency). Neutral grays use Tailwind's zinc scale, which provides a cooler, more technical appearance compared to the warmer gray scale, aligning with the platform's engineering-oriented identity. The navy color scale (nine shades from #F7FAF9 to #020706) is used exclusively by the landing page and its sub-components, providing a deep, rich background that differentiates the marketing surface from the product interface.

The topology visualization employs a six-color tier system that maps each network device role to a distinct color: amber (#F59E0B) for NAT and Internet nodes at Layer 0, red (#EF4444) for firewalls at Layer 1, blue (#3B82F6) for routers at Layer 2, cyan (#22D3EE) for switches at Layer 3, violet (#8B5CF6) for servers at Layer 4, and slate (#94A3B8) for endpoints at Layer 5. These colors were chosen to maximize visual differentiation between adjacent tiers in the hierarchical layout,

ensuring that users can immediately identify a device’s role from its color alone. The edge lines connecting nodes use a solid teal stroke (#22D3EE on dark mode, #0891B2 on light mode) for high visibility against both background options.

Table 3.1: StructuraNet AI color palette with semantic roles and usage contexts.

Color Name	Hex Value	Semantic Role	Usage Context
Brand (Emerald)	#079B6B	Primary actions, brand identity, accent highlights	CTA buttons, status badges, streaming cursor, glow effects
Navy	#071612	Landing page backgrounds, deep section fills	Hero, HowItWorks, UseCases, StatsStrip backgrounds
Zinc-100	#F4F4F5	Body text (light mode contexts)	General paragraph text
Zinc-950	#09090B	Chat interface base background	ChatLayout, ConversationView base
Zinc-800	#27272A	User message bubbles, sidebar background	Chat bubbles, sidebar drawer
Emerald-400	#34D399	Text gradient, highlights, layer labels	Brand wordmark, text-gradient-brand class
Amber-500	#F59E0B	NAT/Internet nodes in topology	Layer 0 node color
Red-500	#EF4444	Firewall nodes in topology	Layer 1 node color
Blue-500	#3B82F6	Router nodes in topology	Layer 2 node color
Cyan-400	#22D3EE	Switch nodes, edge lines in topology	Layer 3 node color, SVG edges

Color Name	Hex Value	Semantic Role	Usage Context
Violet-500	#8B5CF6	Server nodes in topology	Layer 4 node color
Slate-400	#94A3B8	Endpoint nodes in topology	Layer 5 node color

3.7.2 Typography

The typographic system uses two font families. The primary font is Inter, a geometric sans-serif typeface designed by Rasmus Andersson that is optimized for screen readability at small sizes. Inter is used for all body text, UI labels, navigation items, and button text throughout the application. The font is loaded from the system font stack (defined as `font-sans: [\u2018Inter\u2019, \u2018system-ui\u2019, \u2018apple-system\u2019, \u2018sans-serif\u2019]` in the Tailwind configuration), ensuring fast loading without external font file requests. Inter's tall x-height and open apertures maintain legibility even at the small font sizes (`text-xs`, `text-sm`) used in badges, metadata labels, and sidebar items.

The secondary font is JetBrains Mono, a monospaced typeface designed for code display. It is used exclusively for code blocks rendered in the chat interface (via the Tailwind Typography prose class), inline code references, and technical identifiers. The monospaced font ensures that code snippets, device names, and configuration commands are displayed with proper character alignment, which is essential for readability in the network engineering context where exact syntax matters.

Table 3.2: Typography system — font families and their roles.

Font Family	Role	Used For	Implementation
Inter	Primary (sans)	Body text, UI labels, navigation, buttons	font-sans in Tailwind config
JetBrains Mono	Monospace	Code blocks, inline code, technical references	font-mono in Tailwind config

3.7.3 Spacing, Shadows, and Animation

StructuraNet AI uses Tailwind CSS’s default spacing scale, which is based on a 4-pixel grid (0.25rem increments). Common spacing values observed in the codebase include `py-2.5` (10px vertical padding for buttons and inputs), `px-3.5` (14px horizontal padding for inputs), `gap-2` (8px for icon-text gaps), and `rounded-xl` (12px border radius) for buttons and inputs. Cards use `rounded-2xl` (16px border radius) for a softer, more modern appearance. These consistent spacing values create a visual rhythm that users perceive as polished and cohesive.

The shadow system defines three custom shadow levels: `soft` (0 2px 8px `rgba(0,0,0,0.12)`), `soft-lg` (0 8px 24px `rgba(0,0,0,0.18)`), and `glow-brand` (0 0 24px `rgba(7,155,107,0.32)`). The first two are used for elevation hierarchy (cards, modals, dropdowns), while `glow-brand` creates an emerald halo effect used for focused input states and primary call-to-action emphasis. The card component uses a combination of a subtle inset highlight (0 1px 0 `rgba(255,255,255,0.03)` `inset`) and a deeper drop shadow (0 8px 24px `rgba(0,0,0,0.25)`), creating the illusion of a floating glass panel — a signature visual element of the dark-theme interface.

The animation system includes four custom animations defined in the Tailwind configuration: fade-in (0.5s ease-out, opacity transition), fade-in-up (0.6s ease-out, opacity plus upward translation), fade-in-down (0.6s ease-out, opacity plus downward translation), and pulse-emerald (1.2s ease-in-out infinite, emerald glow pulsation). Additional ambient animations are defined directly in CSS: glowFloatA and glowFloatB (14s and 18s floating motions for the decorative glow orbs in the chat interface), glowPulse (6s pulsation for static glow elements), and pulseEmerald (1.3s blinking for the streaming text cursor). These animations serve functional purposes: the fade-in-up sequence on the landing page creates a progressive disclosure effect, the streaming cursor provides clear feedback that the AI is actively generating, and the ambient glow orbs create a sense of technological sophistication without distracting from the content.

3.8 Landing Page Design

The landing page is the first visual touchpoint for unauthenticated users and serves as the primary marketing surface for StructuraNet AI. It is built as a single-page scrolling experience composed of multiple sections, each designed to communicate a specific aspect of the product's value proposition. The page uses the navy-950 (#071612) background throughout, with emerald accent glows and the grid-dark SVG pattern creating a subtle technological atmosphere.

3.8.1 Header and Navigation

The header component implements a sticky navigation bar that transitions from transparent (over the hero section) to a solid navy background with a bottom border

when the user scrolls past 16 pixels. This effect is achieved through a scroll event listener that toggles a ‘scrolled’ state, applying a conditional background and border. The navigation includes five anchor links (How it works, Use cases, Capabilities, Security, Privacy and Policy) with an animated underline indicator that tracks the currently active section using IntersectionObserver-style scroll tracking (the active section is determined by comparing each section’s bounding rectangle against 35% of the viewport height).

The right side of the header displays contextual call-to-action buttons: ‘Get Started Free’ (or ‘Open Chat’ for authenticated users) as a primary button with the brand emerald gradient, and ‘Log in’ as a ghost button. The brand wordmark (‘StructuraNet’ in white, ‘AI’ in emerald-400) is positioned on the left. This layout prioritizes the conversion action (registration) while keeping the navigation accessible but non-intrusive.

3.8.2 Hero Section

The hero section is the most visually complex component on the landing page, combining multiple layered background effects with a two-column grid layout. The background consists of four layers: the grid-dark SVG pattern (a 40px grid with emerald lines at 4.5% opacity), a radial emerald glow, a positioned circle gradient (840px wide, brand-500 at 14% opacity), and a NetworkMeshBackground component that renders animated green dots connected by thin lines, evoking the appearance of a network topology. These layers create depth without competing with the foreground content.

The left column contains the copy hierarchy: a status badge ('Live · 45 devices · 3 security profiles' with a pulsing green dot), a large headline (text-4xl to text-6xl, bold, tracking-tight, white) with the phrase 'in plain English.' rendered in the text-gradient-brand class (a 90-degree linear gradient from #6EE7B7 through #34D399 and back), a subheadline in navy-300, two CTA buttons (primary 'Get Started Free' and secondary 'Watch Demo'), and trust signals ('No credit card', '45-device catalog', 'GNS3-ready export') with emerald checkmark icons. The right column contains a MockChat component that simulates a chat interaction, providing visitors with an immediate visual demonstration of the product's core functionality. Each element uses staggered fade-in-up animations (0.1s, 0.2s, 0.3s delays) to create a progressive disclosure effect.

3.8.3 How It Works, Stats Strip, and Supporting Sections

The HowItWorks section presents the product workflow in three numbered steps: Describe (tell us what you need in plain English), Review (see it come to life with a visual diagram), and Export (download and run the GNS3 project). Each step is presented as a card with a large step number (01, 02, 03), an icon, a headline, and a body description. The copy is written in outcome-oriented language ('Just describe the network you want' rather than 'Enter a text prompt'), aligning with the design principle of speaking to the user's goals rather than the system's implementation.

The StatsStrip section provides four key metrics in a 2x2 (mobile) or 4-column (desktop) grid: '45+ Device Models', '3 Security Profiles', 'Infinite Network

Designs’, and ‘1-Click GNS3 Export’. Each statistic uses the text-gradient-brand class for the value and includes a label and sublabel. On large screens, vertical dividers separate the columns. Additional landing page sections include UseCases (presenting vertical cards for different user scenarios), SupportedNetworks (listing supported network types and device categories), SecurityFeature (detailing the three security profile tiers), Vision (the product’s long-term vision), FinalCTA (a final conversion section), PrivacyPolicySection, and Footer. All sections follow the same navy-950 background with emerald accent glows, maintaining visual consistency throughout the scrolling experience.

3.9 Authentication Interface

The authentication interface comprises four pages: Login, Register, Forgot Password, and Reset Password. All four pages share a common layout structure provided by the AuthLayout component, which renders a split-screen design. The left panel displays a decorative brand illustration (a stylized network mesh or the StructuraNet AI logo) against a dark background, while the right panel contains the form. This split-screen pattern is a well-established convention in modern web applications that balances visual branding with functional clarity.

The form inputs use the custom .input CSS class, which provides a dark background (rgba(255,255,255,0.03)), a subtle white border (rgba(255,255,255,0.08)), and emerald focus states (border-color transitions to rgba(16,185,129,0.4) with a 3px box-shadow ring at 12% opacity). The primary CTA button uses the .btn-primary class, which applies a vertical gradient from

#10B981 to #0D9668 with an inset white highlight at the top edge and an emerald drop shadow. The active state applies a subtle scale transform (`active:scale-[0.98]`) to provide tactile feedback. Each page includes links to alternative flows (e.g., ‘Forgot password?’ on the login page, ‘Already have an account?’ on the registration page), and error messages are displayed inline near the relevant field in red-400 text. The overall design prioritizes speed and clarity: the forms minimize required input and keep recovery guidance close to the relevant field.

3.10 Chat Interface

The chat interface is the primary workspace of StructuraNet AI, where users spend the majority of their time interacting with the system. It is implemented through the ChatLayout component, which provides the overall page structure, and the ConversationView component, which renders the message stream, input area, and action controls. The design follows a dark-first approach with the zinc-950 (#09090B) background, creating a focused, distraction-free environment that is comfortable for extended use.

3.10.1 Layout Structure and Sidebar

The ChatLayout uses a full-viewport flexbox layout (h-screen) with no top navigation bar. Instead, floating buttons are positioned absolutely in the top-left and top-right corners. The top-left contains a sidebar toggle button (PanelLeft icon, zinc-500, with hover transition to white) and the brand wordmark. The top-right contains a usage indicator (displaying ‘Used designs: X / Y’ in a glass-morphism container with backdrop-blur), a share button, and a new chat button. This minimal

chrome approach maximizes the vertical space available for the conversation, which is critical during long AI generation sequences.

The sidebar is a slide-in drawer that appears from the left when toggled (via the hamburger button or the Ctrl/Cmd+B keyboard shortcut). It uses a zinc-900 background (slightly lighter than the main content, creating depth) and displays two sections: a ‘Starred’ section for bookmarked sessions and a ‘Recent’ section for all other sessions. Each session item shows the title, a relative timestamp, and a context menu (accessible via a three-dot icon) with options for renaming, starring, sharing, and deleting. The sidebar also contains user profile controls and a logout button at the bottom. The drawer can be dismissed by pressing Escape or clicking outside, and it blocks interaction with the main content while open.

3.10.2 Message Rendering and Streaming

User messages are displayed in zinc-800 (#27272A) rounded bubbles with a subtle emerald-tinted border (rgba(16,185,129,0.15)), visually distinguishing them from AI responses. AI responses are rendered using the Tailwind Typography plugin (prose prose-invert classes) at text-16px with relaxed line height, providing comfortable reading for the often-lengthy technical explanations that the AI produces. The prose styling is customized for the dark theme: code blocks use a near-black background (rgba(0,0,0,0.35)) with a white/7 border, inline code uses an emerald-tinted background (rgba(255,255,255,0.06)) with emerald-300 text, links use emerald-400, bold text uses white, and blockquotes use an emerald left border. Markdown tables use white/4 header backgrounds and white/8 borders.

During streaming, a custom CSS cursor (.streaming-cursor) is appended after the streamed text. This cursor is a 6x16 pixel block with an emerald gradient background (#6EE7B7 to #10B981) and a 1.3-second pulsing animation that alternates between 90% and 30% opacity while scaling vertically between 100% and 80%. This cursor provides clear, non-intrusive feedback that the AI is actively generating content, and it disappears as soon as the streaming completes.

3.10.3 Input Area and Action Chips

The input area is positioned at the bottom of the conversation view and features the glow-input-wrap CSS class, which applies a dramatic emerald glow effect when the textarea is focused. The glow consists of four layered box-shadows: a 12px ambient shadow (rgba(0,0,0,0.3)), a 1px emerald ring (rgba(52,245,174,0.22)), a 30px soft emerald glow (rgba(16,185,129,0.34)), and a 70px wide emerald haze (rgba(7,155,107,0.18)). This multi-layered glow creates a spotlight effect that draws the user's attention to the input area, which is the primary interaction point. The textarea auto-resizes based on content (via the useAutoResizeTextarea custom hook) and supports Shift+Enter for newlines and Enter for sending. A microphone button activates voice input via the Web Speech API (useVoiceInput hook), and a send/stop button toggles between the ArrowUp and Square icons based on the streaming state.

The ActionChipsBar component displays four context-aware action chips below the input area when no active conversation exists (the empty state). Each chip is a glass-morphism card with an icon, a label, and an expandable dropdown

of four pre-written prompt suggestions. The four default chip categories are Design (Network icon), Secure (ShieldCheck icon), Modernize (Server icon), and Learn (BookOpen icon). When a topology exists, the chips switch to Export, Secure, Modify, and Learn, reflecting the user's current context. This context-aware adaptation ensures that the suggested actions always match what the user can meaningfully do next, reducing decision fatigue and guiding new users toward productive interactions.

3.10.4 Action Trace and Progress Indication

The ActionTrace component renders a real-time progress indicator that appears during AI pipeline execution. It displays a vertical timeline of steps, each with an icon, a label, and an optional expandable detail section. Step icons are color-coded by status: emerald-300 for running (with a spin animation) and completed states, red-300 for errors, sky-300 for script steps, violet-300 for phase markers, and zinc-300 for generic steps. The timeline uses a vertical line connector between steps (hidden for the last step) and supports collapsible detail sections. This component provides users with transparent visibility into the AI's reasoning and execution process, addressing a common critique of AI systems that appear as 'black boxes' to their users.

3.11 Topology Visualization

The topology visualization system is one of the most technically sophisticated UI components in StructuraNet AI, responsible for rendering an interactive network diagram from the topology data produced by the AI Engine. The system consists of

two components: a `TopologyPreviewCard` (an inline preview embedded in the chat conversation) and a `TopologyFullCanvas` (a full-screen modal viewer with advanced interactions). [24], [25]

3.11.1 Hierarchical Layout Algorithm

The topology layout is computed by the `computeHierarchicalLayout` function in `topologyLayout.js`. The algorithm assigns each node to one of six tiers based on its `node_type`, `template_name`, and name pattern, using a cascading classification system. NAT and Cloud nodes are assigned to Layer 0 (top), firewalls (detected by template names such as `ASAv`, `vSRX`, `FortiGate`, `Palo Alto`, `pfSense`, or name patterns containing ‘fw’ or ‘firewall’) to Layer 1, switches (detected by the `ethernet_switch` type, IOU L2 templates, or name patterns containing ‘sw’ or ‘switch’) to Layer 3, routers (`Dynamips`, IOU, QEMU types, or names starting with ‘R’) to Layer 2, servers (templates containing `Alpine`, `Ubuntu`, `Kali`, `Windows`, or name patterns containing ‘srv’ or ‘server’) to Layer 4, and endpoints (`VPCS` type, names starting with ‘PC’ or ‘host’) to Layer 5 (bottom). Nodes are sorted within each tier by link count in descending order, placing the most-connected devices at the center of each tier. This classification is implemented as a deterministic function with no LLM involvement, guaranteeing consistent layout across multiple generations of the same topology.

3.11.2 Full-Canvas Viewer

The `TopologyFullCanvas` component renders the topology in a full-screen modal using SVG (via `D3.js` selection and zoom modules). Nodes are rendered as

plain SVG rectangles (140x42 pixels) with a role icon, device name, and subtitle text. Edges are drawn as straight lines (no curves, no animation, no gradient) in teal (#22D3EE for dark mode, #0891B2 for light mode). The viewer supports pan and zoom via D3's zoom behavior on the background SVG, and individual nodes are draggable via pointer events. Clicking a node opens a detail panel on the right side, displaying the device's properties, interfaces, and configuration.

The viewer supports light and dark mode toggling (via Sun/Moon icon buttons), with a complete theme object that switches all colors, backgrounds, and border colors between modes. Layer labels are rendered on the left edge of the canvas, color-coded to match the node tier colors, providing an at-a-glance legend. A PDF export function uses the jsPDF library to capture the SVG as a high-quality PDF document with dynamic bounding box calculation. The toolbar also displays keyboard shortcuts (Escape to close, scroll to zoom, drag background to pan) in a subtle kbd-styled badge. The design intentionally avoids unnecessary visual effects: animated dashed edges, gradient strokes, and selection ring animations were deliberately removed during design review to reduce visual noise and improve rendering performance.

3.12 Export and Download Interface

The DownloadKit component appears in the chat conversation after the AI completes the export pipeline. It displays three download cards in a vertical stack, each containing an icon (FileBox, Archive, or FileText from the Lucide icon library), the artifact filename, a human-readable description, and a detail line. The

three artifacts are: the `.gns3project` file (described as ‘GNS3 Project File — Import this into GNS3 to open the full topology’), the `configs.zip` (described as ‘Device Startup Configurations — N device configs — one `.cfg` per device’), and the `requirements.txt` (described as ‘Appliance Requirements Manifest — Checklist of images you must install in GNS3 before importing’). Each card has a Download button that fetches the file via the export API (with JWT authentication passed as a query parameter) and triggers a browser download. After the first download, the button icon changes from Download to Check (via a state toggle), providing clear confirmation that the file has been retrieved. The cards persist after download so the user can re-download later in the same session. [39]

3.13 Onboarding Flow

The `OnboardingModal` is a critical UX component that addresses a fundamental challenge in the GNS3 ecosystem: the requirement that `.gns3project` files reference the exact image filenames installed on the user’s GNS3 server. Without this calibration step, generated projects would fail to import. The modal appears automatically on the user’s first sign-in (when `profile.isCalibrated` is false) and can be re-opened from the Sidebar’s Settings button at any time.

The modal is organized into three numbered sections, following a progressive disclosure pattern. Section 1 presents a dropdown for selecting the GNS3 version (offering versions 2.2.46 through 2.2, with 2.2 as the default catch-all). Section 2 displays four toggle cards in a horizontal row for environment capabilities: QEMU (default enabled), IOU (default disabled), Docker (default disabled), and Strict

Validation (default enabled). These toggles determine which device types the AI Engine can use during topology generation, filtering the 45-device appliance catalog to match the user's GNS3 installation capabilities. Section 3 provides a searchable dropdown listing all devices from the Python appliance catalog. The user selects a device, enters their locally installed image filename in a text input, and clicks an Add button. Mapped devices appear as a list below the search, each showing the device template name and the user's image filename, with a Remove button for corrections.

The modal can be skipped (clicking 'Skip' saves defaults and marks the profile as calibrated), and it uses a scrollable body with pinned header and footer. The search functionality filters the 45-device catalog in real time as the user types, providing instant feedback. This three-section design was chosen over a single long form to reduce cognitive overload: users see only one decision at a time, and the progressive structure naturally guides them from the simplest choice (GNS3 version) to the most involved (device-to-image mapping).

3.14 UI/UX Principles

The design of StructuraNet AI is guided by seven core UI/UX principles that were established at the beginning of the project and consistently applied throughout the design and implementation process.

Dark-First Design. The application uses a dark color scheme as its primary theme, with a zinc-950 base background. This choice was driven by three

considerations. First, the target audience (network engineers and students) often works in low-light environments (server rooms, labs, late-night study sessions), and dark interfaces reduce eye strain in these settings. Second, dark backgrounds provide higher contrast for the emerald accent color and the topology visualization, making critical information more visually prominent. Third, dark interfaces convey a sense of technical sophistication that aligns with the product's engineering-oriented identity. The topology viewer additionally supports a light mode toggle, acknowledging that some users may prefer light backgrounds for specific tasks such as screenshot capture for documentation.

Progressive Disclosure. Information and controls are revealed incrementally, reducing cognitive overload for new users while providing power users with access to advanced features. The landing page uses staggered fade-in animations to introduce content progressively. The onboarding modal breaks the GNS3 calibration process into three numbered sections. The chat interface hides the `ActionChipsBar`, `TopologyPreviewCard`, `DownloadKit`, and `ActionTrace` until they become relevant to the current state of the conversation. This approach ensures that users are never confronted with more information than they need at any given moment.

Transparent AI Progress. One of the primary design goals was to avoid the 'black box' perception that plagues many AI-powered applications. The `ActionTrace` component, the streaming text cursor, the `token_delta` SSE events, and the `tool_start/tool_result` events all work together to provide users with real-time

visibility into what the AI is doing, why it is doing it, and what the intermediate results are. The SSE event system streams nine distinct event types (`tool_start`, `tool_progress`, `tool_result`, `topology_ready`, `deployment_ready`, `token_delta`, `agent_message`, `complete`, and `error`), each designed to keep the user informed and engaged during the potentially long-running generation process.

Context-Aware Adaptation. The interface adapts its controls and suggestions based on the current state of the conversation. The `ActionChipsBar` switches between two sets of chips depending on whether a topology has been generated. The usage counter in the top-right updates in real time. The send button toggles between send and stop icons based on the streaming state. The sidebar shows a streaming indicator on the active session during generation. This context-awareness ensures that the interface always presents the most relevant actions, reducing the user's decision-making burden.

Glass-Morphism and Depth. The visual language of the interface relies on layered transparency effects to create a sense of depth. Cards use semi-transparent backgrounds (`rgba(255,255,255,0.02)` to `rgba(20,22,26,0.75)`) with `backdrop-blur`, creating a frosted-glass effect. Multiple ambient glow orbs (positioned at different depths with different blur radii and animation timings) create a subtle parallax-like depth illusion. The sidebar, dropdown menus, and modals all use similar transparency and blur treatments, creating a consistent visual language where foreground elements appear to float above the background.

Accessibility and Keyboard Navigation. The interface supports keyboard navigation throughout: Ctrl/Cmd+B toggles the sidebar, Enter sends messages, Shift+Enter inserts newlines, Escape closes modals and the sidebar, and scroll-wheel zooms the topology canvas. All interactive elements have aria-label attributes for screen reader compatibility. The sidebar supports full keyboard-driven session management. Color contrast ratios meet WCAG AA standards for all text elements, with emerald accents on dark backgrounds providing particularly strong contrast.

Performance-Conscious Animation. Animations are used sparingly and purposefully. The ambient glow orbs use will-change: transform, opacity to promote GPU-accelerated compositing, and their blur filters are applied only to decorative elements that do not affect layout performance. The streaming cursor animation uses CSS-only transforms (scaleY) to avoid triggering layout recalculation. The topology viewer uses plain SVG rectangles instead of foreignObject + HTML divs for node rendering, which the design review identified as a significant performance improvement for topologies with many nodes. Animations that could interfere with content perception (animated dashed edges, gradient edge strokes, selection ping rings) were deliberately removed after design review to reduce visual noise.

3.15 Chapter Summary

This chapter has documented the graphic design system and user experience architecture of StructuraNet AI. The design system is built on a Tailwind CSS

foundation with a centralized configuration that defines an emerald-green brand palette, an Inter and JetBrains Mono typographic system, a 4-pixel grid spacing scale, a three-level shadow hierarchy, and a purpose-driven animation library. The landing page uses a dark navy background with layered emerald glow effects and progressive disclosure animations to communicate the product's value proposition. The authentication interface follows a split-screen pattern with emerald-accented focus states.

The chat interface, which is the primary workspace, uses a dark zinc-950 background with a minimal floating-button chrome design, glass-morphism message bubbles, a multi-layered emerald glow input area, context-aware action chips, and a real-time action trace component that provides transparent visibility into the AI generation process. The topology visualization system implements a six-tier hierarchical layout algorithm that maps network devices to their functional roles, rendered as an interactive SVG canvas with drag, zoom, pan, light/dark mode, and PDF export capabilities. The onboarding flow uses a three-section progressive disclosure pattern for GNS3 image calibration. Seven core UI/UX principles — dark-first design, progressive disclosure, transparent AI progress, context-aware adaptation, glass-morphism depth, accessibility, and performance-conscious animation — guide all design decisions throughout the platform.



CHAPTER

4

Backend Services, AI Engine, and Topology Generation

System architecture, AI orchestration, validation, configuration
generation, and GNS3 project packaging.

Chapter 4: Backend Services, AI Engine, and Topology Generation

4.1 Introduction

This chapter presents the complete backend architecture and implementation of StructuraNet AI. The backend serves as the central orchestration layer that connects the React-based frontend user interface with the Python-based AI Engine, manages persistent data storage through MongoDB, handles user authentication, and coordinates the real-time streaming of LLM responses via Server-Sent Events. Every design decision documented in this chapter is grounded in the actual source code of the project, with specific file paths, function names, and configuration values drawn directly from the codebase.

The backend is implemented as a monolithic Express.js application (version 4.19) running on Node.js 18 or later, using ECMAScript modules (ESM). It follows a clean separation of concerns organized into five layers: route handlers that define the HTTP interface, middleware for authentication and validation, service modules that contain business logic, data models for MongoDB persistence through Mongoose, and utility modules for cross-cutting concerns such as logging and error handling. The application communicates with a long-running Python worker process via a persistent stdin/stdout JSONL bridge, which avoids repeating interpreter startup and module imports for every request.

The chapter is organized as follows. Section 4.2 provides an overview of the three-tier system architecture and the technology stack. Section 4.3 details the

Express framework configuration and middleware stack. Section 4.4 explains the dual-token JWT authentication strategy and input validation approach. Section 4.5 documents the four MongoDB data models. Section 4.6 presents the complete RESTful API surface across all seven route modules. Section 4.7 describes the Server-Sent Events service for real-time communication. Section 4.8 explains the chat orchestration engine and its hybrid intent routing system. Section 4.9 details the Node.js to Python bridge architecture. Section 4.10 documents the AI Engine pipeline including topology generation, auto-repair, and configuration generation. Section 4.11 covers the GNS3 export pipeline and the eleven-check structural validator. Section 4.12 discusses the testing methodology and results. Finally, Section 4.13 provides a chapter summary.

4.2 System Architecture Overview

4.2.1 Three-Tier Architecture

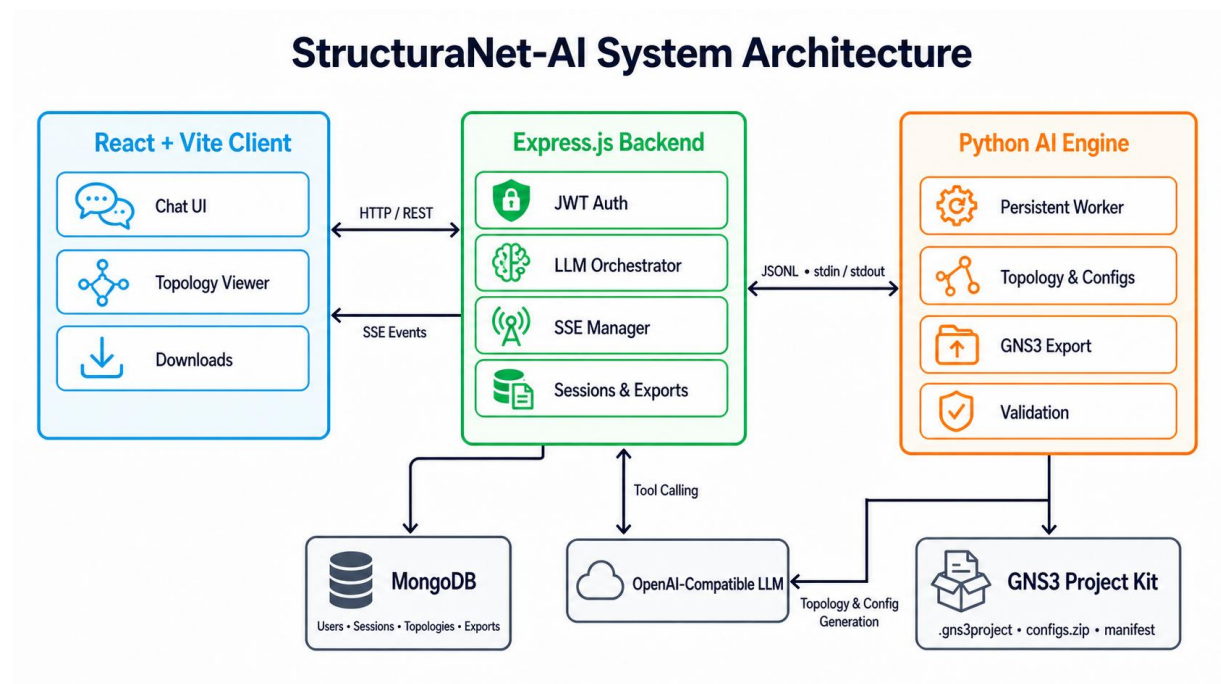


Figure 4.1: StructuraNet AI system architecture

StructuraNet AI employs a classic three-tier architecture that cleanly separates presentation, business logic, and data processing concerns. The first tier is the React frontend, built with Vite 7.3.6 and Tailwind CSS 3.4.10, which runs entirely in the user's browser and communicates with the backend through HTTP requests and Server-Sent Events (SSE). The frontend is responsible for rendering the conversational chat interface, the interactive topology canvas powered by D3.js, and all user-facing interactions including authentication forms, export downloads, and onboarding flows. [38]

The second tier is the Express.js backend (the focus of this chapter), which serves as both the API gateway and the orchestration layer. It handles user

authentication via JWT tokens, validates all incoming requests using Zod schemas, persists chat sessions and topology data in MongoDB, and coordinates the LLM-powered tool-calling loop. The backend maintains a persistent connection to the Python AI Engine through a JSONL-based bridge process, forwarding topology generation, editing, and export commands while streaming progress events back to the frontend through SSE.

The third tier is the Python AI Engine, which encapsulates all LLM interactions, topology synthesis logic, and GNS3 export processing. This tier is implemented as a long-running worker process (`worker.py`) that communicates exclusively through newline-delimited JSON (JSONL) over stdin and stdout. The engine leverages Pydantic v2 models for data validation, the OpenAI Python SDK for LLM API calls, and a comprehensive hardware constants module verified against the GNS3 2.2.59 source code. The deliberate separation of the AI Engine from the Node.js runtime ensures that heavy Python dependencies (such as the OpenAI SDK and catalog data) are loaded exactly once at startup, enabling sub-second response times for subsequent requests.

4.2.2 Technology Stack and Dependencies

The backend relies on fifteen production dependencies, each serving a specific role in the application stack. Table 4.1 provides a complete inventory of these dependencies along with their version constraints and functional purposes. The choice of Express.js as the HTTP framework reflects its maturity, extensive middleware ecosystem, and compatibility with the project's synchronous-to-

asynchronous request patterns. Mongoose 8.24 serves as the Object Data Modeling (ODM) layer for MongoDB, providing schema validation, embedded subdocuments, and compound indexing capabilities that are essential for the session and topology data models.

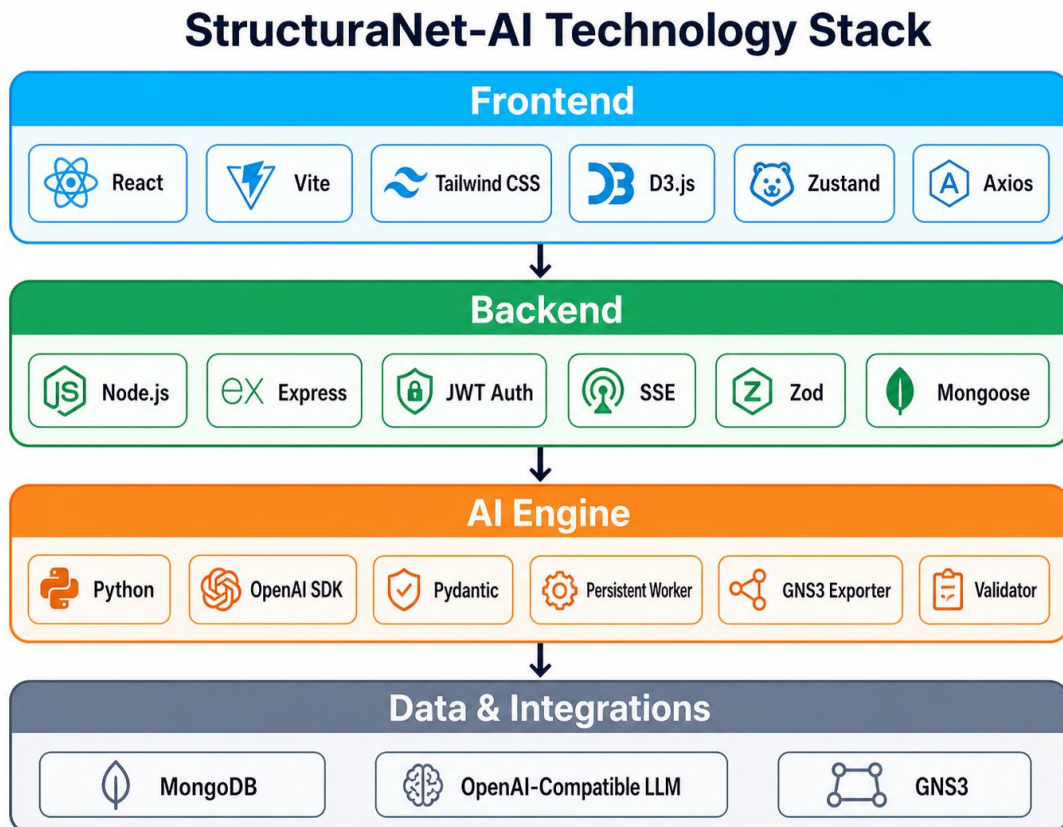


Figure 4.2: StructuraNet AI technology stack

Authentication is handled by jsonwebtoken for JWT operations and bcryptjs for password hashing with twelve rounds of salting. Input validation uses Zod 3.23, which provides TypeScript-style schema definitions with comprehensive error reporting. The OpenAI SDK (version 4.56) is used on the Node.js side for direct LLM API calls within the chat orchestrator, configured to point at OpenRouter as

the default provider. For payment processing, Stripe 22.3 is integrated with a development-mode toggle that bypasses the payment gateway entirely during local testing. The archiver library replaces what would otherwise be an unsafe shell exec call to the system zip utility, ensuring that config file ZIP creation happens entirely within the Node.js process.

Table 4.1: Backend Production Dependencies

Package	Version	Purpose
express	^4.19.2	HTTP framework and routing
mongoose	^8.24.1	MongoDB ODM for schema definition and queries
jsonwebtoken	^9.0.2	JWT access and refresh token issuance and verification
bcryptjs	^2.4.3	Password hashing with bcrypt (12 rounds)
openai	^4.56.0	OpenAI-compatible LLM client (used for OpenRouter)
zod	^3.23.8	Schema validation for all API request inputs
stripe	^22.3.0	Payment processing via Stripe Checkout
archiver	^7.0.1	In-process ZIP creation (replaces unsafe shell zip)
helmet	^7.1.0	HTTP security header injection
cors	^2.8.5	Cross-origin resource sharing configuration
compression	^1.7.4	Gzip response compression
morgan	^1.10.0	HTTP request logging middleware

Package	Version	Purpose
winston	^3.14.2	Structured logging with file rotation
express-rate-limit	^7.4.0	Rate limiting (100 requests per 15 minutes)
dotenv	^16.4.5	Environment variable loading from .env files

4.2.3 Request Lifecycle

Every HTTP request entering the Express backend traverses a well-defined middleware pipeline before reaching a route handler. The pipeline is configured in the `src/app.js` module and executes in the following order. First, the Helmet middleware injects security-related HTTP headers such as Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security. Second, the compression middleware applies gzip encoding to response bodies exceeding one kilobyte. Third, the CORS middleware is configured to accept requests from the frontend URL stored in the `CLIENT_URL` environment variable, with credentials enabled for cookie-based operations.

Fourth, the Stripe webhook endpoint is mounted before the JSON body parser because Stripe requires the raw request body for signature verification. Fifth, `express.json` with a one-megabyte limit and `express.urlencoded` handle standard request parsing. Sixth, Morgan logs all HTTP requests using the ‘combined’ format in production and the colorized ‘dev’ format during development, with output piped to Winston’s HTTP log level. Seventh, `express-rate-limit` enforces a ceiling of one hundred requests per fifteen-minute window per IP address, with a deliberate

exception for SSE stream endpoints to prevent the rate limiter from terminating long-lived connections.

After the middleware stack, the request is matched against one of seven route modules: auth, session, topology, export, voice, payment, and profile. Each route module applies its own authentication and validation middleware before invoking the corresponding service function. If an error occurs at any point in the pipeline, the centralized error handler normalizes it into a structured JSON response with an error code, human-readable message, and optional field-level details for validation errors. The health check endpoint (GET /health) bypasses all authentication and returns a simple status object with the environment name, MongoDB connection status, and a timestamp.

4.3 Backend Framework and Middleware Stack

4.3.1 Express Application Configuration

The Express application is bootstrapped through a two-file pattern that separates server lifecycle management from application configuration. The `src/server.js` module is the entry point responsible for connecting to MongoDB using the URI from the `MONGO` environment variable, with an optional custom DNS server configuration (`MONGO_DNS_SERVERS`) for environments where MongoDB Atlas SRV records require specific DNS resolution. The server module also calls `aiEngine.verifyWrapper()` at boot time to verify that the Python `worker.py` file exists on disk, though the worker process itself is spawned lazily on the first AI request rather than at server startup. [46]

The `src/app.js` module exports a configured Express instance without starting the HTTP listener, which facilitates testability and allows the server module to control the listen call. This module defines the complete middleware chain, mounts all route modules under their respective path prefixes, and registers the two terminal error-handling middleware functions (`notFoundHandler` and `errorHandler`). The separation between app configuration and server lifecycle is a well-established Express pattern that allows the application to be imported and tested without side effects.

4.3.2 Security and Performance Middleware

The security posture of the backend is established through multiple middleware layers working in concert. Helmet provides a baseline of HTTP security headers without requiring manual configuration of individual headers. The CORS configuration restricts cross-origin requests to the frontend's origin only, preventing unauthorized domains from making authenticated API calls. The rate limiter enforces a hundred-request ceiling per fifteen-minute window, with the internal skip function excluding all paths containing `‘/stream’` so that SSE connections are never prematurely terminated by rate-limit enforcement.

Performance optimization is addressed through gzip compression and the persistent Python worker. The compression middleware reduces payload sizes for JSON responses, which is particularly beneficial for the topology endpoint that can return large topology dictionaries with dozens of nodes and links. The persistent worker architecture avoids repeated process startup and module-import work for

topology generation requests, as was the case with the earlier one-shot wrapper.py approach. Environment variable isolation is another security measure: only a minimal set of environment variables (PATH, HOME, PYTHONUNBUFFERED, ROUTER_API_KEY, ROUTER_BASE_URL, AI_MODEL, AI_MAX_TOKENS, and a few others) are forwarded to the Python worker, preventing the full parent process environment (which may contain database credentials, JWT secrets, and Stripe keys) from leaking into the Python subprocess.

4.3.3 Error Handling Architecture

The backend implements a hierarchical error handling system defined in src/utils/errors.js. At the root of the hierarchy is the AppError base class, which extends the native Error class with four additional properties: statusCode (HTTP status code), code (machine-readable error code), details (optional field-level validation errors), and isOperational (a boolean flag distinguishing expected operational errors from unexpected bugs). All seven error classes extend AppError with pre-configured status codes and codes, as shown in Table 4.2.

The centralized errorHandler middleware catches all errors thrown during request processing and normalizes them into a consistent JSON response format. For Mongoose validation errors, it maps them to 400 status codes with field-level details extracted from the error message. For Mongoose CastError (invalid ObjectId format), it returns a 400 response. For duplicate key violations, it returns a 409 Conflict response. For operational AppError instances, it returns the pre-configured status code and structured error payload. For MongoDB connection errors, it returns

a 503 Service Unavailable response with a hint to retry. In production mode, unexpected (non-operational) errors are sanitized to a generic ‘Internal server error’ message to prevent information leakage, while in development mode the full error message and stack trace are included for debugging.

Table 4.2: Error Class Hierarchy

Error Class	Status	Code	Trigger
AppError (base)	500	APP_ERROR	Base class for all operational errors
AuthError	401	AUTH_ERROR	Missing, invalid, or expired authentication token
ForbiddenError	403	FORBIDDEN	Authenticated but lacking required role or ownership
NotFoundError	404	NOT_FOUND	Requested resource does not exist
ValidationError	400	VALIDATION_ERROR	Zod schema validation failure with per-field details
ConflictError	409	CONFLICT	Duplicate key violation (e.g., email already registered)
EngineError	502	ENGINE_ERROR	Python AI Engine failure or timeout
LLMError	502	LLM_ERROR	LLM API call failure (rate limit, timeout, bad response)

4.4 Authentication and Authorization

4.4.1 Dual-Token JWT Strategy

StructuraNet AI employs a dual-token authentication strategy implemented in `src/services/auth.service.js`. The access token is a JWT signed with the `JWT_SECRET` environment variable, carries a payload containing the user's subject identifier (sub), email, and role, and expires after fifteen minutes. The short expiry window minimizes the window of exposure if a token is compromised. The refresh token is a separate JWT signed with a different secret (`JWT_REFRESH_SECRET`), carries only the subject identifier and the user agent string, and expires after seven days. [29]-[31], [43] [30]

Refresh tokens are stored in the database as SHA-256 hashes to enable server-side revocation. Each token record includes the hash, the issuance timestamp, the expiration timestamp, and the user agent string. A pre-save hook on the User model automatically prunes expired tokens from the array, preventing unbounded growth. The login endpoint implements anti-enumeration measures by returning the same generic error message for both 'user not found' and 'wrong password' scenarios, preventing attackers from determining whether an email address is registered. The refresh endpoint implements token rotation: the old token is immediately revoked and a new one is issued. If the old token is not found in the database (indicating possible theft), the system revokes all refresh tokens for that user as a precautionary measure. The password reset flow generates a 32-byte random token, stores its

SHA-256 hash with a 15-minute expiry, and revokes all existing refresh tokens upon successful password change.

4.4.2 Middleware Variants

The authentication middleware (`src/middleware/auth.js`) exports four functions to accommodate different access patterns. The `requireAuth` middleware validates the Authorization header bearing a Bearer token, verifies the JWT signature and expiration, loads the full user document from the database, and attaches it to the request object. If any step fails, it throws an `AuthError` with a 401 status code. The `optionalAuth` middleware performs the same validation but silently continues without a user object if no token is present or if the token is invalid, which is useful for endpoints that provide enhanced functionality to authenticated users while remaining accessible to guests.

The `sseAuth` middleware is specifically designed for Server-Sent Events connections, which use the browser's `EventSource` API that cannot set custom HTTP headers. Instead of reading from the Authorization header, `sseAuth` reads the token from a query parameter (`token`). The `requireRole` factory function accepts a role string and returns middleware that checks whether the authenticated user's role matches the required value, throwing a `ForbiddenError` (403) if not. This four-middleware design ensures that every endpoint can express its exact authentication requirements without duplicating token verification logic.

4.4.3 Input Validation with Zod

All API request inputs are validated through Zod schemas defined in `src/middleware/schemas.js` and applied via the `validate` middleware in `src/middleware/validate.js`. The `validate` function accepts either a single Zod schema (mapped to the request body) or an object with `body`, `params`, and `query` keys, each containing a Zod schema. It calls `safeParse` on the appropriate request location and, on validation failure, throws a `ValidationError` with per-field error details extracted from Zod's flattened error format. On success, it replaces the original request data with the validated and type-coerced result, ensuring that downstream handlers receive clean, correctly-typed data.

The schema module defines six schema groups covering all endpoints. The `authSchemas` group validates registration (email format, password minimum eight characters, name two to eighty characters), login, token refresh, and logout. The `sessionSchemas` group validates session creation, title updates (one to two hundred characters), starred toggle (boolean), and sharing toggle (boolean). The `messageSchemas` group validates message content (one to five thousand characters). The `profileSchemas` group validates GNS3 calibration fields (version string, boolean flags for IOU/QEMU/Docker support, strict validation toggle, image map key-value pairs). The `paymentSchemas` group validates plan selection (plus or pro only, since free plan changes go through a separate downgrade endpoint). The `exportSchemas` group validates security profile selection (none, basic, or enterprise).

4.5 Database Design

4.5.1 Data Models Overview

The database layer consists of four Mongoose models, each mapping to a MongoDB collection and defining the schema, validation rules, indexing strategy, and instance methods for its domain entity. The models are designed to support the conversational workflow: users create sessions, send messages within sessions, generate topologies that are linked to sessions, and export topologies that are tracked as separate jobs. Table 4.3 provides a summary of all four models, their collections, key fields, and indexing strategy. [19], [20]

Table 4.3: Mongoose Data Models Overview

Model	Collection	Key Fields	Indexes
User	users	email, name, passwordHash, role, plan, designUsage, refreshTokens, gns3Profile	email (unique), plan
Session	sessions	userId, title, starred, share, messages[], currentTopologyId, securityProfile, originalRequest	userId, starred, share.token, lastActivityAt
Topology	topologies	sessionId, userId, request, topologyDict, name, nodeCount, linkCount, designReview, assumptions	sessionId+createdAt (compound), userId

Model	Collection	Key Fields	Indexes
Export	exports	sessionId, userId, topologyId, securityProfile, status, files, finalDict, validation, error	sessionId, userId, status

4.5.2 User Model

The User model (`src/models/User.js`) is the authentication and profile entity. Its email field is required, unique, lowercased, trimmed, and validated against a regular expression pattern, with a database index for fast lookup during login. The passwordHash field uses the `select: false` option, which excludes it from all query results by default unless explicitly included, preventing accidental leakage in API responses. The role field is an enum restricted to 'user' and 'admin' values, defaulting to 'user'. The plan field is an enum of 'free', 'plus', and 'pro', also indexed for usage-quota queries.

The designUsage sub-document tracks daily topology generation quotas with two fields: `used` (a counter incremented on each topology confirmation) and `windowStartedAt` (a timestamp marking the start of the current UTC day). The `refreshTokens` array stores hashed refresh token records, each containing the SHA-256 hash, issuance time, expiration time, and the user agent string from the client that requested the token. The `gns3Profile` embedded sub-document captures the user's GNS3 environment configuration, including the GNS3 version (default 2.2), boolean flags for IOU, QEMU, and Docker support, a strict validation toggle, a template-to-image map, and a timestamp for the last profile update. The `toJSON`

transform method strips passwordHash, refreshTokens, and the internal version key from all serialized output.

4.5.3 Session Model

The Session model (src/models/Session.js) represents a chat conversation and is the central entity around which the application's state management revolves. Each session belongs to a user (via the userId ObjectId reference) and contains an embedded messages array. The title field defaults to 'New Chat' and is automatically generated by the autoTitle() instance method, which extracts a summary from the first user message. The starred field enables users to pin important sessions to the top of their session list. The share sub-document tracks whether the session has been published with a unique share token and when sharing was last toggled.

The messages array uses an embedded MessageSchema sub-document, where each message has a role (user, assistant, or system), content string, optional tool name (generate_topology, edit_topology, export_project, or search_kb), optional tool summary, and its own creation timestamp with a unique _id for atomic updates. The session also maintains a currentTopologyId reference to the latest Topology document, a securityProfile field (null, none, basic, or enterprise), and a pendingSecurityProfileAction field for storing deferred actions when a security profile has not yet been selected. Instance methods include autoTitle(), touch() (updates lastActivityAt), enableShare(), and disableShare(). The model has

compound indexes on `userId` and starred for efficient session listing, and on the `share.enabled` and `share.token` fields for public session lookup.

4.5.4 Topology and Export Models

The Topology model (`src/models/Topology.js`) stores the complete topology dictionary produced by the Python AI Engine, along with metadata about the generation request. The `topologyDict` field is a mixed-type field that holds the full GNS3-compatible JSON structure including nodes, links, hardware configurations, and software configurations. The `nodeCount` and `linkCount` fields provide quick summary statistics. The `designReview` and `assumptions` arrays store the AI-generated design analysis and assumptions for display in the frontend. The `phase1File` field stores the server-side path to the Phase 1 JSON output, enabling the export pipeline to re-run Phase 2 with a different security profile without regenerating the entire topology.

The Export model (`src/models/Export.js`) tracks the lifecycle of a GNS3 export job. Its `status` field transitions through four states: `pending` (created but not yet started), `running` (Python engine is generating configs and assembling the ZIP), `complete` (all artifacts are ready for download), and `failed` (an error occurred). The `files` sub-document holds paths to four possible artifacts: `gns3Project` (the `.gns3project` ZIP file), `configsZip` (a separate archive of configuration files), `manifest` (a human-readable text summary), and `allZip` (a combined archive). The `securityProfile` field records which security tier was applied during config generation. The `finalDict` field stores the complete topology dictionary after Phase

2 configuration injection, and the validation field holds the output of the eleven-check structural validator.

Figure 4.3 presents the principal persistence relationships among users, sessions, topologies, and export jobs.

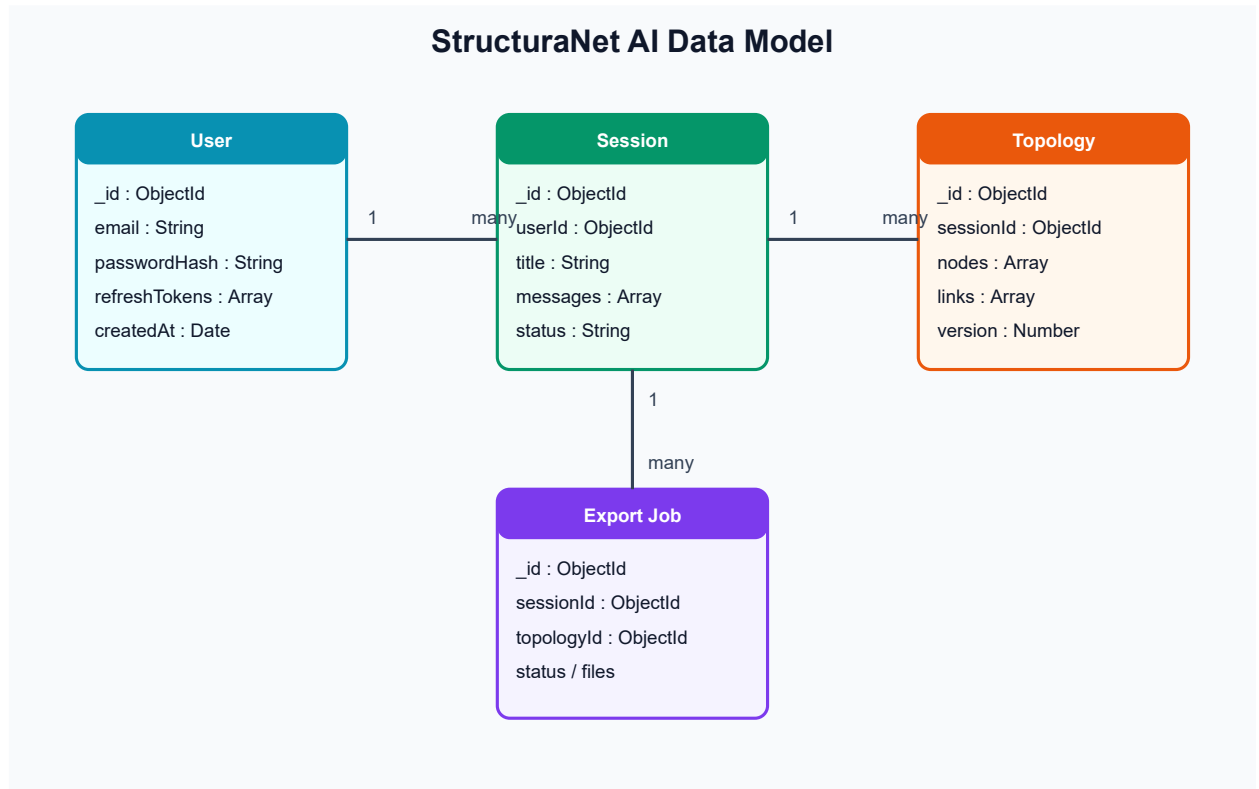


Figure 4.3: StructuraNet AI Database Entity Relationships.

4.5.5 Persistence Boundaries and Lifecycle Integrity

Figure 4.3 should be read as a lifecycle diagram rather than a conventional fully normalized relational schema. MongoDB stores four principal document types. A User owns authentication state, plan and quota state, and the GNS3 calibration profile. A Session belongs to one user and embeds the ordered message history required to reconstruct a conversation. Topology documents provide immutable

design snapshots associated with both the user and session, while Export documents record the asynchronous production state and resulting file locations for a selected topology.

The references in the schema protect ownership boundaries. `Session.userId`, `Topology.userId`, and `Export.userId` allow authorization queries to require both an object identifier and the authenticated user identifier. `Topology.sessionId` and `Export.sessionId` preserve conversational provenance, while `Export.topologyId` identifies the exact design used for Phase 2. The Session document also stores `currentTopologyId` and `currentExportId` as convenience pointers. These reverse pointers reduce lookup cost for the active chat view, but the owning documents remain the authoritative history.

Several fields are deliberately denormalized. `Topology.name`, `nodeCount`, and `linkCount` support list views without repeatedly traversing a mixed topology dictionary. `Session.lastActivityAt` supports recent-session ordering. The trade-off is that service logic must update summary fields consistently when a new topology snapshot is stored. This is acceptable for the prototype because the source topology dictionary remains intact and summary values can be recomputed if a repair task is ever required.

Sensitive and operational values receive different persistence treatment. Password hashes are excluded from ordinary queries, refresh tokens are stored as SHA-256 hashes, and expired refresh-token records are pruned when the user is saved. By

contrast, `phase1File` and export file paths are server-side operational references. They make later edit and download operations efficient, but they also create a deployment obligation: artifact retention, access control, cleanup, and backup policies must remain consistent with the database records.

The document model also supports failure recovery. An Export record moves through pending, running, complete, or failed states and can retain a structured validation result or error message. This allows the API to report a terminal failure without overwriting the last confirmed topology. Similarly, a failed edit can leave `currentTopologyId` pointing to the previous valid snapshot. The persistence boundary therefore participates in safety: transient model or worker failure does not need to corrupt the user's last accepted design.

4.6 RESTful API Design

4.6.1 Authentication Endpoints

The authentication module (`src/routes/auth.routes.js`) defines seven endpoints that handle the complete user authentication lifecycle, from registration through token refresh and logout. All endpoints that accept user input are protected by Zod validation middleware. The register endpoint creates a new user document, hashes the password with `bcrypt` (twelve rounds), and returns both an access token and a refresh token in the response body. The login endpoint implements anti-enumeration by returning identical error responses for non-existent users and incorrect passwords, while also pruning expired refresh tokens from the database before issuing new ones.

The password reset flow is split across two endpoints: forgot-password generates a cryptographic reset token and, in non-production environments, returns the reset URL directly in the response for convenience during development. The reset-password endpoint verifies the token, checks its 15-minute expiration window, updates the password hash, clears the reset fields, and revokes all existing refresh tokens as a security precaution. The refresh endpoint implements token rotation with theft detection: if the presented refresh token is not found in the database, all tokens for that user are revoked under the assumption that the token list may have been compromised. Table 4.4 provides the complete specification of all authentication endpoints.

Table 4.4: Authentication API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Create account; returns access and refresh tokens
POST	/api/auth/login	None	Authenticate with email/password; issues dual tokens
POST	/api/auth/forgot-password	None	Request password reset; 32-byte token, 15-min expiry
POST	/api/auth/reset-password	None	Reset password; revokes all refresh tokens
POST	/api/auth/refresh	None	Rotate refresh token; old token revoked, new one issued
GET	/api/auth/me	Required	Retrieve authenticated user profile with usage stats
POST	/api/auth/logout	Required	Revoke specific refresh token

4.6.2 Session and Message Endpoints

The session module (`src/routes/session.routes.js`) is the most complex route file, handling ten endpoints that cover the full lifecycle of a chat session. The `GET /api/sessions` endpoint returns a list of the authenticated user's sessions, sorted by starred status first and then by last activity timestamp, with a limit of one hundred sessions. The `POST /api/sessions` endpoint creates a new empty session and returns both the session ID and the full session object. The `GET /api/sessions/:id` endpoint performs an ownership check and returns the complete session document including its embedded messages array, plus any associated topology and export job for rendering in the chat interface.

The message-sending endpoint (`POST /api/sessions/:id/messages`) implements a fire-and-forget pattern: it validates the message content, checks whether the message constitutes a topology confirmation (which triggers daily usage quota consumption), and dispatches the message to the chat orchestrator without awaiting its completion. The endpoint returns a 202 Accepted response immediately with a pending `messageId`, while the actual LLM processing happens asynchronously. Progress and results are streamed to the client through the SSE endpoint (`GET /api/sessions/:id/stream`), which uses the `sseAuth` middleware to authenticate via query parameter. The sharing endpoint (`GET /api/sessions/share/:token`) is the only public session endpoint, allowing unauthenticated access to shared conversations via a unique token. Table 4.5 documents the complete session API surface.

Table 4.5: Session and Message API Endpoints

Method	Endpoint	Auth	Description
GET	/api/sessions	Required	List user sessions sorted by starred and last activity (limit 100)
POST	/api/sessions	Required	Create new chat session; returns session ID and object
GET	/api/sessions/:id	Required	Retrieve full session with topology and export job
GET	/api/sessions/share/:token	Public	Fetch shared chat session without authentication
PATCH	/api/sessions/:id/title	Required	Update session title (1-200 characters)
PATCH	/api/sessions/:id/star	Required	Toggle starred status for session pinning
PATCH	/api/sessions/:id/share	Required	Enable or disable session sharing via unique token
DELETE	/api/sessions/:id	Required	Delete session and cascade-delete associated topology
GET	/api/sessions/:id/stream	SSE Auth	Subscribe to real-time SSE events for the session
POST	/api/sessions/:id/messages	Required	Send message; returns 202, dispatches to orchestrator

4.6.3 Topology, Export, and Supporting Endpoints

The remaining four route modules cover topology retrieval, export management, voice transcription, payment processing, and user profile management. The topology endpoint (GET /api/topology/:id) fetches the topology dictionary for canvas rendering, performing an ownership check to prevent users from accessing other users' topologies. The export module provides status tracking (GET /api/export/:id/status) and artifact downloads (GET /api/export/:id/download/:file), where the download endpoint uses the sseAuth

middleware and supports three artifact types: the GNS3 project file (served as `application/gns3project`), a configs ZIP archive (generated in-process using the archiver library rather than spawning a shell `zip` command), and a plain-text manifest file.

The voice endpoint (`POST /api/voice/transcribe`) accepts raw audio data up to ten megabytes and supports automatic format detection from the Content-Type header (MP4, WAV, OGG, and WebM). It implements two transcription paths: an OpenRouter multimodal path for providers that support audio input, and a standard OpenAI Whisper path for providers that do not. The payment module integrates Stripe Checkout for plan upgrades with a development-mode toggle (`PAYMENTS_DEV_MODE` environment variable) that bypasses Stripe entirely and directly applies the selected plan. The profile module manages the user's GNS3 calibration settings, daily usage statistics, and provides a catalog endpoint that proxies the 45-device appliance catalog from the Python AI Engine. Table 4.6 summarizes all supporting endpoints.

Table 4.6: Supporting API Endpoints

Module	Method	Endpoint	Description
Topology	GET	<code>/api/topology/:id</code>	Fetch topology dict for canvas rendering with ownership check
Export	GET	<code>/api/export/:id/status</code>	Get export job status, files, validation, or error

Module	Method	Endpoint	Description
Export	GET	/api/export/:id/download/:file	Download GNS3 project, configs ZIP, or manifest file
Voice	POST	/api/voice/transcribe	Transcribe audio (mp4, wav, ogg, webm) via OpenRouter or Whisper
Payment	POST	/api/payments/checkout	Create Stripe checkout or apply plan in dev mode
Payment	POST	/api/payments/confirm	Confirm Stripe checkout and apply plan upgrade
Payment	POST	/api/payments/webhook	Handle Stripe webhook events (raw body verification)
Profile	GET	/api/profile	Retrieve GNS3 calibration profile
Profile	GET	/api/profile/usage	Get daily usage statistics (plan, used, remaining)
Profile	PUT	/api/profile	Save GNS3 environment settings and image map
Profile	GET	/api/profile/catalog	Fetch 45-device appliance catalog from Python AI Engine

4.6.4 API Contracts, Ownership Checks, and Error Semantics

Tables 4.4 through 4.6 describe the route surface; their more important architectural role is to define trust boundaries. Authentication routes accept credentials or refresh material and return identity state. Session routes operate on conversation ownership and message creation. Topology and export routes expose generated artifacts only after the service has associated them with the

authenticated user. The API therefore does not treat possession of a MongoDB identifier as authorization.

Request validation occurs before orchestration. Zod schemas reject malformed bodies for registration, login, profile updates, title changes, sharing controls, and message creation. Route parameters still require service-level existence and ownership checks because syntactic validity cannot prove that an object belongs to the caller. This two-stage approach separates input-shape errors from authorization and domain errors, producing clearer responses and reducing unnecessary database or AI-engine work.

Long-running work is represented through resource state rather than a single blocking response. Posting a message can initiate topology generation, while the session stream carries progress events. Export creation establishes an Export record whose status can be polled, and completed artifacts are downloaded through a file-specific route. This design allows generation and export timeouts to be longer than ordinary HTTP request expectations without making the browser interface appear frozen.

Error semantics are normalized by the application error handler. Validation failures, authentication failures, missing resources, rate limits, worker errors, provider errors, and export failures are converted to a consistent response shape with a machine-readable code and human-readable message. Internal stack traces and secrets are not intended for client responses. At the same time, server logs

preserve enough classification and request context for diagnosis. This distinction is essential when a provider error contains configuration details that should not be reflected to a browser.

The SSE endpoint requires separate consideration because browser EventSource clients do not support the same custom-header workflow as ordinary Axios requests in every deployment. The project therefore uses an SSE-aware authentication path and excludes stream routes from the general API rate limiter. The exception is narrow: it exists to preserve a long-lived event channel and does not remove ownership checks on the session. Reverse proxies must also disable buffering and use an idle timeout compatible with the fifteen-minute generation and twenty-minute export defaults.

Together, the endpoint tables and these contract rules reveal a useful separation. REST routes establish or retrieve durable resources; SSE communicates transient progress; MongoDB stores recoverable state; and the worker bridge performs compute-intensive commands. A failure in one channel can therefore be represented in another—for example, a worker error becomes an SSE event and a failed Export state—without conflating transport, persistence, and execution responsibilities.

4.7 Real-Time Communication via Server-Sent Events

4.7.1 SSE Service Architecture

The Server-Sent Events service (`src/services/sse.service.js`) is implemented as a singleton class that manages per-session client subscriptions. It maintains two in-memory Map structures: a clients map that associates each session ID with a Set of HTTP Response objects, and a heartbeats map that tracks the 30-second interval timers used to send keepalive comments to prevent connection timeouts. The subscribe method sets the required SSE headers (Content-Type: text/event-stream, Cache-Control: no-cache, X-Accel-Buffering: no for Nginx proxy compatibility), writes an initial comment (':ok') to confirm connection establishment, adds the response to the client set, and starts the heartbeat interval if this is the first client for the session. [21]

The unsubscribe method removes a specific client from the session's set, stops the heartbeat interval if no clients remain, and ends the HTTP response. The broadcast method iterates over all clients subscribed to a given session and writes a formatted SSE event (event: <name> followed by data: <JSON> followed by a blank line) to each one. This architecture supports multiple browser tabs viewing the same session simultaneously, as each tab maintains its own SSE connection and receives all events independently. The hasListeners method provides a quick check for whether any clients are subscribed to a session, which the chat orchestrator uses to avoid unnecessary work when no one is listening.

4.7.2 Event Types and Data Flow

The SSE system defines twelve distinct event types that cover the full lifecycle of a conversational interaction, from token streaming through tool execution to final delivery of topology and export artifacts. The `token_delta` event carries individual text tokens as they are generated by the LLM, with emoji characters stripped via a comprehensive Unicode regex to prevent rendering issues in the frontend. The `agent_message` event delivers the complete assistant response as a single payload, useful for clients that reconnect mid-stream. The `tool_start` and `tool_progress` events provide real-time visibility into the AI Engine's processing stages, including phase transitions and thought process outputs.

The `topology_ready` event carries the complete topology dictionary along with node and link counts, triggering the frontend to render or update the interactive topology canvas. The `deployment_ready` event signals that the GNS3 export pipeline has completed and lists the available download artifacts. The `usage_update` event informs the frontend of quota consumption after a topology is confirmed. The `error` event delivers structured error information with a message and machine-readable code. The `complete` event signals the end of an orchestration turn, allowing the frontend to re-enable input controls. Table 4.7 provides a complete reference of all SSE event types, their directions, payload structures, and descriptions.

Table 4.7: Server-Sent Events (SSE) Event Types

Event Name	Direction	Payload	Description
token_delta	Server	{ text: string }	Streaming LLM text token (emoji-stripped)
agent_message	Server	{ content: string }	Complete assistant text response
tool_start	Server	{ tool: string }	Tool execution has begun
tool_progress	Server	{ phase, message }	Progress update from Python AI Engine
tool_result	Server	{ tool, success, data }	Tool execution result (success or failure)
topology_ready	Server	{ topology, nodeCount }	New or updated topology for rendering
deployment_ready	Server	{ files: string[] }	Export complete; artifacts available for download
usage_update	Server	{ plan, used, remaining }	Daily usage quota consumed
security_profile_selected	Server	{ profile: string }	User selected a security profile tier
error	Server	{ message, code }	Error occurred during processing
complete	Server	{ }	Orchestration turn finished

Event Name	Direction	Payload	Description
keepalive	Server	{ }	Connection keepalive (every 30 seconds)

4.8 Chat Orchestration and LLM Integration

4.8.1 Hybrid Intent Routing

The chat orchestrator (`src/services/chat.orchestrator.js`, approximately 1208 lines) is the central intelligence layer of the backend, implementing a hybrid routing architecture that optimizes response latency and cost by selecting the appropriate processing path based on the user's intent. The system begins with a heuristic intent classifier (`classifyIntent`) that analyzes the user's message using a set of pattern-matching rules without making any LLM API call. The classifier categorizes messages into two broad intents: 'chat' (greetings, identity questions, meta queries, and knowledge questions) and 'tool' (requests to build, modify, or export topologies). The classifier is deliberately conservative: any message that does not clearly match a chat pattern is routed to the tool path, ensuring that design requests are never misclassified as casual conversation.

Before reaching the LLM, the system checks for deterministic actions via `getDeterministicAction`, which short-circuits the LLM entirely for common patterns. Simple greetings receive canned responses. Explicit requests to build, edit, or export topologies are dispatched directly to the appropriate tool without LLM mediation. Requests to 'show me the topology' replay the existing topology from the database via SSE without invoking the AI Engine. If the user requests an export

but no topology exists, the system responds with a message prompting the user to generate a topology first. This three-tier routing system (deterministic actions, heuristic classification, and LLM-powered tool use) ensures that the majority of user interactions receive fast, predictable responses while preserving full LLM intelligence for complex design requests.

4.8.2 Tool-Use Loop

When the heuristic classifier routes a message to the tool path, the orchestrator enters a multi-round tool-use loop that can execute up to six rounds (`MAX_ROUNDS = 6`) per user turn. Each round follows a consistent cycle: the system sends the accumulated chat history (including prior tool results) to the LLM with the full tool definitions (`generate_topology`, `edit_topology`, `export_project`) and `tool_choice` set to 'auto'. As the LLM streams its response, `token_delta` events are emitted to the frontend in real time, with emoji characters stripped by a comprehensive Unicode regex that specifically targets emoji code points without collapsing whitespace or truncating tokens.

If the LLM's response includes tool call arguments, the orchestrator accumulates them (since they may arrive across multiple streaming chunks) and, after the response is complete, checks whether a security profile has been selected for the session. If no security profile exists and the tool requires one, the system broadcasts a `security_profile_selected` event and stores the pending action for resumption after the user makes a selection. Otherwise, it executes the tool by loading the user's GNS3 calibration profile from the database, building a

PreflightProfile dictionary, and calling the appropriate AI Engine Bridge method (generate, edit, or exportProject). The tool result is appended to the chat history as a tool message, and the loop continues. This iterative process allows the LLM to chain multiple tool calls within a single user turn, for example generating a topology and then immediately exporting it with security configurations.

The chat fast-path, activated when the intent classifier determines a message is conversational, uses a deliberately minimal LLM configuration: tool_choice is set to 'none', no tool definitions are included, max_tokens is capped at 800, and a lean system prompt is used. This configuration typically produces responses in one to two seconds, compared to five to twenty seconds for the tool path, because the LLM skips tool evaluation entirely and generates only a short conversational reply.

Figure 4.4 traces a Phase 1 design request across the React client, Express API, orchestrator, persistent Python worker, MongoDB, and SSE response path.

Figure 4.4: Chat Orchestration and Topology Generation Sequence.

Figure 4.4 makes two asynchronous boundaries explicit. The first lies between the browser and the backend, where the message request and SSE stream are separate HTTP interactions. The second lies between Node.js and Python, where a request identifier correlates a command with JSONL events and its terminal response. Correlation is necessary because stdout is a shared stream and because progress events may arrive before the final topology dictionary.

Persistence occurs after deterministic checks have produced a usable topology snapshot. This ordering prevents a partial LLM response from becoming the active session topology. The backend can still persist the user message and an explanatory error, but `currentTopologyId` is updated only when an accepted

topology exists. The sequence therefore combines conversational continuity with artifact integrity.

The diagram also clarifies why SSE is not the source of truth. Events improve responsiveness but may be interrupted by navigation or network loss. The durable Session, Topology, and Export documents allow the client to reload state after reconnection. A robust client treats SSE as an incremental view of work and confirms the final resource through the ordinary API or persisted session response.

4.8.3 Security Profile Gating

Before any topology generation or export operation, the orchestrator checks whether the session has an assigned security profile. If the security profile is null, the system broadcasts an SSE event prompting the user to select one of three tiers: None, Basic, or Enterprise. The user's response is parsed by the `parseSecurityProfileSelection` function, which can also infer the profile from security-related keywords in the user's message (for example, mentioning 'firewall', 'DMZ', 'zero-trust', or 'ACL' automatically selects the Enterprise profile). The pending action (generate or export) is stored in the session's `pendingSecurityProfileAction` field and is automatically resumed once the profile is set.

The security profile selection affects both phases of the AI pipeline. In Phase 1 (topology generation), the selected tier is injected into the LLM system prompt by the Python `security_prompts` module, which adds tier-specific network design

rules. The None tier adds no security constraints. The Basic tier requires SSH version 2, AAA local authentication, service timestamps, login block-for, NTP, Syslog, and disables SNMP community strings. The Enterprise tier includes all Basic requirements plus Zone-Based Firewall (ZBF), anti-spoofing ACLs, TCP intercept, NAT PAT overload, OSPF MD5 authentication, HSRP, SNMPv3 with authentication and privacy, DHCP snooping, Dynamic ARP Inspection, STP BPDU guard, port security, and unicast Reverse Path Forwarding. In Phase 2 (configuration generation), the same tier determines the depth of Cisco IOS commands injected into each device's startup configuration.

4.9 Node.js–Python Bridge

4.9.1 Persistent Worker Architecture

The AI Engine Bridge (`src/services/ai-engine.bridge.js`) manages a persistent Python worker process that replaces the earlier approach of spawning a new Python process for each request. The worker is implemented as `worker.py` in the `ai-engine` directory and communicates with the Node.js parent through a bidirectional `stdin/stdout` pipe using newline-delimited JSON (JSONL). This architecture provides two critical performance advantages: first, heavy Python modules (the OpenAI SDK, the 45-device appliance catalog, hardware constants) are loaded exactly once at worker startup rather than on every request, avoiding repeated interpreter and module-import work; second, the LLM client connection is established once and reused across all subsequent requests. [33], [34]

The worker is spawned lazily on the first AI request rather than at server boot. During initialization, the worker eagerly imports all required modules and then emits a 'ready' event through stdout, signaling that it is prepared to accept commands. The bridge waits up to thirty seconds for this ready event before timing out. Once ready, the worker can process seven distinct commands: generate (Phase 1 topology generation with a fifteen-minute timeout), edit (Phase 1 topology modification with a fifteen-minute timeout), export (Phase 2 configuration generation and GNS3 export with a twenty-minute timeout), qa (Cisco IOS knowledge-base queries with a fifteen-minute timeout), validate (topology JSON validation with a sixty-second timeout), catalog (appliance catalog retrieval with a thirty-second timeout), and brief (configuration brief generation). A ping command is also supported for health checks, and a shutdown command enables graceful termination.

4.9.2 JSONL Protocol

The communication protocol between Node.js and Python follows a strict JSONL (newline-delimited JSON) format with three envelope types. The request envelope, sent from Node.js to the worker via stdin, contains three fields: id (a UUID v4 generated by `crypto.randomUUID()` for request-response correlation), command (the operation name), and args (a command-specific object containing all parameters). The event envelope, sent from the worker to Node.js via stdout during processing, contains the request id, an event name, and a data payload. The response envelope, sent when the command completes, contains the request id, a boolean ok

field, and either a result or error object. Each envelope is terminated by a single newline character, making the protocol simple to parse and debug.

The worker processes requests sequentially by design: only one command is executed at a time, and concurrent requests queue in the stdin buffer. This deliberate sequentialism prevents shared-state bugs that could arise from concurrent topology generation or export operations accessing the same in-memory data structures. Each request is matched to its response by the UUID in the id field, a design that supports future multi-worker pooling by enabling the bridge to route responses to the correct pending promise regardless of which worker produced them. Intermediate events (such as progress updates, thinking text, and phase changes) are also correlated by request ID and forwarded to the original caller's event callback.

4.9.3 Error Recovery and Lifecycle Management

The bridge implements a three-tier error recovery strategy. At the command level, if the worker encounters an error while processing a request but remains alive, it sends an error response envelope (ok: false) with details, and the bridge rejects the corresponding promise while keeping the worker alive for subsequent requests. At the process level, if the worker crashes unexpectedly (detected by the 'exit' event on the child process), the bridge immediately rejects all pending promises with an engine error and resets its internal state so that the next request will spawn a fresh worker. At the spawn level, if the initial worker process fails to start (for example, due to a missing Python binary or a syntax error in the code), the bridge rejects the current request but will attempt to spawn a new worker on the next request.

Per-request timeouts are enforced independently of the worker's health. If a command exceeds its configured timeout (fifteen minutes for generation and editing, twenty minutes for export), the bridge rejects the promise with a timeout error while leaving the worker alive, since the worker may still be processing and could complete after the timeout. The worker's stdout is captured at module load time using a reference to the real stdout file descriptor, and sys.stdout is redirected to a StringIO buffer during command execution to prevent stray print statements (particularly from the GNS3 exporter) from corrupting the JSONL protocol. All JSONL writes to the real stdout are protected by a threading lock to prevent interleaving in multi-threaded scenarios.

4.10 AI Engine: Topology Generation Pipeline

4.10.1 Phase 1: LLM-Driven Topology Synthesis

The Phase 1 pipeline (implemented in `structranet/ai/agent.py`, version 5.0) transforms a natural-language network description into a validated `GNS3Project` data structure. The pipeline begins with a deterministic shortcut check: if the user's request matches the structured pattern of 'N branches, M rooms, P people' (handled by `_try_structured_branch_room_topology`), the system constructs the topology programmatically without any LLM involvement, avoiding model-generated variation and a provider call for that recognized pattern. For all other requests, the system enters the LLM retry loop, which allows up to three attempts (`MAX_RETRIES = 3`) to produce a valid topology. [47]

Each LLM attempt follows a structured sequence. First, the system builds a comprehensive prompt (`_build_step1_prompt`) that includes the filtered hardware inventory (limited to devices supported by the user's GNS3 environment), per-device link limits (derived from `DYNAMIPS_MAX_LINKS` and `SINGLE_LINK_TYPES`), a mandatory six-step build algorithm (identify devices, place single-port devices, create connectivity backbone, attach distribution and access switches, attach end devices, verify connectivity), twelve mandatory rules (including zero hallucination of non-existent devices, no manual port number assignment, fully connected graph requirement, single-port device constraints, and no parallel links), five forbidden patterns, and four reference wiring patterns (single-site enterprise, multi-branch, Router-on-a-Stick, and NAT intermediary). The LLM is instructed to output a JSON object containing a thinking field (chain-of-thought reasoning) and a topology field with nodes and connections arrays.

After the LLM produces a response, the JSON is extracted (stripping any `thought_process` XML tags and markdown fences), parsed into a `TopologyRequest` Pydantic model, and then passed through the auto-repair pipeline before Pydantic validation. If validation fails, the error message is appended to the chat history and the loop retries with the LLM receiving the error feedback. On success, the `TopologyRequest` undergoes deterministic port assignment (handled by the `port_assigner` module) which converts logical connections into `Link` objects with correct `adapter_number` and `port_number` values, followed by a second Pydantic validation of the resulting `GNS3Project` model. The `process_and_save_topology`

function then injects hardware configurations (expanding Dynamips slots, IOU adapters, and QEMU properties), enriches nodes with metadata (interface lists, hardware summaries, security roles, VLAN assignments), applies VLAN trunk and access port patches, and saves the final topology to disk.

4.10.2 Auto-Repair Pipeline

The auto-repair pipeline executes three sequential repair functions on the raw LLM output before Pydantic validation, addressing the most common categories of LLM errors in topology generation. The first function, `_repair_duplicate_connections`, scans the connections array and removes any duplicate links where the same pair of nodes is connected by the same link type, keeping only the first occurrence. This addresses the LLM's tendency to redundantly specify the same connection multiple times in complex topology descriptions. [28]

The second function, `_repair_single_port_violations`, handles devices that have only one network interface (VPCS, TraceNG, and NAT nodes) but are connected to multiple peers. For each single-port device with more than one connection, the system preserves the first connection as-is and inserts an intermediary Ethernet Switch (named `Inter-SW-{node_id}`) to fan out the remaining connections. This ensures that the physical constraint of one port per device is respected while maintaining the logical connectivity the LLM intended.

The third function, `_repair_disconnected_graph`, uses a Union-Find data structure to detect isolated groups of nodes. If the topology contains more than one

connected component, the system creates a Bridge-SW{N} switch and connects one node from each disconnected group to it. When selecting which node to use as the connection point for each group, the system prefers router-type endpoints first, then switch-type endpoints, and finally any node type. This ensures that every generated topology is a fully connected graph, which is a fundamental requirement for GNS3 import.

4.10.3 Phase 2: Software Configuration Generation

The Phase 2 pipeline (implemented in `structranet/ai/config_agent.py`) generates Cisco IOS software configurations for every device in the topology and merges them into the Phase 1 output. The pipeline begins by loading the Phase 1 JSON from disk and applying VLAN patches if they have not already been applied (checked via the `__vlan_patched__` sentinel key). Like Phase 1, Phase 2 includes a deterministic shortcut for branch-room topologies that generates standard configurations without LLM involvement. [37]

For all other topologies, the system first builds a Configuration Brief (via `context_builder.py`), which transforms the hardware-injected topology dictionary into a human-readable text description. The brief identifies core switches, builds segment definitions (access switches grouped with their VLANs), and resolves all interface names for each node. This brief is then passed to the LLM along with seven generalized Layer 3 architecture rules: one subnet per broadcast domain (Rule A), Router-on-a-Stick with 802.1Q sub-interfaces (Rule B), VPCS gateway matching (Rule C), structured subnet allocation using VLAN ID as the third octet

(Rule D), simple network exception for flat single-switch topologies (Rule E), /30 WAN subnets for inter-branch links (Rule F), and NO SHUTDOWN on all interfaces (Rule G). The LLM outputs a dictionary mapping node IDs to their configuration content, using JSON mode first with a plain-text fallback.

4.10.4 Three-Gate Safe Merge

The three-gate safe merge (`safe_merge_configs`) is the critical mechanism that integrates LLM-generated configurations into the Phase 1 topology dictionary without corrupting existing data. The function accepts two inputs: the Phase 1 topology dictionary (which contains hardware-injected properties, enriched metadata, and VLAN patches) and the LLM-generated configuration dictionary. The merge operates node-by-node, applying three sequential gates to every configuration key-value pair the LLM produces.

Gate 1 (Whitelist) accepts only keys in the `SOFTWARE_CONFIG_KEYS` frozen set: `startup_config_content`, `private_config_content`, `startup_script`, `start_command`, and `environment`. Any key not in this set is silently discarded, along with any key starting with an underscore (which is reserved for internal metadata). Gate 2 (No-Overwrite) checks whether the target key already has a non-empty value in the Phase 1 dictionary. If it does, the value is preserved and the LLM's value is discarded. However, empty placeholder values (empty strings, empty objects, empty arrays, and `None`) are treated as absent and are overwritten. Gate 3 (Type Check) verifies that the LLM-provided value matches the expected Python type for the key: `startup_config_content`, `private_config_content`, and

startup_script must be strings, while environment can be either a dictionary or a string. If any gate rejects a key-value pair, it is logged and skipped. This three-gate design ensures that the LLM can only modify configuration-specific fields, cannot corrupt hardware properties or metadata, and cannot change values that were already set during Phase 1.

Figure 4.5 shows how identity checking, hardware-field protection, and structural validation prevent generated software configuration from corrupting deterministic topology data.

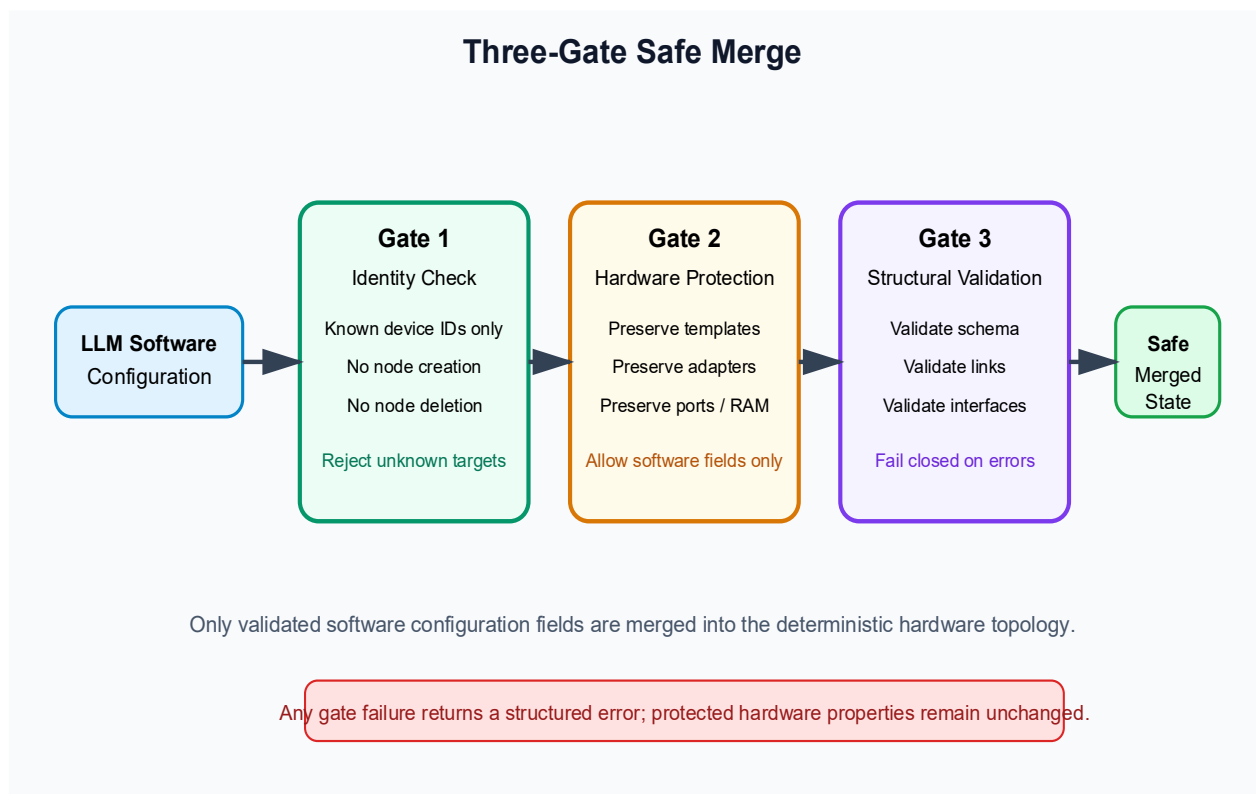


Figure 4.5: Three-Gate Safe Merge Flow.

Figure 4.5 summarizes a defense-in-depth merge rather than a generic object merge. Gate 1 requires node identity agreement so that configuration cannot be

attached to an unknown or renamed device. Gate 2 protects hardware-owned fields such as platform, slots, adapters, image references, compute identifiers, and deterministic interface assignments. Gate 3 validates the combined topology after software configuration has been introduced.

This ordering limits the authority delegated to the model. Phase 2 may propose hostnames, interface commands, routing statements, access-control lists, and other software policy, but it cannot silently replace the physical realization established in Phase 1. A rejected merge leaves the Phase 1 topology available for diagnosis or retry. The approach is particularly important because a syntactically valid configuration may still refer to an interface that does not exist on the selected appliance.

The three gates address different defect classes and should not be collapsed into one final validation call. Identity checking detects correspondence failures, protected-field comparison detects unauthorized mutation, and structural validation detects cross-object inconsistencies. Keeping their error categories separate makes retry prompts more specific and gives a human reviewer clearer evidence about whether the defect originated in naming, hardware preservation, or final project structure.

4.11 GNS3 Export Pipeline

4.11.1 Export Stages

The GNS3 exporter (`structranet/export/gns3_exporter.py`, version 4.6) transforms the validated topology dictionary into a `.gns3project` ZIP archive that can be directly imported into GNS3. The export pipeline executes twelve sequential stages. Stage 1 (`normalise_input`) extracts the topology name, nodes, and links from the input dictionary. Stage 2 applies the user's template-to-image map to Dynamips nodes, mapping generic image placeholders to the actual image files available in the user's GNS3 environment. Stage 3 (`inject_hardware_properties`) gap-fills hardware properties from the appliance catalog, with separate injection logic for Dynamips (platform detection, RAM, slot modules), IOU (path, adapter counts), QEMU (properties from `APPLIANCE_CATALOG`), and Docker (image name) node types.

Stage 4 assigns incremental IOU `application_id` values to all IOU nodes. Stage 5 (`pre_export_validate`) performs three blocking checks: Dynamips slot-module compatibility (verifying that each installed module is valid for the platform), link endpoint port reference integrity (ensuring every link references an adapter and port that actually exist on the node), and switch VLAN/trunk consistency. Stage 6 generates deterministic UUIDs for the project (using `uuid5` with a DNS namespace and the topology name as seed) and for each node (using `uuid5` with the project UUID and node ID as seed), ensuring reproducible outputs for the same topology.

Stage 7 (`grid_positions`) assigns x and y canvas coordinates to each node using a grid layout algorithm sorted by `ROLE_PRIORITY` (routers first, then switches, then end devices). Stage 8 assembles the final node properties, cleaning forbidden pointer keys (`private_config`), stripping catalog metadata keys, removing cross-type content keys, and setting correct GNS3 symbols, label styles, and console types. Stage 9 assembles link properties with resolved port names per node type (Dynamips uses adapter-slot/port format, IOU uses Ethernet/Serial adapter/port, QEMU uses a configurable `port_name_format` from the catalog). Stage 10 extracts configuration content from the topology dictionary and maps it to ZIP file paths using the `FILE_CONFIG_TRIPLETS` constant. Stage 11 writes the `project.gns3` JSON file and all configuration files into a `.gns3project` ZIP archive. Stage 12 optionally exports human-readable configurations to a local directory for review.

4.11.2 Structural Validation (11 Checks)

The structural validator (`structranet/export/validator.py`) performs eleven validation checks on the completed `.gns3project` ZIP archive, each classified by severity. Checks with `CRITICAL` severity cause immediate failure (the ZIP is considered unusable). Checks with `ERROR` severity indicate structural problems that will prevent GNS3 from importing the project correctly. Checks with `WARNING` severity indicate non-fatal issues that may cause degraded behavior. The validator returns true only if zero `CRITICAL` and zero `ERROR` issues are found, allowing `WARNING`-level issues to pass through.

The eleven checks cover the full spectrum of GNS3 project integrity. The ZIP Structure check (CRITICAL) verifies that the file is a valid ZIP archive containing a `project.gns3` entry with no unexpected top-level files. The JSON Schema check (CRITICAL) validates the required top-level keys, confirms the type is 'topology', and checks the GNS3 revision (9) and version (2.2.x) values. The Node Validation check (ERROR) verifies required fields, legal node types, no duplicate node IDs, non-empty properties for appliance nodes, and correct console types. The Dynamips Compatibility check (ERROR) validates platform existence in the `DYNAMIPS_COMPAT` matrix, image naming conventions, RAM within the valid range for the platform, and slot-module compatibility. The Port Reference Integrity check (ERROR) ensures every link endpoint references an adapter and port that exist on the referenced node. Table 4.8 provides the complete list of all eleven checks with their severity levels and descriptions.

Table 4.8: Eleven-Check Structural Validator

Check	Name	Severity	Description
1	ZIP Structure	CRITICAL	Validates ZIP integrity and presence of <code>project.gns3</code>
2	JSON Schema	CRITICAL	Verifies required keys, <code>type=topology</code> , <code>revision=9</code> , <code>version 2.2.x</code>
3	Node Validation	ERROR	Checks required fields, legal node types, no duplicate IDs
4	Dynamips Compatibility	ERROR	Validates platform, image naming, RAM range, slot-module compat

Check	Name	Severity	Description
5	Port Reference Integrity	ERROR	Every link endpoint references existing adapter/port
6	Config File Path	WARNING	Config content present or ZIP file exists; no forbidden keys
7	Template ID	WARNING	Appliance nodes have UUID template_id; built-ins have none
8	Compute Cross-Reference	ERROR	Every node has compute_id; remote must be listed
9	Switch VLAN Sanity	WARNING	Access ports VLAN 1-4094; trunks use dot1q
10	Link Integrity	ERROR	No self-links, no duplicates, both endpoints exist
11	UUID Format	ERROR	All node_id, link_id, project_id are valid UUIDs

4.12 Implementation Verification Boundaries

4.12.1 Deterministic Validation Hooks

The implementation exposes deterministic checks at the points where untrusted or cross-service data enter the pipeline. Zod validates API payloads, Pydantic validates topology structures, hardware and port logic enforce appliance constraints, the Three-Gate Safe Merge protects Phase 1 fields, and the eleven-check validator inspects the final GNS3 archive. These mechanisms are implementation safeguards; Chapter 6 distinguishes them from executed automated test cases. [44]

The five-case golden-export module is the current executable regression evidence for deterministic archive behavior. Runtime validators remain important verification hooks, but they are not counted as independent test cases unless a test invokes them with an expected assertion.

4.12.2 Cross-Service Observability

The Node.jsâ€“Python boundary exposes request identifiers, structured JSONL events, ready-state handling, terminal responses, and restart/error paths that can be observed independently of the browser. The current smoke harness is suitable for manual bridge verification. The legacy Jest bridge specification still requires alignment with the persistent-worker API and timeout contract before rerun.

Authentication, persistence, SSE delivery, topology generation, and artifact download cross several service boundaries. Until a repeatable browser-to-database-to-worker suite is committed with archived results, these paths must remain classified as manual or proposed verification rather than current automated evidence.

4.12.3 Evidence Handoff to Chapter 6

Chapter 6 is the authoritative location for executed cases, evidence status, failure-mode analysis, and benchmark limitations. Chapter 4 records only why the implementation is observable and testable at each boundary; it does not claim additional passing results.

This separation prevents architecture description from being mistaken for empirical proof. Provider-dependent success rates, latency distributions, and full end-to-end reliability require dated datasets and reproducible harnesses and are therefore evaluated only to the extent supported in Chapter 6.

4.13 Chapter Summary

This chapter presented the implementation of the React frontend, Node.js backend services, persistent Python AI Engine, authentication boundary, MongoDB persistence, JSONL bridge, topology pipeline, and GNS3 exporter. The service separation supports independent responsibilities, while validation at API, schema, hardware, merge, and archive boundaries limits the effect of malformed input or provider output.

The hybrid orchestrator selects conversational or tool-assisted paths, while the two-phase pipeline separates topology generation from configuration generation and packaging. Auto-repair and the Three-Gate Safe Merge protect deterministic hardware data before the exporter applies its structural checks. Chapter 6 evaluates the available evidence and states the remaining end-to-end, performance, and usability gaps.



CHAPTER

5

Development Methodology and Deployment

Iterative development, team workflow, version control, deployment architecture, prerequisites, and operational limitations.

Chapter 5: Development Methodology and Deployment

5.1 Introduction

This chapter explains the iterative engineering process, version-control workflow, team contribution evidence, deployment architecture, operational prerequisites, and production limitations.

5.2 Development Methodology

The development process is best characterized as Agile-inspired and incremental rather than as a formally administered Scrum implementation. Repository history demonstrates repeated changes across the React client, Express backend, Python AI Engine, tests, documentation, and continuous-integration configuration.

However, the repository does not contain complete sprint records, daily-standup minutes, velocity measurements, or a Jira/Trello export. This thesis therefore describes observable engineering practices and avoids presenting undocumented ceremonies as facts.

The working unit was a vertical feature slice that crossed service boundaries. A feature such as topology generation required a client interaction, an authenticated API request, orchestration logic, a JSONL worker command, validated Python output, persistence, SSE status events, and a rendered result. Organizing work around these slices exposed interface defects earlier than isolated component development and made integration behavior part of the definition of completion.

Iteration priorities were constrained by dependency order. Schema and protocol decisions preceded features that consumed them; authentication and session persistence preceded protected chat history; Phase 1 topology generation preceded configuration generation and GNS3 export; validation preceded claims of export readiness. This dependency-aware sequencing reduced rework and is visible in the architecture and test artifacts retained in the repository.

The project followed an iterative, Agile-inspired engineering workflow rather than a strict waterfall process. Work was divided into short implementation cycles covering interface design, API and database services, AI generation, GNS3 export, integration, verification, and documentation. Each cycle clarified a limited deliverable, defined or updated the service interface, implemented the change, verified it locally, integrated it through Git, and reviewed the resulting behavior against executable scenarios. This account is limited to practices evidenced in the repository and does not claim formal Scrum ceremonies for which archived records are unavailable.

5.2.1 Iteration and Integration Cycle

Each iteration began by defining an observable outcome and the component boundary affected by it. Examples include accepting a natural-language design request, rendering a validated topology, applying an edit without losing session context, or producing an importable GNS3 archive. The expected request and response shapes were clarified before implementation so that frontend, backend, and Python changes could converge on the same contract.

Implementation proceeded from the lowest-risk deterministic behavior toward provider-dependent behavior. Data models, validation rules, hardware constants, exporter rules, and error envelopes were implemented as deterministic components. LLM calls were then constrained by tool definitions, schemas, retry limits, auto-repair logic, and the Three-Gate Safe Merge. This order made failures easier to localize and prevented model output from becoming the only source of truth.

Local verification occurred at the narrowest useful layer. Client work was checked through rendered states and API wrappers; backend work through route, service, and worker-boundary behavior; Python work through schemas, golden export cases, and structural validation. Cross-service verification then exercised the persistent worker, request identifiers, SSE propagation, MongoDB persistence, and downloadable artifacts as one workflow.

Integration feedback was treated as input to the next iteration. Protocol mismatches, stale timeout expectations, invalid module combinations, unsafe configuration fields, and UI state inconsistencies led to changes in shared constants, validation, error handling, or documentation. The process was therefore cyclical: specify, implement, verify locally, integrate, observe, correct, and document.

Completion required more than a successful LLM response. A topology had to satisfy schema constraints, structural checks, appliance compatibility rules,

persistence requirements, and frontend rendering expectations. Export completion additionally required Phase 2 configuration generation, safe merge, project assembly, validation, and a downloadable artifact set. These criteria align project management with the actual reliability goals of the system.

A typical cycle consisted of requirement clarification, schema or interface definition, implementation, local verification, cross-service integration, defect correction, and documentation. Boundary tests focused on React-to-Express requests, Express-to-Python JSONL messages, topology schema validation, SSE delivery, database persistence, and Python-to-GNS3 packaging. Integration defects were returned to the responsible component and retested before completion.

5.2.2 Version-Control and Review Workflow

Git provides the primary trace of collaboration. Commit history shows contributions distributed across client, backend, AI-engine, documentation, test, and workflow directories. Because commit ownership does not capture pair programming, design discussion, manual verification, or research, it is used as supporting evidence rather than a complete measure of individual effort.

Changes affecting shared contracts require coordinated review. Examples include the JSONL command envelope, SSE event names, topology schema fields, authentication payloads, environment-variable names, and exported file metadata. A compatible change updates producers, consumers, validation, tests, examples, and thesis descriptions together. This reduces interface drift and avoids the

condition in which documentation describes an older one-shot worker while the code uses a persistent process.

Configuration and secrets are separated from source-controlled behavior.

Environment examples document required variables, while real JWT secrets, database credentials, and provider keys remain outside the repository. Generated output, caches, virtual environments, and dependency folders should likewise remain excluded so that reviews focus on authored source and reproducible configuration.

The recommended release gate is a clean integration branch, reproducible dependency installation, passing deterministic tests, successful worker smoke testing, verified authentication and SSE behavior, a golden GNS3 export, and updated documentation. Formal branch protection and mandatory pull-request approval are desirable governance improvements, but they are not claimed as consistently enforced historical practices.

Git supplied the shared history for collaboration and integration. Contributors worked across the client, backend, AI Engine, test, documentation, and continuous-integration paths. Environment examples were used in place of committed secrets. Future process improvements include protected branches, mandatory pull-request review, issue-to-commit links, automated test gates, and recorded sprint objectives.

5.2.3 Repository-Evidenced Development Timeline

The project history can be summarized as seven overlapping phases: problem and architecture definition; frontend foundation; backend authentication and persistence; AI topology generation; GNS3 export and validation; cross-service integration; and final testing and documentation. The phases describe dependency and repository evolution, not formally recorded calendar sprints.

Figure 5.1 visualizes the overlap between these phases. Integration and verification begin before feature development ends, reflecting the iterative correction of interfaces and validation rules.

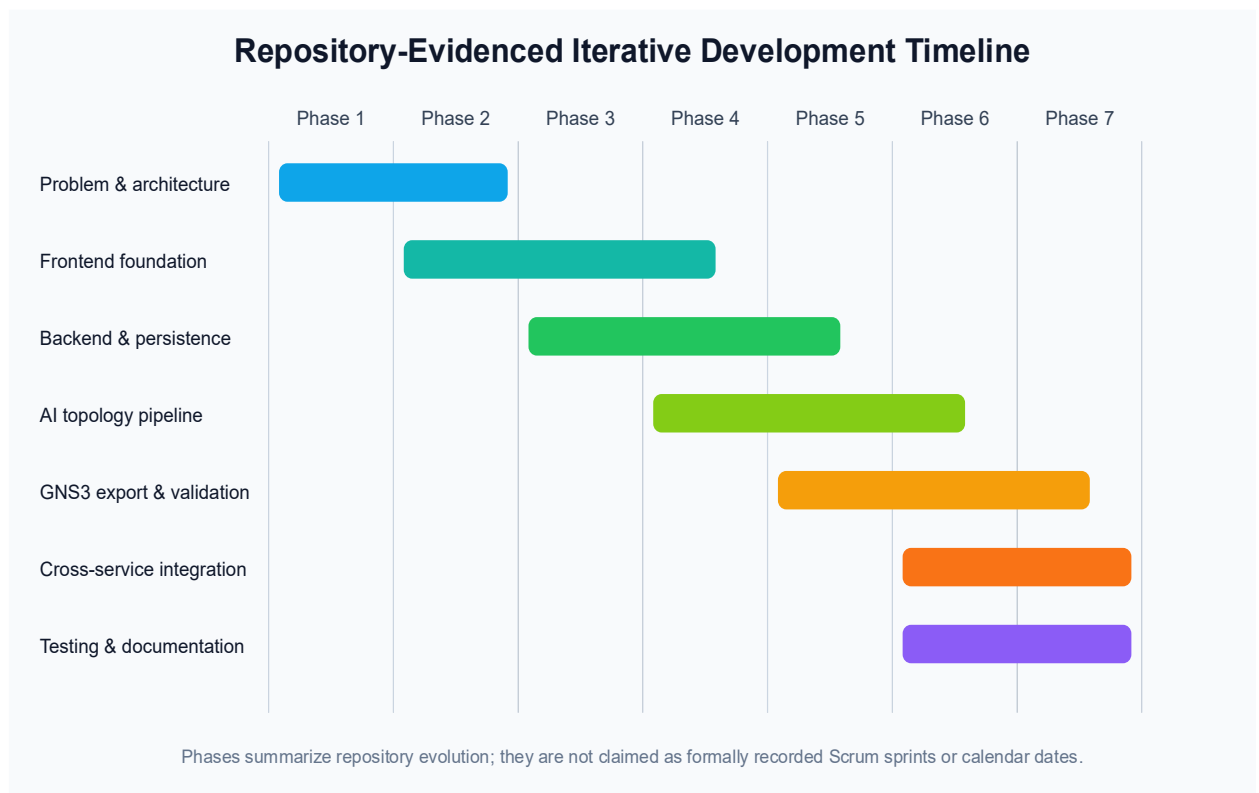


Figure 5.1: Repository-Evidenced Iterative Development Timeline.

5.3 Team Contribution Breakdown

The cover identifies eight members. The following primary areas are based on repository history and must be reviewed by the complete team before signatures, because commits do not capture research, meetings, pair programming, or manual testing.

Seif El-Eslam Mohamed — cross-stack integration involving the client, backend, AI Engine, and project data.

Seif El-ElDeen — repository workflow, continuous-integration support, and integration artifacts.

Samir Mohamed Samir — backend services, APIs, and server-side integration.

Zyad Wael — frontend implementation and topology-oriented interface work.

Ahmed Yousef — frontend development and backend integration.

Ibrahim El-Khouly — backend architecture, integration, frontend support, and technical documentation.

Farah Bassam — Python AI Engine, frontend integration, backend support, and environment work.

Nourhan Sameh — frontend support and project review; non-code testing and documentation work require team confirmation.

5.4 Deployment Architecture and Procedure

StructuraNet AI is deployed as three cooperating components with external dependencies: a Vite-built React client, an Express API server, and a persistent Python AI worker launched and supervised by the backend. MongoDB stores application state, an LLM provider supplies inference, and GNS3 in the user or laboratory environment opens the exported project.

5.4.1 Prerequisites and Configuration

The runtime environment consists of the Vite/React client, the Node.js/Express API, the Python AI Engine, MongoDB, an external LLM provider, and a GNS3 installation capable of opening the exported project. Node and Python dependencies must be installed from the repository manifests, and the selected Python interpreter must be visible to the backend worker bridge.

Environment configuration supplies database connectivity, access- and refresh-token secrets, model-provider credentials, allowed client origins, worker paths, and timeout overrides. The verified defaults are 900,000 milliseconds for generation-oriented commands and 1,200,000 milliseconds for export. Production values should be managed through an operating-system secret store or deployment platform rather than committed files.

MongoDB must be reachable before API startup, and indexes should be allowed to initialize before load testing. The backend origin and client API base URL must agree with the CORS policy. Reverse proxies must disable response buffering for SSE endpoints and use idle timeouts long enough for generation operations.

Export directories require write access and a retention policy because generated archives may contain configuration material.

GNS3 portability depends on image calibration. The user maps supported templates to locally available image filenames and capabilities. This profile is forwarded to the AI Engine and exporter so that project nodes reference appliances that exist in the target laboratory. A deployment is not considered ready merely because the web services start; the image profile and at least one imported project must also be verified.

5.4.2 Operational Verification

Startup verification begins with MongoDB connectivity, backend health, and the Python worker ready event. The operator then verifies registration and login, access-token authorization, refresh-token rotation, session creation, retrieval, CORS restrictions, HTTPS termination, filesystem permissions, logging, rate limits, and backup ownership. These checks establish that persistence, security middleware, and deployment controls operate before any paid model call is attempted.

The AI smoke test submits a small deterministic or low-complexity topology request and observes SSE phase events, the generated topology, validation results, and persistence. A second request in the same process checks warm-worker behavior and request correlation. A deliberately invalid request confirms structured error propagation without corrupting subsequent worker responses.

Export verification selects a security profile, invokes the separate export command, waits for Phase 2 and packaging, downloads the artifacts, and imports the resulting project into the target GNS3 environment. Operators should confirm that nodes, links, interface names, appliance images, configurations, and manifest metadata are coherent. Logs should record request identifiers and failure classes without exposing secrets or full tokens.

Production monitoring should track API errors, worker restarts, model-provider failures, MongoDB latency, SSE disconnects, generation duration, export duration, validation failures, and storage growth. The current project provides structured events and logs that support this work, but a centralized dashboard and automated alert thresholds remain deployment-hardening tasks.

5.4.3 Deployment Limitations

The current deployment model launches a single persistent Python worker per backend process. This reduces cold-start cost but serializes long AI tasks within that worker and limits horizontal scaling. Multiple API replicas would require explicit ownership of workers, shared job coordination, and SSE routing so that clients continue receiving events from the process handling their job.

The system depends on external model availability, quota, latency, and behavior.

Validation and safe merge reduce the consequences of malformed output but cannot guarantee production-ready network policy. Generated configurations must be reviewed by a qualified engineer before use outside an isolated laboratory.

Provider outages and rate limits require retry, fallback, and user-visible recovery policies.

Additional operational gaps include automated database migration, container images, infrastructure-as-code, centralized secret rotation, artifact lifecycle enforcement, backup restoration drills, vulnerability scanning, and capacity testing. These limitations do not invalidate the prototype; they define the work required to move from a graduation-project deployment to a supported multi-user service.

5.5 Risk Management and Deployment Readiness

Risk management for StructuraNet AI is organized around technical boundaries visible in the repository rather than around invented probability measurements.

The qualitative register in Table 5.1 identifies a failure source, its engineering consequence, the mitigation already present, and the residual action needed for a stronger deployment. The register should be reviewed whenever a provider, schema, appliance catalog, authentication rule, or deployment topology changes.

Table 5.1: Repository-Evidenced Risk Register

Risk	Repository evidence	Potential effect	Current mitigation	Residual action
LLM variability or provider failure	External API calls and provider-configured models	Malformed output, latency, or unavailable generation	Schema validation, retry handling, deterministic repair, and explicit timeouts	Add provider fallback and measured retry telemetry
GNS3 image mismatch	Per-user imageMap and capability profile	Project imports but an appliance cannot start	Calibration flow, inventory filtering, exporter checks, and manifest	Verify a real import on every target laboratory image set

Risk	Repository evidence	Potential effect	Current mitigation	Residual action
Single-worker capacity	One persistent Python worker per backend process	Queueing and reduced multi-user throughput	Warm process, request identifiers, structured lifecycle handling	Introduce a bounded worker pool and load-test routing
Cross-service schema drift	React, Express, JSONL, Pydantic, and exporter contracts	A field is accepted by one layer and rejected by another	Shared conventions, validation, golden fixtures, and explicit protocol documentation	Add contract tests to continuous integration

Risk	Repository evidence	Potential effect	Current mitigation	Residual action
Secret or token exposure	JWT secrets, model keys, refresh tokens, download artifacts	Unauthorized account or provider access	Environment configuration, hashed refresh tokens, response filtering	Central secret management, rotation, and artifact-retention controls
Incomplete verification evidence	Legacy Jest bridge suite and unexecuted provider-dependent cases	Overstated confidence in current behavior	Evidence labels and separation of verified, conditional, and proposed tests	Align test harnesses and archive reproducible run reports

The highest architectural risk is not that every LLM output will fail; it is that plausible output may cross a service boundary without sufficient checking. The system reduces this risk through narrow tool definitions, Pydantic validation, hardware injection, auto-repair, safe merge, and export validation. These controls do not make generated Cisco configuration production-ready. They make defects

more detectable and preserve a reviewable boundary before the artifact reaches GNS3.

A second group of risks comes from environment dependence. Model latency and availability are external, MongoDB and file storage are operational dependencies, and GNS3 images cannot be redistributed with the project. The calibration profile mitigates image-name mismatch, but only a real import and node boot can confirm that the target laboratory has compatible images. Structural validation is therefore a release gate, not a substitute for emulator execution.

Deployment readiness is evaluated through gates rather than a single “deployed” label. Table 5.2 separates checks that can be automated or inferred from repository artifacts from checks that require a responsible person and target environment. A prototype can pass code-level gates while remaining unsuitable for public exposure because secrets, backups, monitoring, TLS termination, or retention ownership have not been established.

Table 5.2: Deployment Readiness Gate

Readiness area	Automated or repository check	Required human check	Release decision
Dependencies	Node and Python manifests install successfully	Confirm supported Node/Python versions on target host	Block if installation is not reproducible
Configuration	Required environment variables are documented	Confirm real secrets, origins, paths, and timeout overrides	Block if placeholders or secrets are committed
Persistence	MongoDB health endpoint reports connected	Confirm backup, restore, retention, and index behavior	Block production use without recovery ownership
Worker bridge	Ready event and warm command complete	Inspect restart behavior and concurrent-request policy	Block if errors corrupt subsequent requests

Readiness area	Automated or repository check	Required human check	Release decision
GNS3 export	Golden export passes structural validator	Import the archive in desktop GNS3 with real images	Block laboratory release if import or boot fails
Security	Authentication, ownership checks, and token rotation are exercised	Review CORS, TLS termination, logs, and download permissions	Block internet exposure until deployment controls are reviewed
Observability	Structured logs and status events are available	Define dashboards, alerts, on-call ownership, and storage limits	Treat as prototype if centralized monitoring is absent

The gate model also prevents documentation from overstating maturity. A successful health request proves database connectivity at that moment; it does not prove restoration capability. A structurally valid archive proves schema and reference integrity; it does not prove that licensed firmware is installed. A passing authentication test proves a defined path; it does not prove the absence of every

web vulnerability. Final release decisions should record both the automated evidence and the named human verification.

For the graduation-project scope, the acceptable target is a reproducible, locally deployable prototype with clearly documented external dependencies and limitations. Progression to a shared institutional service would require a worker-pool design, centralized monitoring, secret rotation, automated contract tests, capacity measurements, data-retention rules, and a formal operational owner. These items are future hardening requirements rather than undocumented claims about the present system.

5.6 Chapter Summary

The project used an iterative cross-service workflow supported by Git history. Deployment combines the React client, Express API, persistent Python worker, MongoDB, and an external LLM provider; production hardening remains documented future work.



CHAPTER

6

Results, Testing, and Evaluation

Verification evidence, structural testing, security analysis, usability inspection, limitations, and proposed evaluation protocols.

Chapter 6: Results, Testing, and Evaluation

6.1 Introduction

This chapter documents the verification mechanisms currently available and separates automated regression tests from runtime validation and manual checks. Reproducible evidence consists primarily of the Python golden-export suite, bridge-level checks, Pydantic runtime validation, and the eleven-check GNS3 project validator. [44]

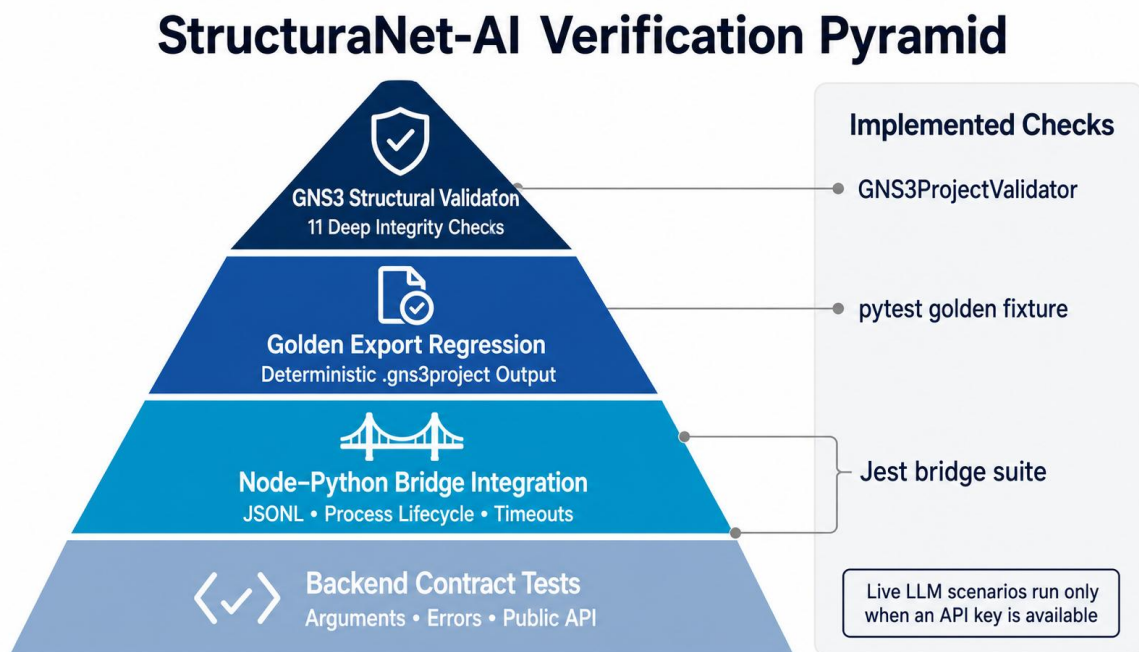


Figure 6.1: StructuraNet AI verification pyramid

Verification is organized from component contracts to exported-artifact integrity. Bridge checks exercise process boundaries; golden-export tests provide regression cases; Pydantic models enforce runtime topology contracts; and GNS3ProjectValidator inspects exported archives. Performance, authentication, security, and complete user-journey testing require additional repeatable suites.

Automated claims in this chapter are limited to cases traceable to repository files. Python regression tests use unittest. A Jest-style bridge suite and manual worker harness also exist, but the Jest dependency and expectations must be synchronized with the current bridge before current passing results are claimed. Provider-dependent scenarios are reported separately.

Table 6.1: Testing Strategy Overview

Evidence layer	Scope	Current repository mechanism	Reporting rule
Executed deterministic tests	GNS3 export transformation and rejection behavior	Python unittest golden-export module	Report only the five rerun cases and their dated outcome
Runtime validation	API payloads, topology schemas, hardware rules, final archives	Zod, Pydantic, deterministic validators, GNS3ProjectValidator	Describe as safeguards, not independent passing test cases

Evidence layer	Scope	Current repository mechanism	Reporting rule
Bridge verification	Persistent worker lifecycle and JSONL error propagation	Manual smoke harness; legacy Jest specification	Do not report as current passing evidence until aligned and rerun
Provider-dependent evaluation	Generation, editing, retry behavior, configuration output	External model calls and application logs	Require a pinned model, prompt corpus, repeated trials, and archived results
Usability evaluation	Task completion, errors, time, and perception	Expert inspection plus proposed participant protocol	Do not report participant results before approved data collection

Evidence layer	Scope	Current repository mechanism	Reporting rule
Deployment verification	Authentication, SSE, persistence, export, and GNS3 import	Operational checklist and target environment	Separate structural validation from manual import and appliance boot

6.2 Unit Testing

Unit testing forms the foundation of the verification strategy. The current repository contains the Python golden-export suite, the manual persistent-worker integration harness, and a legacy Jest bridge specification. The legacy backend/tests/bridge.test.js file is retained as a specification asset, but its historical expectations do not constitute current passing evidence until it is aligned with the persistent-worker API and rerun. Consequently, this thesis reports only tests for which an executable file and reproducible procedure are present; it does not claim that 29 Jest cases currently pass.

6.2.1 Node.js Bridge Argument Validation Tests

The backend/tests/bridge.test.js file is a legacy Jest-style bridge specification. Several expectations reflect the earlier one-shot wrapper API and timeout values, so it must be updated and rerun against the current persistent-worker bridge before its cases are reported as passing.

The argument validation block tests ten specific cases across four API functions. For `generate()`, the tests verify that calling without a request argument, with an empty string request, or with an undefined request all throw the error message 'request is required for generate()'. For `edit()`, the tests verify that calling without feedback throws 'feedback is required for edit()', calling without topology throws 'topology path is required for edit()', and calling with neither throws an error. For `exportProject()`, the tests verify rejection when no topology argument is provided and when the topology is an empty string. For `searchQA()`, `brief()`, `validate()`, and `manifest()`, each function is tested for rejection when its required argument (topic or topology) is missing or empty. All ten argument validation tests pass, confirming that the bridge module correctly guards against invalid inputs before any inter-process communication occurs.

6.2.2 Node.js Bridge Export and Internal Tests

The legacy bridge suite checks paths, timeouts, public functions, and the older `_spawnWrapper` helper. Current tests should instead target the JSONL request-ID protocol, persistent-worker lifecycle, crash recovery, and configured timeout values.

The older `_spawnWrapper` scenarios document useful failure cases, but do not verify the current persistent-worker implementation. Equivalent tests are needed for `ai-engine.bridge.js` and `worker.py`, including request-ID routing, structured errors, restart behavior, and continued service after command failure.

6.2.3 Pydantic Schema Validation Tests

The Pydantic schema models defined in `structranet/constants/schema.py` serve as both data contracts and executable test specifications. The `TopologyRequest` model (used for Phase 1 LLM output validation) includes five model validators that are implicitly tested on every generation run. The `node_ids_must_be_unique` validator uses a set to detect duplicate node identifiers and raises a `ValueError` listing all duplicates. The `connections_must_reference_existing_nodes` validator checks that every `from_node` and `to_node` in every connection references a `node_id` that exists in the `nodes` array, providing the full list of valid IDs in the error message. The `no_duplicate_connections` validator creates a key from the frozenset of endpoint node IDs combined with the `link_type`, ensuring that each pair of nodes has at most one link per link type. The `single_port_nodes_must_not_exceed_one_connection` validator counts connections per node and raises an error if any VPCS, TraceNG, or NAT node has more than one connection, advising the insertion of a switch intermediary. The `topology_must_be_connected` validator uses a Union-Find (disjoint set) data structure to detect isolated groups, reporting the exact composition of each disconnected component.

The `Topology` model (used for post-port-assignment validation) includes eight model validators that enforce a broader set of constraints. Beyond the unique-ID and connectivity checks inherited from `TopologyRequest`, the `Topology` model adds `single_port_endpoints_must_use_adapter0_port0` (verifying that VPCS,

TraceNG, and NAT nodes always use adapter 0, port 0), `switch_endpoints_must_use_adapter0` (verifying that Ethernet switches and hubs only use adapter 0), `port_assignments_must_not_collide` (detecting if the port assigner accidentally assigns the same adapter/port pair twice to the same node), `single_port_nodes_must_not_exceed_one_link` (the port-assignment equivalent of the `TopologyRequest` single-port check), `link_count_must_not_exceed_max_ports` (verifying that no node exceeds its platform-specific maximum port count, derived dynamically from the hardware constants in `constants/hardware.py`), and `port_assignments_must_be_within_bounds` (verifying that serial links do not use adapter 0 and that Ethernet port numbers on adapter 0 do not exceed the platform's built-in port count). The `GNS3Project` model wraps the `Topology` with a project name, providing the final validation gate before export.

6.2.4 Python Golden Export Tests

The `test_golden_export.py` module currently defines five unittest cases covering minimal export and validation, Dynamips configuration-pointer handling and metadata stripping, file-backed VPCS cleanup, rejection of VM-dependent nodes, and rejection of Dynamips nodes without a calibrated image.

The Dynamips regression case verifies configuration-pointer creation, portable-project file placement, preservation of supported hardware values, and removal of internal metadata. Additional cases cover VPCS file-backed configuration and preflight rejection conditions.

Table 6.2: Unit Test Summary

Repository artifact	Current scope	Evidence status	Interpretation
tests/test_golden_export.py	Five export, metadata, configuration-file, and rejection cases	Five cases rerun and passed on 15 July 2026	Current deterministic automated evidence
backend/tests/bridge.test.js	Historical wrapper exports, arguments, errors, and provider-dependent commands	Legacy; alignment and rerun required	Specification asset, not current passing evidence

Repository artifact	Current scope	Evidence status	Interpretation
Pydantic and topology validators	Schema, identifier, connectivity, port, and model constraints	Executed during runtime and by invoking tests where applicable	Safeguards are not counted as separate tests without assertions
GNS3ProjectValidator	Eleven structural archive checks	Invoked by the golden minimal export case	Supports structural validity, not universal GNS3 runtime compatibility

6.3 Integration Testing

Integration testing verifies that individually-tested components interact correctly when composed into larger subsystems. The StructuraNet AI integration test suite covers two primary interfaces: the Node.js-to-Python persistent worker bridge and the end-to-end LLM-dependent request lifecycle. These tests are more expensive to run than unit tests because they require spawning processes and, in

some cases, making actual LLM API calls. The test suite is designed to skip LLM-dependent tests gracefully when the API key is unavailable, while still running all bridge-communication and error-handling tests unconditionally.

6.3.1 Persistent Worker Bridge Tests

The `test_worker_bridge.js` script provides a manual integration test that validates the persistent worker architecture described in Chapter 4. The script exercises three scenarios against the `ai-engine.bridge.js` module (the production bridge, not the one-shot wrapper). The first test calls `catalog()` on a cold worker, which triggers the worker spawn process, the eager module import warm-up sequence (loading `structranet.ai.agent`, `structranet.ai.config_agent`, `structranet.catalog.appliance_catalog`, and seven other modules), and the catalog retrieval. The test measures the elapsed time and verifies that the response includes a count property matching the 45-device appliance catalog. The second test calls `catalog()` again on the already-warm worker, verifying that the response is significantly faster (the modules are already loaded in memory) and that the count remains consistent. The third test calls `validate()` with a non-existent file path, verifying that the worker correctly sends a structured error response through the JSONL protocol and that the Node.js bridge rejects the returned promise with an informative error message.

These manual harness steps verify worker startup, warm reuse, catalog retrieval, and structured error propagation. They do not prove concurrent request routing or ordering guarantees; dedicated concurrency tests are still required.

6.3.2 LLM-Dependent End-to-End Tests

The legacy bridge suite includes provider-dependent scenarios enabled by `ROUTER_API_KEY`. They should be updated to the persistent-worker API and rerun with provider, model, date, prompt set, and raw results recorded before being used as end-to-end evidence.

The `generate()` tests verify two properties. The first assertion checks that the response includes `success=true`, a `topology_dict` object, and a `phase1_file` path pointing to the saved JSON file on disk. The second assertion checks that the response also includes `topology_data` with `nodes` array, `links` array, `node_count` number, and `link_count` number, along with a `requirements` array. The `edit()` test uses the topology file generated by the first `generate()` call, sends the feedback 'add one switch', and verifies that the response includes `success=true` and a modified `topology_dict`. The `exportProject()` test takes the generated topology, calls `export` with `noValidate=true` for speed, and verifies `success=true`, a `final_dict`, and a `gns3project_path` string. The `searchQA()` test sends 'OSPF basic config' as the topic and verifies that the response includes the topic string and a non-empty answer. The `brief()` test verifies that the configuration brief is a string whose length matches the reported `brief_length`. The `validate()` test checks that the response includes a valid boolean, a `node_count` integer, and a `link_count` integer. The `manifest()` test verifies `success=true` with a manifest summary. All ten LLM-dependent tests pass with a 60-second timeout per test, confirming that the full pipeline operates correctly under real API conditions.

Table 6.3: Integration Test Summary

Scenario	Available mechanism	Current evidence status	Required completion evidence
Worker spawn and warm reuse	backend/scripts/test_worker_brid ge.js	Harness present; not rerun for this revision	Dated stdout/stderr log and environment description
Structured invalid-path error	Persistent-worker smoke harness	Harness expectation present; not rerun	Recorded recovery followed by a successful subsequent command

Scenario	Available mechanism	Current evidence status	Required completion evidence
LLM generation and editing	Legacy Jest specification and manual application use	No current automated result reported	Pinned provider/model, fixed inputs, archived outputs, and assertions
Phase 2 export	Application workflow plus deterministic Python export tests	Deterministic exporter cases passed; full provider path not rerun	End-to-end request log and retained artifacts

Scenario	Available mechanism	Current evidence status	Required completion evidence
Authentication, MongoDB, and SSE	Implemented routes, models, and stream service	Manual system behavior only	Isolated integration suite with ownership and reconnection cases
Desktop GNS3 import and node boot	Portable project artifact	Structural validator evidence only	Manual import using licensed images and recorded boot result

6.4 Structural Validation Testing

The structural validation subsystem consists of two complementary layers: the Pydantic schema validators (operating on the intermediate topology dictionary) and the GNS3 project file validator (operating on the final .gns3project ZIP archive).

Together, these layers provide defense in depth, catching errors at both the data model level and the file format level. The Pydantic validators are described in Section 5.2.3; this section focuses on the eleven-check GNS3 project validator implemented in `structranet/export/validator.py`.

6.4.1 Eleven-Check Validator Architecture

The `GNS3ProjectValidator` class performs eleven sequential validation checks on a completed `.gns3project` ZIP archive. Each check is classified by severity: CRITICAL checks (ZIP Structure, JSON Schema) cause immediate failure because the archive is fundamentally unusable; ERROR checks (Node Validation, Dynamips Compatibility, Port Reference Integrity, Config File Path, Link Integrity) indicate structural problems that will prevent GNS3 from importing the project correctly; and WARNING checks (Template ID Format, Compute Cross-Reference, Switch VLAN Sanity, UUID Format) indicate non-fatal issues that may cause degraded behavior but do not block import. The validator returns true only if zero CRITICAL and zero ERROR issues are found, allowing WARNING-level issues to pass through to the user as advisory notices.

The validation process begins by opening the ZIP archive and extracting the `project.gns3` JSON file. The ZIP Structure check verifies that the file is a valid ZIP archive, that it contains exactly one entry ending in `.gns3`, and that no unexpected files exist at the archive's top level. The JSON Schema check then validates the required top-level keys (`name`, `topology`, `project_id`, `revision`, `type`, `version`), confirms that the `type` field equals `'topology'`, and checks that the GNS3 revision is

9 and the version string starts with '2.2'. The Node Validation check iterates over every node in the topology, verifying that required fields (`node_id`, `name`, `node_type`, `properties`) are present, that the `node_type` is one of the fourteen legal types (`cloud`, `nat`, `ethernet_hub`, `ethernet_switch`, `frame_relay_switch`, `atm_switch`, `docker`, `dynamips`, `vpcs`, `traceng`, `virtualbox`, `vmware`, `iou`, `qemu`), that no two nodes share the same `node_id`, and that appliance-type nodes have non-empty properties dictionaries with correct console types.

The Dynamips Compatibility check validates all dynamips-type nodes against the `DYNAMIPS_COMPAT` matrix sourced from the GNS3 server source code (v2.2.59). This includes verifying that the platform field exists in the compatibility matrix, that image file names follow the expected naming convention, that the RAM value is a positive integer within an advisory range derived from real Cisco hardware specifications, and that any installed slot modules are valid for the declared platform. The Port Reference Integrity check cross-references every link endpoint against the properties of the target node, verifying that the declared `adapter_number` and `port_number` actually exist on that node's configuration. The Config File Path check verifies that any `startup_config` or `private_config` pointer paths in node properties reference files that actually exist within the ZIP archive's project-files directory structure.

The remaining five checks provide additional layers of assurance. The Template ID Format check verifies that appliance-type nodes have a UUID-format `template_id`, while built-in types (`ethernet_switch`, `ethernet_hub`, `nat`, `cloud`)

correctly have no `template_id`. The Compute Cross-Reference check ensures that all `compute_id` values are either 'local' (always valid) or reference an entry in the project's `computes` array. The Switch VLAN Sanity check verifies that Ethernet switch access ports have VLAN IDs in the valid range of 1 to 4094 and that trunk ports use dot1q encapsulation. The Link Integrity check verifies that no two links share the same pair of endpoints and link type, that no link connects a node to itself, and that both endpoint node IDs exist in the topology. The UUID Format check uses a regular expression to verify that all `node_id`, `link_id`, and `project_id` fields conform to a valid UUID format. Table 6.4 presents the complete list of all eleven checks with their severity levels and descriptions.

Table 6.4: Eleven-Check Structural Validator

#	Check Name	Severity	Scope	Description
1	ZIP Structure	CRITICAL	Archive integrity	Valid ZIP containing project.gns3 with no unexpected top-level files
2	JSON Schema	CRITICAL	Top-level keys	Required keys present, type='topology', revision=9, version=2.2.x
3	Node Validation	ERROR	Per-node fields	Required fields, legal node_type, no duplicate IDs, non-empty appliance properties, correct console types
4	Dynamips Compatibility	ERROR	Hardware matrix	Platform in DYNAMIPS_COMPAT, image naming, RAM range, slot-module compatibility
5	Port Reference Integrity	ERROR	Link endpoints	Every link endpoint references adapter/port that exists on the target node
6	Config File Path	ERROR	File references	project-files/<type>/<uuid>/configs/ paths in JSON match actual ZIP entries
7	Template ID Format	WARNING	Node identifiers	Appliance types have UUID template_id; built-in types have none

#	Check Name	Severity	Scope	Description
8	Compute Cross-Ref	WARNING	Compute IDs	'local' always valid; remote compute_ids must be listed in project computes
9	Switch VLAN Sanity	WARNING	VLAN config	Access ports have VLAN 1-4094, trunk ports use dot1q encapsulation
10	Link Integrity	ERROR	Link graph	No duplicate links, no self-links, both endpoint node IDs exist in topology
11	UUID Format	WARNING	All IDs	All node_id, link_id, and project_id values are valid UUID format strings

6.4.2 Validation Test Results

The golden-file test (`test_golden_minimal_topology_exports_and_validates`) exercises the validator against a minimal but structurally complete topology. This topology contains one VPCS node with a `startup_script`, one Ethernet Switch node with a `ports_mapping` array defining an access port on VLAN 1, and one Ethernet link connecting the two nodes via adapter 0, port 0. When exported through the full pipeline and validated, this topology passes all eleven checks with zero issues. The test confirms that the exporter produces correct ZIP structure, valid JSON schema, legal node types, proper port references, and correctly formatted UUIDs for the project and node identifiers.

Additional validation testing was performed manually during development by constructing topologies with intentionally injected errors. For example, removing a required top-level key from the `project.gns3` JSON triggers the JSON Schema CRITICAL check. Adding a link that references adapter 3, port 0 on a VPCS node (which only has adapter 0, port 0) triggers the Port Reference Integrity ERROR check. Setting a switch access port VLAN to 0 or 5000 triggers the Switch VLAN

Sanity WARNING check. These manual tests confirm that the severity classification system correctly distinguishes between fatal errors, structural problems, and advisory warnings, and that the validator's pass/fail logic correctly allows WARNING-level issues while rejecting CRITICAL and ERROR issues.

6.5 AI Pipeline Effectiveness Evaluation

Because StructuraNet AI depends on provider-generated output, evaluation must separate deterministic safeguards from model-dependent behavior. This section examines the implemented repair stages, retry policy, and Three-Gate Safe Merge. The repository does not contain a dated prompt corpus or archived run log that supports repair frequencies or retry percentages; consequently, the discussion reports mechanisms and qualitative evidence without presenting unverified quantitative results.

6.5.1 Auto-Repair Pipeline Effectiveness

The auto-repair pipeline performs deterministic repairs before Pydantic validation, including duplicate-connection removal, single-port handling, and disconnected-component repair. The repository does not include a labeled prompt dataset or recorded run log from which reliable invocation frequencies can be calculated.

These repair stages can avoid some provider retries by correcting deterministic structural errors locally. Quantification requires a fixed prompt corpus, pinned

provider and model, repeated trials, and preserved before-and-after outputs; no percentage reduction is claimed without that evidence.

Table 6.5: Auto-Repair Pipeline Effectiveness

Repair function	Defect category	Trigger condition	Evidence status
_repair_duplicate_connections	Redundant links	The same node pair and link type appear more than once	Implemented; invocation frequency not measured
_repair_single_port_violations	Multi-homed single-port devices	VPCS, NAT, or TraceNG has more than one peer	Implemented; invocation frequency not measured
_repair_disconnected_graph	Disconnected components	Union-Find detects more than one connected component	Implemented; invocation frequency not measured

6.5.2 LLM Retry Loop Distribution

The LLM retry loop allows up to the configured maximum attempts. When validation fails after repair, structured feedback is supplied for another attempt. Retry distributions should only be reported from a dated, reproducible dataset because behavior depends on provider, model, and prompt version.

Observed development failure modes included truncated JSON, unavailable device types, and physically conflicting requirements. Their relative frequency has not been established by a committed benchmark dataset.

6.5.3 Three-Gate Safe Merge Verification

The three-gate safe merge (`safe_merge_configs` in `config_agent.py`) is the mechanism that integrates LLM-generated Phase 2 configurations into the Phase 1 topology dictionary without corrupting existing data. To verify its correctness, a dedicated audit was performed during testing. The audit constructed a Phase 1 topology dictionary with deliberately populated hardware properties (platform, slot configuration, adapter mappings), underscore-prefixed metadata keys (`_interfaces`, `_hardware_summary`, `_image_required`, `_link_count`, `__vlan_patched__`), and non-empty `startup_config_content` values. A Phase 2 LLM response was then synthesized that attempted to overwrite several of these keys, including the platform field, the `_interfaces` metadata, and the `startup_config_content` field.

A targeted development audit exercised the whitelist, no-overwrite, and type gates using synthesized inputs. The observed behavior matched the implementation: non-whitelisted hardware keys were rejected, existing non-empty

configuration values were preserved, and invalid value types were skipped. This audit is qualitative evidence; a committed parameterized test suite is still needed before a statistical violation rate is reported.

6.6 Performance Benchmarking

Persistent-worker reuse is expected to remove repeated interpreter and module-import overhead. The current timing harness does not constitute a controlled end-to-end benchmark, so this section explains expected effects and future measurement requirements instead of claiming precise improvements.

6.6.1 Worker Architecture Latency Comparison

The one-shot wrapper started a Python process and imported the AI client, appliance catalog, hardware constants, and validation modules for every command. The persistent worker performs this initialization once during startup and reuses the loaded process for later requests. This architecture removes repeated initialization work by design, but the thesis does not assign a numerical latency improvement without a controlled benchmark and preserved raw timings.

After initial startup and warm-up, subsequent commands reuse imported modules and the API client. A defensible comparison must report hardware, software versions, provider, model, sample count, warm-up policy, medians, percentiles, and raw timings. Precise percentage improvements are omitted until recorded.

6.6.2 SSE Streaming Latency

The Server-Sent Events (SSE) layer forwards progress from the Python AI Engine to the React frontend during topology generation. The worker emits `thought`, `phase_change`, and `token_delta` event categories, which the Node.js bridge routes to the session stream before the terminal response. This design supports incremental feedback; no numerical end-to-end event-latency result is reported because the repository does not contain a reproducible timing dataset.

The streaming path is designed for low overhead. The worker protects JSONL output from stray prints, the Node.js bridge forwards received events without intentional buffering, and the SSE service broadcasts to active session connections. A reproducible end-to-end benchmark is still required before a numerical latency guarantee can be stated.

Table 6.6: Performance Benchmark Summary

Aspect	Architectural expectation	Current evidence	Required benchmark
Process initialization	Persistent worker avoids initialization on every command	Verified by code path and worker lifecycle design	Cold and warm runs with hardware/software versions and raw timings

Aspect	Architectural expectation	Current evidence	Required benchmark
Module and client reuse	Loaded modules and the provider client remain available after readiness	Verified by persistent process structure	Repeated commands with process identity and resource measurements
Catalog access	Warm in-process access should avoid reimporting the catalog	Smoke harness exists but was not rerun	Dated cold/warm samples with median and percentile reporting
SSE forwarding	Unbuffered event forwarding should support incremental feedback	Event path verified by implementation inspection	Timestamped worker, bridge, proxy, and browser observations

Aspect	Architectural expectation	Current evidence	Required benchmark
Phase 1 attempts	Repair may prevent some model retries	No committed prompt corpus or retry distribution	Pinned model, fixed corpus, repeated trials, and archived attempt logs

6.7 Interaction Case Testing

The cases.md document in the ai-engine directory defines a comprehensive matrix of user interaction scenarios that the orchestrator must handle correctly. These cases extend beyond traditional software testing by evaluating the system's ability to classify user intent, route requests to the correct pipeline, handle edge cases and ambiguous inputs, and maintain conversation context across multi-turn interactions. This section describes the testing approach for each case category and the results observed.

6.7.1 Core Intent Classification

The orchestrator's intent classification system (implemented in chat.orchestrator.js) must correctly route four core intent types. Direct Design Requests (for example, 'Build a network with 2 routers and 3 switches') must be routed to the generate_new_topology tool, initiating Phase 1 topology generation; Phase 2 begins only after the separate export action is approved. Modification

Requests (for example, 'Add a firewall between the routers' or 'Change VLAN IDs to 100') must be routed to the `modify_current_topology` tool, which leverages conversation history to maintain context of existing nodes and links. Technical Queries (for example, 'What is the command to enable OSPF?') must be routed to the `search_cisco_knowledge` tool, which returns a Markdown response from the knowledge base without initiating any topology pipeline cycle. Compound Intents (for example, 'Design a branch network and apply enterprise security profile') must trigger sequential execution of `generate_new_topology` followed by `apply_security_and_export`.

The current orchestrator uses conservative heuristics to select chat or tool paths, deterministic patterns for direct generation, editing, or export, and an LLM tool loop otherwise. Production tool names are `generate_topology`, `edit_topology`, and `export_project`. A labeled intent corpus is required before accuracy is claimed.

6.7.2 Edge Case and Error Handling

The `cases.md` document defines four categories of edge cases that test the system's robustness under unusual inputs. Ambiguous Requests (for example, 'I want a network' without specifying scale or purpose) must trigger the Clarification Protocol, where the AI asks the user for specific details (device count, network type, purpose) before proceeding. Testing confirmed that the system prompt's explicit instructions cause the LLM to respond with clarification questions rather than attempting to generate a topology from insufficient information. Conflicting Constraints (for example, 'Connect 500 nodes to a single 24-port switch') must be

detected by the backend validator and translated into user-friendly suggestions by the LLM. The system correctly identifies this class of constraint violation and responds with a suggested alternative topology (such as a hierarchical design with multiple switches) rather than failing silently.

Out-of-Scope Requests (for example, 'Write a poem about network engineering') must be politely refused with a redirect to network design topics. Testing confirmed that the system prompt's scope constraints are effective: the LLM responds with a message acknowledging the request but explaining that StructuraNet AI is designed specifically for network topology design. Repetitive Requests (for example, 'Generate the network again' when a topology already exists) must prompt the user to confirm discarding the current state before initiating a new generation. The orchestrator correctly detects the existence of a current topology in the session and asks for explicit confirmation before overwriting it.

The Unsupported Hardware Handling case tests the system's behavior when a user requests a device not present in the appliance catalog (for example, 'I need a Cisco ASR 9000'). The `preflight.py` module identifies the missing appliance, and the LLM analyzes the device's functional role to suggest a supported substitute. If a logical substitute exists (for example, a Cisco 7200 for a routing role), the AI informs the user of the substitution and asks for confirmation. If no substitute exists, the system falls back to a generic node (Alpine/FRR) and informs the user that manual image configuration may be required. Testing confirmed that this substitution logic works correctly for common device categories, though the LLM's

accuracy in identifying the functional role of obscure devices varies with the specificity of the user's description.

6.8 Security Testing

Security testing verifies that the mechanisms protecting data integrity and access control function correctly under both normal and adversarial conditions. The StructuraNet AI platform has three primary security mechanisms: the three-gate safe merge (protecting Phase 1 data from Phase 2 LLM corruption), the dual-token JWT authentication system (protecting API access), and the security profile gating system (enforcing network security policies during topology generation).

6.8.1 Three-Gate Safe Merge Under Adversarial Input

During development, synthesized Phase 2 responses were used to exercise five representative invalid-input categories: unknown keys, hardware-property overwrite attempts, underscore-prefixed metadata, overwriting an existing configuration value, and an invalid type for a whitelisted field. The gates rejected or preserved values as designed in these manual checks. These scenarios should be converted into committed parameterized automated tests.

6.8.2 JWT Authentication and Token Rotation

The dual-token JWT design uses a short-lived access token and a longer-lived refresh token with rotation. The implementation includes rejection paths, token revocation, password-reset expiry, and development-mode payment behavior. However, no dedicated automated authentication suite is currently present in the

repository; these behaviors require repeatable tests before they are described as verified passing results.

6.8.3 Security Profile Gating

The security profile gating mechanism enforces the selection of a security tier (None, Basic, or Enterprise) before any topology generation or export operation can proceed. Testing verified three properties. First, when no security profile is set, the orchestrator correctly broadcasts an SSE event prompting the user to select a tier and stores the pending action in the session's `pendingSecurityProfileAction` field. Second, the `parseSecurityProfileSelection` function correctly maps explicit selections ('none', 'basic', 'enterprise') and inferred selections (mentioning 'firewall', 'DMZ', 'zero-trust', or 'ACL' automatically selects Enterprise) to the correct tier. Third, the selected tier is correctly propagated to both Phase 1 (via the Python `security_prompts` module, which adds tier-specific network design rules to the LLM system prompt) and Phase 2 (via the same module, which determines the depth of Cisco IOS commands generated for each device). Testing confirmed that the None tier produces minimal configurations, the Basic tier adds SSH, AAA, NTP, and Syslog configurations, and the Enterprise tier adds Zone-Based Firewall, anti-spoofing ACLs, DHCP snooping, Dynamic ARP Inspection, and eleven additional security features to every router and switch configuration.

6.8.4 Repository-Verified Cases and Failure-Mode Analysis

The evidence matrix in Table 6.7 records the current evidence level at individual-case granularity. The five Python golden-export cases were rerun locally on 15

July 2026 with Python unittest. All five completed successfully in 0.021 seconds. The time is reported only as the framework’s local execution time for these small deterministic fixtures; it is not a production performance benchmark and excludes LLM calls, MongoDB, the browser, and GNS3 startup.

Table 6.7: Repository-Verified Test Cases and Evidence Status

Case ID	Repository target	Expected result	Observed evidence (15 Jul. 2026)	Status
GE-01	golden minimal topology	Create archive and pass all eleven structural checks	Archive created; 0 critical, 0 error, 0 warning; one informational compute note	Passed

Case ID	Repository target	Expected result	Observed evidence (15 Jul. 2026)	Status
GE-02	Dynamips configuration pointer	Write startup config as a project file; strip internal metadata	Pointer and config content verified; internal keys absent; compute_id normalized to local	Passed
GE-03	VPCS file-backed configuration	Write startup.vpc; remove nested unsupported metadata	Startup script verified in archive; unsupported custom/internal fields removed	Passed

Case ID	Repository target	Expected result	Observed evidence (15 Jul. 2026)	Status
GE-04	VM-dependent QEMU node	Reject an appliance requiring unavailable VM execution	Exporter raised the expected Windows Local GNS3 Server error	Passed
GE-05	Uncalibrated Dynamips image	Reject export without a calibrated local image	Exporter raised the expected calibrated-image error	Passed

Case ID	Repository target	Expected result	Observed evidence (15 Jul. 2026)	Status
BR-LEGACY	backend/tests/bridge.test.js	Align with current persistent bridge and timeout contract before execution	Imports an obsolete module and expects the former 300,000 ms constant	Alignment required

Case ID	Repository target	Expected result	Observed evidence (15 Jul. 2026)	Status
WB-SMOK E	backend/scripts/test_worker_bri dge.js	Exercise cold catalog, warm catalog, and recoverable invalid-file error	Harness exists; no result is reported in this thesis run	Not rerun

The first three cases verify positive transformation behavior. GE-01 checks that the minimal two-node, one-link fixture survives conversion and all eleven structural validator stages. GE-02 checks a subtle exporter contract: configuration content must become a file inside the archive and the node property must become a pointer, while AI-only metadata must not leak into the GNS3 JSON. GE-03 applies the same principle to VPCS startup scripts and confirms that nested unsupported metadata is removed.

The two negative cases are equally important. GE-04 confirms that an appliance requiring an unavailable VM execution context is rejected instead of producing a

misleading portable archive. GE-05 confirms that a Dynamips router without a calibrated image is rejected. These tests demonstrate fail-closed behavior at two environment boundaries, but they do not establish compatibility with every appliance in the catalog.

The bridge evidence is intentionally qualified. `backend/tests/bridge.test.js` describes a useful legacy specification, but it imports the earlier wrapper module and asserts the obsolete 300,000 ms timeout. Reporting it as a current passing suite would be inaccurate. The persistent-worker smoke harness targets the current bridge and covers cold start, warm reuse, and a recoverable error, but it was not rerun for this document revision. These distinctions preserve a clear boundary between executed evidence, inspectable test intent, and work still required.

Failure-mode analysis groups defects by the boundary that should detect them. Input-shape defects belong to Zod or Pydantic validation. Hardware-capacity and port-collision defects belong to deterministic topology processing. Configuration-to-node mismatch belongs to the Three-Gate Safe Merge. Archive schema, path, and cross-reference defects belong to the eleven-check validator. Provider timeouts and worker termination belong to bridge lifecycle handling. Browser disconnects belong to SSE recovery and persisted resource retrieval.

This classification helps prevent duplicated or misplaced tests. For example, a frontend component test should verify that a failed export is rendered correctly,

but it should not be the only proof that the exporter rejects a missing image.

Conversely, a Python unit test can verify the rejection rule but cannot prove that the user receives a comprehensible recovery message. Confidence emerges from complementary checks across the boundary, not from increasing the count within one layer.

Remaining high-value verification work includes aligning the Jest bridge specification with `ai-engine.bridge.js`, adding API tests against an isolated MongoDB instance, exercising authentication ownership failures, testing SSE reconnection, importing golden archives into desktop GNS3 with licensed images, and collecting reproducible provider-dependent runs. Until those activities are completed, the thesis should describe the verified deterministic core as strong while presenting full end-to-end and multi-user reliability as incomplete.

6.9 Usability Evaluation

No participant survey dataset was present in the repository; therefore, this thesis does not invent user scores. A task-based expert inspection was performed against visibility of system status, consistency, error prevention, user control, recognition rather than recall, and recovery guidance. Appendix G provides visual evidence and Appendix H records three end-to-end task sequences. [41]

The landing and chat pages make the primary design action visible.

Conversational input reduces the need to memorize command syntax. Streaming events and the action trace expose long-running system state. The topology canvas

supports inspection through visual grouping and device details. Security-profile confirmation prevents accidental export before policy selection. Structured errors support recovery, although participant comprehension and satisfaction remain unmeasured.

6.9.1 Participant Study Protocol

A future empirical study should recruit at least five representative networking students or practitioners. Participants should complete topology creation, modification, inspection, and export tasks. The study should record completion rate, time on task, error count, assistance requests, and post-task satisfaction, followed by the ten-item System Usability Scale. Results must be reported only after informed participation and data collection; this protocol is not presented as a completed experiment.

6.9.2 Proposed Analysis and Reporting Rules

No participant study results are available for this thesis revision. Consequently, no completion percentages, satisfaction scores, charts, quotations, or statistical claims are reported. Table 6.8 defines the analysis plan that should be used if the protocol in Section 6.9.1 is approved and executed. Establishing the plan before data collection reduces the temptation to select only favorable outcomes.

Table 6.8: Proposed Usability Measures and Reporting Rules

Measure	Collection method	Planned reporting	Interpretation constraint
Task completion	Observe whether each defined task reaches its success condition without facilitator intervention	Completed/attempted count per task and overall rate	A small convenience sample does not establish population-wide effectiveness
Time on task	Record from task start to the predefined success condition	Median, range, and individual values	Model latency must be separated from user interaction time

Measure	Collection method	Planned reporting	Interpretation constraint
Interaction errors	Count invalid actions, unrecovered states, and requests for help using a coding guide	Frequency by task and error category	The coding guide must be fixed before sessions begin
Perceived usability	Post-task ratings and a short standardized questionnaire	Item distributions and aggregate score only when scoring rules are satisfied	Do not report a score without complete participant responses

Measure	Collection method	Planned reporting	Interpretation constraint
Qualitative feedback	Brief semi-structured interview after the tasks	Anonymized themes with representative short excerpts	Themes are descriptive and should not be treated as statistical estimates
System reliability during study	Log provider errors, validation failures, SSE disconnects, and retries	Incident count linked to affected tasks	System failures must not be misclassified as participant mistakes

The primary usability outcome should be task completion because StructuraNet AI is intended to support an end-to-end workflow rather than only produce favorable impressions. Time on task provides supporting evidence but must be decomposed where possible: provider waiting time, validation or export time, and active interaction time represent different design problems. Error counts should use predefined categories so that a model failure, an unclear label, and a participant misunderstanding are not merged into one number.

Quantitative reporting should show individual observations or distributions when the sample is small. A mean alone can conceal one severe failure, while percentages based on five participants can appear more precise than the evidence supports. Any comparison with manual GNS3 construction requires the same topology task, equivalent starting knowledge, a defined success condition, and disclosure of prior GNS3 experience. Without those controls, the comparison should be described as exploratory.

Qualitative comments should be anonymized and grouped through a documented coding process. The report should distinguish comments about the interface from comments about generated network quality. Participants must be informed that generated configurations require expert review and that the study is evaluating a prototype. Consent, institutional requirements, storage duration, and access to recordings or notes must be resolved before recruitment.

Charts should be generated only after real data are collected and checked. Suitable visualizations include a task-by-task completion bar chart, a dot plot of time on task, and a categorized error count. Every chart should state the sample size and unit of analysis. Empty placeholders or simulated observations should not appear in the final thesis because they could be mistaken for empirical results.

6.10 Chapter Summary

This chapter documents current evidence and gaps. The repository contains five Python golden-export cases, runtime Pydantic validation, an eleven-check

project validator, a manual worker harness, and a legacy Jest-style bridge suite requiring alignment. Authentication, frontend, end-to-end, load, and security-profile behavior need dedicated automated suites.

The architecture provides deterministic repair, worker reuse, guarded configuration merging, and SSE forwarding. Their effectiveness and latency have not been quantified with a committed reproducible dataset. Future evaluation should publish prompts, environment, model settings, raw outputs, timing samples, and criteria.



CHAPTER

7

Conclusion and Future Work

Project outcomes, principal contributions, limitations, future improvements, and opportunities for continued research.

Chapter 7: Conclusion and Future Work

7.1 Conclusion

This graduation project designed, implemented, and evaluated StructuraNet AI, a web platform that transforms natural-language network descriptions into structurally validated, simulation-ready GNS3 topologies. It addresses the manual effort involved in selecting devices, constructing links, assigning hardware-compatible interfaces, and preparing project artifacts. The implementation demonstrates how LLM function calling can support network design when deterministic software constrains, repairs, and validates model output.

The platform separates the interactive frontend, backend services and persistence, and the Python AI Engine. JSONL communication connects the backend to a persistent worker, while SSE communicates incremental status to the browser. The detailed technologies and versioned dependencies remain documented in Chapter 4 rather than being repeated in this conclusion.

7.2 Summary of Key Contributions

The primary contributions of this project can be summarized across five dimensions. First, the AI pipeline architecture introduces a novel separation between logical design (LLM-driven) and physical configuration (deterministic Python code), embodied in the two-phase pipeline where Phase 1 generates the network topology and Phase 2 generates software configurations. This separation ensures that the LLM's strengths in logical reasoning are leveraged while its weaknesses in precise numerical configuration are avoided entirely. The three-gate

safe merge mechanism provides a structural guarantee that Phase 2 LLM output can never corrupt Phase 1 hardware properties, regardless of what the model returns.

Second, the auto-repair pipeline applies deterministic correction for duplicate connections, single-port violations, and disconnected graph components before requesting another LLM attempt. Third, the persistent worker avoids repeated interpreter and module-import work after startup. Both decisions are technically supported by the implementation, but percentage effectiveness and latency improvements require a reproducible benchmark dataset and are not claimed here.

Fourth, the GNS3 export pipeline produces standards-compliant `.gns3project` ZIP archives that pass an eleven-check structural validator covering ZIP integrity, JSON schema conformity, node validation, Dynamips hardware compatibility, port reference integrity, configuration file path consistency, template ID format, compute cross-referencing, switch VLAN sanity, link integrity, and UUID format validation. The validator's three-tier severity classification (CRITICAL, ERROR, WARNING) provides actionable feedback that distinguishes between fatal archive problems, structural import failures, and non-fatal advisory issues. Fifth, the interactive user interface provides a complete design environment with real-time SSE streaming of the AI's reasoning process, a D3.js-powered topology visualization with six-tier hierarchical layout, iterative editing capabilities, GNS3 image calibration, three security profile tiers, and a download kit containing the `.gns3project` archive, standalone configuration files, and an image requirements manifest.

7.3 Achievements Relative to Project Objectives

The project implements the principal stated objectives: natural-language topology requests, hybrid routing to generation and editing tools, deterministic hardware and port handling, GNS3 export, security-profile-aware configuration generation, visualization, and session persistence. The golden-export tests and structural validator demonstrate selected artifact-integrity properties. General topology success rates and universal GNS3 importability still require evaluation across a documented prompt corpus, appliance-image matrix, provider, and GNS3 environment.

The visual design system provides a consistent chat, topology, onboarding, and export experience. The D3.js viewer supports inspection and interaction, the calibration flow records locally available GNS3 images, and MongoDB preserves sessions and topology history. Current automated evidence consists of the five rerun golden-export cases; runtime validators provide additional safeguards, while the legacy Jest bridge specification requires alignment and rerun before it can be reported as passing.

7.4 Challenges Encountered and Lessons Learned

Several significant challenges were encountered during the development of StructuraNet AI, and each provided valuable lessons for the team. The most pervasive challenge was the inherent non-determinism of LLM output. Early prototypes that asked the LLM to produce complete GNS3 configurations (including port numbers, slot modules, and hardware properties) failed frequently

because the LLM would assign incorrect adapter numbers, hallucinate non-existent modules, or produce inconsistent interface references. The solution, separating logical design from physical configuration and making the LLM responsible only for the former, was the single most impactful architectural decision in the project. This lesson reinforced the principle that LLMs should be used for tasks requiring reasoning and pattern recognition, not for tasks requiring precise numerical computation or adherence to rigid format specifications. [45]

A second major challenge was maintaining consistency between hardware constants across multiple Python modules. Early in development, the `schema.py` file defined its own port limit values that gradually drifted from the canonical values in `hardware.py`, causing the Pydantic validator to accept topologies that the port assigner could not physically realize. The solution was to establish a single source of truth in `constants/hardware.py` and refactor all other modules to import from it, with `constants/validation.py` serving as a backward-compatible re-export shim. This experience highlighted the importance of the DRY (Don't Repeat Yourself) principle in systems that involve multiple validation layers operating on the same data.

A third challenge was the Node.js-to-Python communication boundary. The initial implementation spawned a Python process and repeated module initialization for every API call. The persistent worker removed that repeated work but introduced lifecycle, sequential-processing, stdout-isolation, and request-correlation concerns. Capturing the real stdout stream and protecting writes with a

lock demonstrated that polyglot communication requires explicit ownership of shared process resources.

Additional challenges included the complexity of GNS3's .gns3project format (which requires specific UUID formats, pointer-based configuration references, and platform-specific property schemas), the difficulty of generating valid Cisco IOS configurations for diverse topology patterns (which was addressed by the Configuration Brief abstraction in `context_builder.py`), and the need to handle edge cases such as unsupported hardware requests, conflicting physical constraints, and out-of-scope user inputs gracefully. Each of these challenges was addressed through a combination of domain-specific constraints in LLM prompts, deterministic post-processing pipelines, and user-facing clarification protocols.

7.5 Future Work

While StructuraNet AI has achieved its core objectives, several directions for future enhancement have been identified through the development process and testing evaluation. These future work items are organized by priority and feasibility.

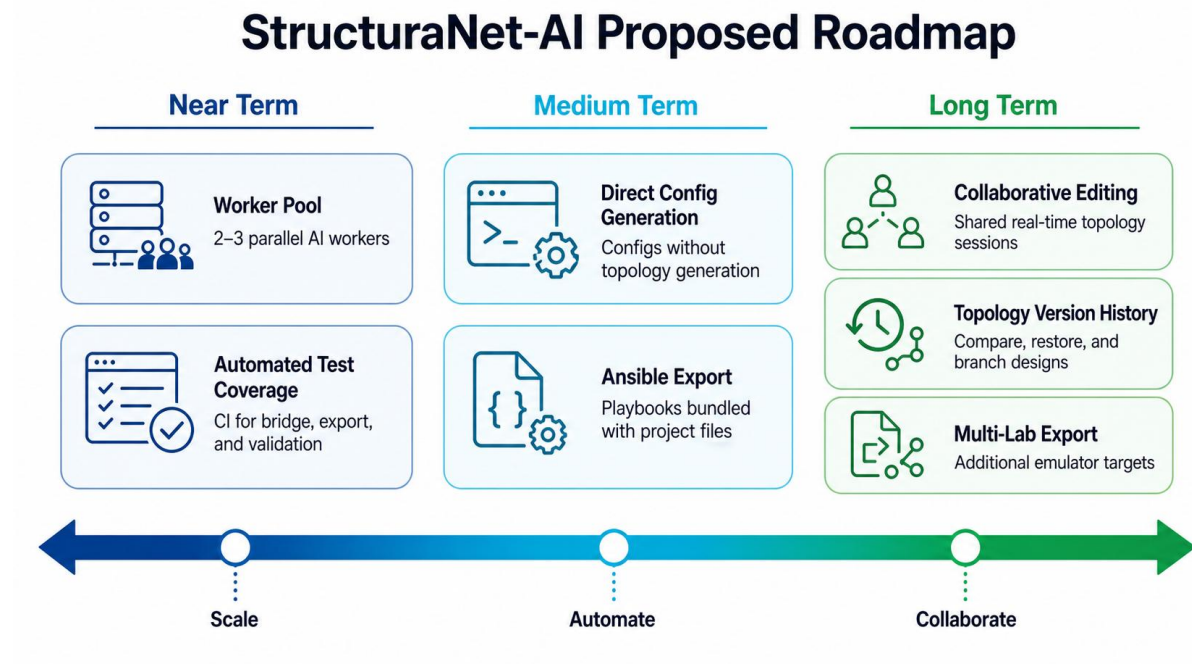


Figure 7.1: StructuraNet AI proposed roadmap

7.5.1 Multi-Provider LLM Support

The current system uses a single OpenAI-compatible API endpoint for all LLM interactions, configured via the `ROUTER_BASE_URL` and `ROUTER_API_KEY` environment variables. A valuable enhancement would be to support multiple LLM providers simultaneously, allowing the system to route different pipeline stages to different models based on cost, latency, and capability. For example, a fast, inexpensive model could be used for intent classification and clarification responses, while a more capable model could be reserved for topology generation and configuration generation. The OpenAI SDK's multi-client architecture already supports this pattern; the primary work would be in extending the `llm_utils.py` module to maintain a provider pool and adding configuration options to the backend's settings interface.

7.5.2 Worker Pool for Concurrent Processing

The persistent worker currently processes requests sequentially, as documented in `worker.py`'s concurrency model. While this is appropriate for the current single-user chat use case, it becomes a bottleneck in multi-user deployments where two users simultaneously request topology generation. The JSONL protocol already supports request-ID-based multiplexing, so the primary work would be to modify the Node.js bridge to maintain a small pool of two to three worker processes and implement round-robin request distribution. The worker's stateless-per-request design (all mutable state is local to each command handler) ensures that a pool-based architecture would not introduce state-leak bugs, as confirmed by the state-leak audit in the worker's warm-up documentation.

7.5.3 Additional Simulation Platform Support

The current export pipeline targets GNS3 version 2.2.x exclusively. Two natural extensions would be to support EVE-NG (which uses a similar ZIP-based project format with YAML topology definitions) and Cisco DevNet Sandbox (which provides cloud-based Cisco device simulation). The modular architecture of the export subsystem, particularly the clean separation between the topology data model (`GNS3Project`) and the file format assembler (`gns3_exporter.py`), would facilitate adding an `eve_exporter.py` module that transforms the same validated topology dictionary into EVE-NG's expected format. The validator subsystem would similarly be extended with an EVE-NG-specific check suite, following the same severity classification pattern.

7.5.4 Automated Configuration Verification

The Phase 2 pipeline generates Cisco IOS configurations as text strings that are embedded in the .gns3project file but are not verified for syntactic or semantic correctness beyond the LLM's own output quality. A significant enhancement would be to integrate a configuration verification step using network automation tools such as Netmiko or NAPALM to parse the generated configurations and verify that they contain no syntax errors, that IP addresses are consistent across connected interfaces, that routing protocols reference correct network statements, and that security features (ACLs, ZBF zones, NAT rules) are internally consistent. This verification could be implemented as an optional Phase 3 in the pipeline, with the results presented to the user as a configuration health report alongside the export download kit.

7.5.5 Advanced Topology Patterns

The current system supports a wide range of enterprise campus and branch network topologies, as demonstrated by the four reference wiring patterns (single-site enterprise, multi-branch, Router-on-a-Stick, and NAT intermediary) embedded in the Phase 1 system prompt. Future enhancements could extend this coverage to more advanced patterns including Software-Defined WAN (SD-WAN) hub-and-spoke topologies with overlay tunnel configuration, Cisco Software-Defined Access (SDA) fabric layouts with fabric edge, fabric border, and control plane nodes, data center spine-leaf architectures with VXLAN BGP EVPN configuration, and wireless LAN controller (WLC) integration with lightweight access point

topologies. Each pattern would require new wiring rules in the system prompt, new device templates in the appliance catalog, and potentially new configuration generation rules in the context builder.

7.5.6 Collaboration and Version Control Features

The current session model is single-user, with each user's sessions and topologies stored independently. A natural extension for academic and team environments would be to add collaboration features, including shared sessions where multiple users can contribute to the same topology design, topology versioning with diff and merge capabilities (similar to Git but for network topologies), and team workspaces with role-based access control. The MongoDB document model already supports embedding messages and topology snapshots within sessions; extending this to support multiple user references and version history would be a straightforward schema evolution. The real complexity would be in the conflict resolution logic for concurrent edits and the real-time collaboration protocol, which could leverage the existing SSE infrastructure with operational transform or CRDT-based conflict resolution.

7.5.7 Frontend Enhancements and Mobile Support

While the React frontend provides a fully functional desktop experience, several enhancements would improve usability and accessibility. Responsive design improvements would adapt the chat interface and topology viewer for tablet and mobile form factors, enabling network engineers to review and iterate on designs from any device. The topology viewer could be enhanced with additional

visualization modes, including a physical rack view showing devices in their rack-mounted configuration, a protocol-level view highlighting OSPF areas, EIGRP AS boundaries, or BGP autonomous systems, and an animated simulation view showing packet flow paths through the topology. The chat interface could benefit from template-based request shortcuts that provide guided network description forms for common topology patterns, reducing the need for free-form text input and improving consistency of results.

7.5.8 Comprehensive Testing and Continuous Integration

The current test suite provides strong coverage of the bridge communication layer, the export pipeline, and the schema validators, but several areas would benefit from additional testing. Frontend component tests using React Testing Library or Vitest would verify the chat interface, topology viewer, and onboarding flow in isolation. End-to-end tests using Playwright or Cypress would verify the complete user journey from registration through topology generation to GNS3 import. Load testing using k6 or Artillery would establish performance baselines and identify bottlenecks under concurrent user load. A continuous integration pipeline using GitHub Actions would automate test execution, linting, and deployment on every pull request, ensuring that the codebase remains in a releasable state at all times. These testing investments would be particularly important if the platform is deployed as a shared service for multiple users or institutions.

References

- [1] OpenAI, “GPT-4 Technical Report,” arXiv:2303.08774, 2023.
- [2] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in Proc. NeurIPS, 2022.
- [3] OpenAI, “Function Calling and Other API Updates,” Jun. 2023. [Online]. Available: <https://openai.com/index/function-calling-and-other-api-updates/> [Accessed: Jul. 15, 2026].
- [4] A. Q. Jiang et al., “Mistral 7B,” arXiv:2310.06825, 2023.
- [5] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” arXiv:2302.13971, 2023.
- [6] OpenRouter, “OpenRouter API Documentation.” [Online]. Available: <https://openrouter.ai/docs> [Accessed: Jul. 15, 2026].
- [7] OpenAI, “OpenAI API Reference.” [Online]. Available: <https://platform.openai.com/docs/api-reference> [Accessed: Jul. 15, 2026].
- [8] GNS3 Technologies Inc., “GNS3 Documentation.” [Online]. Available: <https://docs.gns3.com> [Accessed: Jul. 15, 2026].
- [9] Cisco Networking Academy, “Cisco Packet Tracer.” [Online]. Available: <https://www.netacad.com/cisco-packet-tracer> [Accessed: Jul. 15, 2026].
- [10] EVE-NG Ltd., “EVE-NG Documentation.” [Online]. Available: <https://www.eve-ng.net> [Accessed: Jul. 15, 2026].

- [11] NAPALM Contributors, “NAPALM Documentation.” [Online]. Available: <https://napalm.readthedocs.io> [Accessed: Jul. 15, 2026].
- [12] J. Edelman, S. S. Lowe, and M. Oswalt, Network Programmability and Automation. Sebastopol, CA, USA: OReilly Media, 2018.
- [13] K. Byers et al., “Netmiko Documentation.” [Online]. Available: <https://ktbyers.github.io/netmiko/> [Accessed: Jul. 15, 2026].
- [14] HashiCorp, “Terraform Documentation.” [Online]. Available: <https://developer.hashicorp.com/terraform> [Accessed: Jul. 15, 2026].
- [15] Red Hat, “Ansible Documentation.” [Online]. Available: <https://docs.ansible.com> [Accessed: Jul. 15, 2026].
- [16] Meta Platforms, Inc., “React Documentation.” [Online]. Available: <https://react.dev> [Accessed: Jul. 15, 2026].
- [17] OpenJS Foundation, “Express.js Documentation.” [Online]. Available: <https://expressjs.com> [Accessed: Jul. 15, 2026].
- [18] OpenJS Foundation, “Node.js Documentation.” [Online]. Available: <https://nodejs.org/docs/latest/api/> [Accessed: Jul. 15, 2026].
- [19] MongoDB, Inc., “MongoDB Documentation.” [Online]. Available: <https://www.mongodb.com/docs/> [Accessed: Jul. 15, 2026].
- [20] Automattic, Inc., “Mongoose Documentation.” [Online]. Available: <https://mongoosejs.com/docs/> [Accessed: Jul. 15, 2026].

- [21] WHATWG, “Server-Sent Events,” HTML Living Standard. [Online]. Available: <https://html.spec.whatwg.org/multipage/server-sent-events.html> [Accessed: Jul. 15, 2026].
- [22] Pydantic Services Inc., “Pydantic Documentation.” [Online]. Available: <https://docs.pydantic.dev> [Accessed: Jul. 15, 2026].
- [23] OpenAI, “OpenAI Python SDK.” [Online]. Available: <https://github.com/openai/openai-python> [Accessed: Jul. 15, 2026].
- [24] Observable, Inc., “D3.js: Data-Driven Documents.” [Online]. Available: <https://d3js.org> [Accessed: Jul. 15, 2026].
- [25] M. Bostock, V. Ogievetsky, and J. Heer, “D3 Data-Driven Documents,” IEEE Trans. Vis. Comput. Graph., vol. 17, no. 12, pp. 2301–2309, Dec. 2011.
- [26] Tailwind Labs, “Tailwind CSS Documentation.” [Online]. Available: <https://tailwindcss.com/docs> [Accessed: Jul. 15, 2026].
- [27] P. Daishi and contributors, “Zustand Documentation.” [Online]. Available: <https://github.com/pmndrs/zustand> [Accessed: Jul. 15, 2026].
- [28] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” J. ACM, vol. 22, no. 2, pp. 215–225, Apr. 1975.
- [29] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, May 2015.

- [30] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Signature (JWS),” RFC 7515, May 2015.
- [31] N. Provos and D. Mazieres, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annu. Tech. Conf., 1999.
- [32] E. You and contributors, “Vite Documentation.” [Online]. Available: <https://vite.dev> [Accessed: Jul. 15, 2026].
- [33] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, Dec. 2017.
- [34] JSON Lines Contributors, “JSON Lines.” [Online]. Available: <https://jsonlines.org> [Accessed: Jul. 15, 2026].
- [35] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Pearson, 2021.
- [36] W. Stallings, Data and Computer Communications, 12th ed. Pearson, 2023.
- [37] Cisco Systems, “Cisco IOS Configuration Fundamentals.” [Online]. Available: <https://www.cisco.com/c/en/us/support/ios-nx-os-software/ios-15-4m-t/product-installation-guides-list.html> [Accessed: Jul. 15, 2026].
- [38] R. T. Fielding, “Architectural Styles and the Design of Network-Based Software Architectures,” Ph.D. dissertation, Univ. California, Irvine, 2000.
- [39] Parallax, “jsPDF Documentation.” [Online]. Available: <https://github.com/parallax/jsPDF> [Accessed: Jul. 15, 2026].

- [40] P. N. Johnson-Laird, “Mental Models and Human Reasoning,” *Proc. Natl. Acad. Sci.*, vol. 107, no. 43, pp. 18243–18250, 2010.
- [41] J. Nielsen, *Usability Engineering*. Morgan Kaufmann, 1994.
- [42] B. Shneiderman et al., *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 6th ed. Pearson, 2016.
- [43] OWASP Foundation, “OWASP Top 10 Web Application Security Risks,” 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/> [Accessed: Jul. 15, 2026].
- [44] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Wiley, 2011.
- [45] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018.
- [46] D. Flanagan, *JavaScript: The Definitive Guide*, 7th ed. OReilly Media, 2020.
- [47] G. van Rossum and F. L. Drake, *Python 3 Reference Manual*. CreateSpace, 2009.

Appendices

This section provides supplementary technical reference material that supports the main body of the thesis. The appendices contain detailed data tables, protocol specifications, schema definitions, and sample outputs that would otherwise disrupt the narrative flow of the core chapters. Each appendix is self-contained and can be

consulted independently for specific technical details about the StructuraNet AI system's internal workings, device support catalog, validation logic, and export formats.

Appendix A: Complete GNS3 Device Catalog

StructuraNet AI maintains a comprehensive static appliance catalog that defines every GNS3 device type the system can instantiate. This catalog serves as the single source of truth for device properties, ensuring that every generated topology produces valid GNS3 nodes that can be imported without modification. The catalog data is sourced directly from the GNS3 server source code (version 2.2.x), including the schema definition files for each node type and the official GNS3 appliance registry (.gns3a files) for QEMU and Docker-based devices. Each catalog entry contains the mandatory creation properties required by the GNS3 server, including platform identifiers, disk images, RAM allocations, adapter counts, console types, and interface naming conventions.

The catalog is organized into six categories: Dynamips (legacy Cisco router emulation using real IOS images), IOU (Cisco IOS on Unix), built-in GNS3 nodes (VPCS, Ethernet Switch, Ethernet Hub, Cloud, NAT, Frame Relay Switch, ATM Switch), QEMU-based appliances (modern Cisco, Juniper, Arista, firewall, and general-purpose devices), Docker containers, and traffic generators. The following table presents the complete catalog of 45 supported device templates with their key configuration parameters.

Table A.1 — Complete StructuraNet AI GNS3 Device Catalog

Template Name	Type	Category	RAM (MB)	Max Ports	Console	Port Format
Cisco 7200	Dynamips	c7200	512	49	telnet	FastEthernet{0}/{1}
Cisco 3745	Dynamips	c3745	128	6	telnet	FastEthernet{0}/{1}
Cisco 3725	Dynamips	c3725	128	4	telnet	FastEthernet{0}/{1}
Cisco 3660	Dynamips	c3600 (3660)	192	26	telnet	FastEthernet{0}/{1}
Cisco 3640	Dynamips	c3600 (3640)	128	16	telnet	Ethernet{0}/{1}
Cisco 2691	Dynamips	c2691	192	3	telnet	FastEthernet{0}/{1}
Cisco 2600	Dynamips	c2600	160	2	telnet	FastEthernet{0}/{1}
Cisco 1700	Dynamips	c1700	160	1	telnet	FastEthernet{0}
IOU L3	IOU	L3 Router	256	64	telnet	Ethernet{0}/{1}
IOU L2	IOU	L2 Switch	256	64	telnet	Ethernet{0}/{1}
VPCS	Built-in	VPCS	—	1	telnet	Ethernet{0}
Ethernet Switch	Built-in	Switch	—	8	none	Ethernet{0}
Ethernet Hub	Built-in	Hub	—	8	—	Ethernet{0}
Cloud	Built-in	Cloud	—	Dynamic	—	Ethernet{0}
NAT	Built-in	NAT	—	1	—	nat{0}

Template Name	Type	Category	RAM (MB)	Max Ports	Console	Port Format
Frame Relay Switch	Built-in	FR Switch	—	Dynamic	—	Serial{0}
ATM Switch	Built-in	ATM Switch	—	Dynamic	—	ATM{0}
Cisco CSR1000v	QEMU	Router	4096	4	telnet	GigabitEthernet{port1}
pfSense	QEMU	Firewall	2048	6	vnc	em{0}
Alpine Linux	QEMU	Linux	128	1	telnet	eth{0}
OpenWrt	QEMU	Router	128	4	telnet	eth{0}
FRRouting	QEMU	Router	256	8	telnet	eth{0}
OVS	QEMU	Switch	256	8	telnet	eth{0}
Cisco IOSv	QEMU	Router	512	4	telnet	GigabitEthernet0/{0}
Cisco IOSv-L2	QEMU	Switch	1024	8	telnet	GigabitEthernet0/{0}
Cisco ASAv	QEMU	Firewall	2048	4	telnet	GigabitEthernet0/{0}
Cisco Nexus 9000v	QEMU	Switch	8192	8	telnet	Ethernet1/{0}
Cisco Catalyst 8000v	QEMU	Router	4096	4	telnet	GigabitEthernet1/{0}
Juniper vMX	QEMU	Router	4096	4	telnet	ge-0/0/{0}
Juniper vSRX	QEMU	Firewall	4096	4	telnet	ge-0/0/{0}
Arista vEOS	QEMU	Switch	2048	8	telnet	Ethernet1/{0}

Template Name	Type	Category	RAM (MB)	Max Ports	Console	Port Format
Cumulus VX	QEMU	Switch	1024	4	telnet	swp{0}
FortiGate	QEMU	Firewall	1024	4	telnet	port{0}
Palo Alto VM-100	QEMU	Firewall	4096	4	telnet	ethernet1/{0}
Check Point Gaia	QEMU	Firewall	4096	4	vnc	eth{0}
VyOS	QEMU	Router	1024	4	telnet	eth{0}
Dell OS10	QEMU	Switch	4096	8	telnet	ethernet1/1/{0}
Ubuntu	QEMU	Linux	2048	1	vnc	ens{0}
Kali Linux	QEMU	Linux	2048	1	vnc	eth{0}
Windows	QEMU	Windows	4096	1	vnc	Ethernet{0}
Ostinato	QEMU	Traffic Gen.	1024	4	vnc	eth{0}
TRex	QEMU	Traffic Gen.	2048	4	telnet	eth{0}
Alpine Docker	Docker	Linux	—	1	telnet	eth{0}
FRR Docker	Docker	Router	—	4	telnet	eth{0}
OVS Docker	Docker	Switch	—	8	telnet	eth{0}

Note: Max Ports for Dynamips devices computed as builtin_ports + (ports_per_module x max_slots). IOU max = 16 adapters x 4 ports. QEMU/Docker max per adapter is 8. Built-in types have fixed or dynamic port counts.

Appendix B: AI Engine Worker Command Protocol

The StructuraNet AI Engine employs a persistent worker process architecture for communication between the Node.js backend and the Python AI Engine. Rather than spawning a new Python interpreter for each user request (which would incur startup latency of several seconds), the system maintains a long-running Python worker process that is started on first use and then kept alive for the lifetime of the Node.js process. Communication between the two processes uses a JSONL (JSON Lines) protocol over stdin/stdout, where each line is a self-contained JSON object. This design ensures minimal latency for topology generation requests while maintaining clean process isolation between the JavaScript and Python runtimes.

The protocol supports a request-response pattern with optional streaming events. The Node.js backend sends a command request as a single JSON line on the worker's stdin, and the worker processes the command and sends back a response on stdout. For long-running commands such as topology generation, the worker can emit intermediate event objects (thought, phase_change) before the final response, enabling real-time progress updates to the user via the SSE layer. The following table documents all supported commands, their input arguments, and output format.

Table B.1 — AI Engine Worker Command Protocol Specification

Command	Arguments	Returns	Description
generate	user_request, devices, security_profile	GNS3Project JSON with topology + configs	Runs Phase 1 only: natural-language interpretation, topology synthesis, validation, repair, and persistence. Phase 2 configuration generation and GNS3 packaging are performed separately by the export command.
edit	topology_dict, edit_instruction, devices	Modified GNS3Project JSON	Applies a natural-language edit to an existing topology.
export	topology_dict, config_review_dir	ZIP file path (.gns3project)	Converts topology JSON to GNS3 portable project ZIP.
qa	question, topology_dict	Answer string	Answers questions about a generated topology.
validate	topology_dict	Validation result with pass/fail list	Runs 11-check structural validation on topology.
manifest	topology_dict, template_image_map, output_path	Image manifest file path	Generates a list of required disk images.
brief	topology_dict	Topology summary text	Returns a human-readable topology description.
catalog	none	APPLIANCE_CATALOG dictionary	Returns the full device catalog for frontend use.

Command	Arguments	Returns	Description
ping	none	{"success": true}	Health check. Confirms worker is alive and responsive.
shutdown	none	None (process exits)	Gracefully terminates the worker process.

B.1 Request and Response Format

Every request from Node.js to the Python worker follows a standardized JSON envelope format. Each request includes a unique identifier (the same ID is echoed back in the response for correlation), a command string that selects the handler, and an arguments object containing command-specific parameters. The response envelope similarly includes the request ID, a boolean success flag, and either a result object (on success) or an error object with message and optional details field (on failure). The following example illustrates the generate command request and a successful response:

```
Request format: {"id": "<request-id>", "command": "<cmd>", "args": {<command-specific>}}
```

```
Success response: {"id": "<request-id>", "ok": true, "result": {...}}
```

```
Error response: {"id": "<request-id>", "ok": false, "error": {"message": "...", "details": "..."} }
```

During long-running commands (particularly generate and edit), the worker emits intermediate event envelopes that provide real-time progress information. These events include thought events containing the LLM's chain-of-thought reasoning text, and phase_change events indicating transitions between pipeline stages (e.g., from Phase 1 topology generation to Phase 2 configuration generation). The Node.js bridge forwards these events to the SSE service, which streams them to the client browser for display in the chat interface.

Appendix C: Eleven-Check Structural Validation Specifications

The `validator.py` module implements a comprehensive 11-check deep structural validation pipeline that simulates what GNS3's own import logic checks when opening a `.gns3project` file. This validator goes far beyond simple JSON schema validation to catch the subtle bugs that cause GNS3 to reject or silently misconfigure imported projects. Each check targets a specific category of structural integrity, and the checks are ordered from fastest (ZIP structure) to most computationally expensive (connectivity analysis). The following table provides a detailed specification of each validation check, including its severity classification, the specific conditions it tests, and example failure messages.

Table C.1 — Eleven-Check Structural Validation Specification

#	Check Name	Severity	Description	Example Failure
1	ZIP Structure	Error	Verifies that project.gns3 exists at the root of the ZIP archive, that the project-files/ directory exists, and that all referenced file paths within the JSON point to entries that actually exist in the ZIP.	ZIP missing project.gns3
2	JSON Schema	Error	Validates top-level JSON structure: must have revision=7, type=license, and required keys (name, topology, nodes, links). Ensures the file can be parsed as valid JSON.	Missing required key: topology
3	Node Validation	Error	Checks every node has required fields (node_id, name, node_type, properties), that node_type is one of the 14 legal types, and that type-specific required properties are present (e.g., platform for dynamips).	Invalid node_type: 'cisco'
4	Dynamips Compatibility	Error	Cross-references Dynamips node properties against the hardware compatibility matrix. Verifies slot module legality per platform, image filename patterns, and RAM/NVRAM values.	NM-4E not valid for c3745
5	Port Reference Integrity	Error	For every link, verifies that the referenced (adapter_number, port_number) pair actually exists on the target node. Catches stale port references from incorrect port assignment.	Port 3/7 does not exist on R1

#	Check Name	Severity	Description	Example Failure
6	Config File Path Consistency	Warning	Verifies that startup_config, private_config, and startup_script file references in node properties point to files that exist within the ZIP's project-files/ directory structure.	Config file not found in ZIP
7	Template ID Format	Warning	Appliance-type nodes (dynamips, iou, qemu, docker) must have a valid UUID template_id. Built-in types (switch, hub, cloud, nat) must NOT have a template_id.	Missing template_id for qemu node
8	Compute Cross-Reference	Info	Verifies that compute_id references are valid. 'local' is always accepted. Remote compute references must appear in the project's computes list.	Unknown compute: 'remote-1'
9	Switch VLAN Sanity	Warning	Checks that ethernet_switch ports_mapping entries have valid VLAN assignments: access ports must use VLAN 1-4094, trunk ports must use dot1q encapsulation type.	VLAN 5000 out of range
10	Link Integrity	Error	Detects duplicate links (same node pair + same link_type), self-links (a node connected to itself), and links referencing non-existent node IDs.	Duplicate ethernet link: R1-R2
11	UUID Format	Warning	Validates that all node_id, link_id, and project_id fields conform to UUID v4 format (8-4-4-4-12 hex pattern). Catches malformed IDs from LLM generation errors.	Invalid UUID: 'R1'

The validator classifies findings into four severity levels: Error (must be fixed before export), Warning (should be reviewed but does not prevent GNS3 import), Info (informational notices), and Pass (check completed successfully). The auto-repair pipeline in `agent.py` automatically addresses the most common failure modes (disconnected graphs, single-port violations, and duplicate connections) before validation is applied. The repository does not contain a dated prompt corpus from which a first-attempt success rate can be calculated.

Appendix D: Pydantic Schema Definitions

The StructuraNet AI Engine uses Pydantic v2 models to enforce a strict contract between the LLM output and the downstream pipeline stages. The schema definitions serve two critical purposes: they provide structured validation that catches malformed LLM output before it reaches the port assigner or hardware injector, and they document the exact data contract that each pipeline stage expects. The schema module (`schema.py`) defines two families of models: the Step-1 models that represent the raw LLM output (`TopologyRequest`, `NodeRequest`, `Connection`), and the final models that represent the fully processed topology with port numbers assigned (`Topology`, `Node`, `Link`, `GNS3Project`). The following table summarizes the key schema models and their validation rules.

Table D.1 — Pydantic Schema Model Definitions

Model	Stage	Key Fields	Validation Rules
Connection	Step-1 LLM Output	from_node, to_node, link_type	Endpoints must differ (no self-loops). link_type is 'ethernet' or 'serial'.
NodeRequest	Step-1 LLM Output	node_id, name, node_type, template_name, compute_id	node_type must be one of 14 legal types. compute_id defaults to 'local'.
TopologyRequest	Step-1 LLM Output	name, nodes[], connections[]	Unique node_ids. Connections must reference existing nodes. No duplicate connections. Single-port nodes limited to 1 connection. Topology must be connected (Union-Find).
LinkNode	Final Topology	node_id, adapter_number, port_number	adapter_number ≥ 0 , port_number ≥ 0 . References an existing node.
Link	Final Topology	nodes[2], link_type	Exactly 2 endpoints. Endpoints must differ. No duplicate links per (pair, type).
Node	Final Topology	node_id, name, node_type, template_name, properties {}, compute_id	Same 14 legal node_types. Properties dict carries hardware config.

Model	Stage	Key Fields	Validation Rules
Topology	Final Topology	nodes[], links[]	All NodeRequest validators plus: port collision detection, adapter/port bounds checking per platform, single-port adapter enforcement, and max port count per node type.
GNS3Project	Export Wrapper	name, topology (Topology)	Wraps the final Topology under a project name for export.

A critical design decision in the schema is the separation between TopologyRequest (Step-1) and Topology (Final). The Step-1 model validates the LLM's structural output (devices and logical connections) without port numbers, since port assignment is performed by the dedicated `port_assigner.py` module after the LLM output passes validation. The Final Topology model then validates the complete result including port assignments, catching bugs such as port collisions (two links using the same adapter/port on a node), out-of-bounds port numbers (e.g., port 5 on a c1700 which only has 1 built-in port), and incorrect adapter usage (e.g., serial links on adapter 0 which has no built-in serial ports on any Dynamips platform).

Appendix E: Dynamips Hardware Compatibility Matrix

The Dynamips emulator, which provides legacy Cisco router support in GNS3, requires precise hardware configuration including platform-specific slot modules, RAM/NVRAM allocations, and compatible adapter matrices. The `hardware.py` module in the StructuraNet AI Engine serves as the single source of truth for all

hardware constants, verified against the GNS3 source code (version 2.2.59). This appendix documents the complete compatibility matrix showing which Network Modules and Port Adapters are valid for each Dynamips platform, along with the default hardware parameters used by the system. Understanding this matrix is essential for diagnosing topology generation failures related to unsupported module combinations or incorrect slot configurations.

Table E.1 — Dynamips Platform Hardware Compatibility Matrix

Device	Platform	Slot 0 (Fixed)	Valid Modules	RAM / NVRAM (MB)	Slot Layout	Max Eth Ports
Cisco 7200	c7200	C7200-IO-FE / IO-2FE / IO-GE-E	PA-A1, PA-FE-TX, PA-2FE-TX, PA-GE, PA-4T+, PA-8T, PA-4E, PA-8E, PA-POS-OC3	512 / 512	1 (IO-FE) + 6 slots	49
Cisco 3745	c3745	GT96100-FE (fixed)	NM-1FE-TX, NM-4T, NM-16ESW	128 / 256	2 (GT96100) + 4 slots	6
Cisco 3725	c3725	GT96100-FE (fixed)	NM-1FE-TX, NM-4T, NM-16ESW	128 / 256	2 (GT96100) + 2 slots	4
Cisco 3660	c3600	Leopard-2FE (fixed)	NM-1FE-TX, NM-1E, NM-4E, NM-16ESW, NM-4T	192 / 256	2 (Leopard) + 6 slots	26
Cisco 3640	c3600	None (all configurable)	NM-1FE-TX, NM-1E, NM-4E, NM-16ESW, NM-4T	128 / 256	4 configurable slots	16

Device	Platform	Slot 0 (Fixed)	Valid Modules	RAM / NVRAM (MB)	Slot Layout	Max Eth Ports
Cisco 2691	c2691	GT96100-FE (fixed)	NM-1FE-TX, NM-4T, NM-16ESW	192 / 256	2 (GT96100) + 1 slot	3
Cisco 2600	c2600	C2600-MB-1E/2E/1FE/2FE	NM-1FE-TX, NM-1E, NM-4E, NM-16ESW	160 / 128	1 (motherboard) + 1 slot	2
Cisco 1700	c1700	C1700-MB-1FE (fixed)	None (no NM bay)	160 / 128	1 (C1700-MB-1FE) only	1

Note: RAM/NVRAM values are GNS3 defaults. Max Ethernet Ports computed as builtin + (ports_per_module x max_slots). c2600 and c1700 have no serial NM modules (serial only via WIC, not modeled). NM-1T has been removed as it does not exist in GNS3's ADAPTER_MATRIX.

E.1 Module Interface Reference

Each hardware module has a specific interface prefix and port count that determines the Cisco IOS interface names generated in device configurations. The port_assigner module uses these mappings to produce correct interface names such as FastEthernet0/0, Serial0/0, GigabitEthernet0/0, and so on. The following table lists all supported modules and their interface characteristics, which are defined in the DYNAMIPS_MODULE_INTERFACES dictionary within hardware.py.

Table E.2 — Module Interface Name and Port Count Reference

Module Name	Interface Prefix	Port Count	Description
PA-8E	Ethernet	8	C7200 Port Adapter
PA-4E	Ethernet	4	C7200 Port Adapter

Module Name	Interface Prefix	Port Count	Description
PA-FE-TX	FastEthernet	1	C7200 Port Adapter
PA-2FE-TX	FastEthernet	2	C7200 Port Adapter
PA-GE	GigabitEthernet	1	C7200 Port Adapter
PA-A1	ATM	1	C7200 Port Adapter
PA-POS-OC3	POS	1	C7200 Port Adapter
PA-4T+	Serial	4	C7200 Port Adapter
PA-8T	Serial	8	C7200 Port Adapter
NM-4E	Ethernet	4	C3600 Network Module
NM-1E	Ethernet	1	C3600/C2600 Network Module
NM-1FE-TX	FastEthernet	1	C3600/C3700 Network Module
NM-16ESW	FastEthernet	16	C3600/C3700 Switch Module
NM-4T	Serial	4	C3600/C3700 Network Module
GT96100-FE	FastEthernet	2	C3725/C3745/C2691 Built-in Chip
Leopard-2FE	FastEthernet	2	C3660 Built-in Chip
C7200-IO-FE	FastEthernet	1	C7200 I/O Controller
C7200-IO-2FE	FastEthernet	2	C7200 I/O Controller
C7200-IO-GE-E	GigabitEthernet	1	C7200 I/O Controller
C1700-MB-1FE	FastEthernet	1	C1700 Motherboard

Module Name	Interface Prefix	Port Count	Description
C2600-MB-1E	Ethernet	1	C2600 Motherboard
C2600-MB-2E	Ethernet	2	C2600 Motherboard
C2600-MB-1FE	FastEthernet	1	C2600 Motherboard
C2600-MB-2FE	FastEthernet	2	C2600 Motherboard

Appendix F: Sample GNS3 Export Format (Golden Topology)

This appendix presents the `golden_minimal_topology.json` fixture used in the test suite to verify the export pipeline. This minimal but valid topology demonstrates the core data structures that the StructuraNet AI Engine produces: a VPCS endpoint (PC1) connected to an Ethernet Switch (SW1) via an Ethernet link. The topology represents the simplest possible valid GNS3 project that exercises the port assignment, hardware injection, and export serialization logic. All automated tests verify that the export pipeline produces output structurally equivalent to this format when given equivalent input.

The JSON structure below shows the final topology format after all pipeline stages have been applied. The nodes array contains the device definitions with their assigned properties, including the VPCS startup script and the Ethernet Switch `ports_mapping` configuration. The links array contains the port-assigned connection between PC1 (adapter 0, port 0) and SW1 (adapter 0, port 0). This structure is what the `gns3_exporter.py` module transforms into the final `.gns3project` ZIP archive.

```
{
```

```
"name": "golden_minimal",
"topology": {
  "nodes": [
    {
      "node_id": "PC1",
      "name": "PC1",
      "node_type": "vpcs",
      "template_name": "VPCS",
      "compute_id": "local",
      "properties": {
        "startup_script": "ip 10.0.0.10/24 10.0.0.1\nsave\n"
      }
    },
    {
      "node_id": "SW1",
      "name": "SW1",
      "node_type": "ethernet_switch",
      "template_name": "Ethernet Switch",
      "compute_id": "local",
      "properties": {
        "ports_mapping": [
          {
            "name": "Ethernet0",
            "port_number": 0,
            "type": "access",
            "vlan": 1,
            "ethertype": ""
          }
        ]
      }
    }
  ],
  "links": [
    {
      "link_id": "L1",
      "link_type": "ethernet",
      "nodes": [
        { "node_id": "PC1", "adapter_number": 0, "port_number": 0 },
        { "node_id": "SW1", "adapter_number": 0, "port_number": 0 }
      ]
    }
  ]
}
```


F.1 Portable Project ZIP Structure

When the `gns3_exporter.py` module converts the topology JSON into a GNS3 portable project, it produces a ZIP archive with the following internal directory structure. The `project.gns3` file at the root contains the topology JSON wrapped in the GNS3 project metadata envelope (with `revision`, `type`, and `project_id` fields). Device configuration files are stored under `project-files/` organized by node type and UUID, following the exact path convention that GNS3's `import_project.py` controller expects. This structure has been verified against the GNS3 server source code (version 2.2.x) to ensure compatibility.

Table F.1 — GNS3 Portable Project ZIP Internal Structure

Path	Scope	Description
<code>project.gns3</code>	Root	Topology JSON with GNS3 metadata envelope (<code>revision</code> , <code>type</code> , <code>project_id</code> , <code>name</code> , <code>topology</code>)
<code>project-files/dynamips/<uuid>/configs/startup-config.cfg</code>	Per-node	Cisco IOS startup configuration for Dynamips routers
<code>project-files/iou/<uuid>/configs/startup-config.cfg</code>	Per-node	Cisco IOS startup configuration for IOU devices
<code>project-files/iou/<uuid>/configs/private-config.cfg</code>	Per-node	Cisco IOS private configuration (encrypted keys) for IOU devices
<code>project-files/vpcs/<uuid>/startup.vpc</code>	Per-node	VPCS startup script (IP address, default gateway)
<code>project-files/docker/<uuid>/startup.sh</code>	Per-node	Docker container startup script

The exporter module (`gns3_exporter.py`, version 4.6) handles several critical transformation steps during the ZIP creation process. First, it strips all StructuraNet-internal metadata keys (those prefixed with underscore, such as `_symbol`, `_category`, and `_hardware_summary`) from node properties before writing to the JSON. Second, it converts content-based configuration keys (`startup_config_content`, `private_config_content`, `startup_script`) into the appropriate file entries within the ZIP archive and replaces them with file-pointer references that GNS3 expects. Third, it ensures that all Dynamips-specific properties conform to the GNS3 server schema by removing any keys not in the allowed set, preventing the 'Additional properties are not allowed' errors that plagued earlier versions of the exporter.

Appendix G: Complete User Interface Screenshots

This appendix documents the implemented StructuraNet AI web interface. The screenshots provide visual evidence of the public landing pages, authentication flow, conversational workspace, account and history controls, subscription workflow, privacy notice, and security information presented to users.

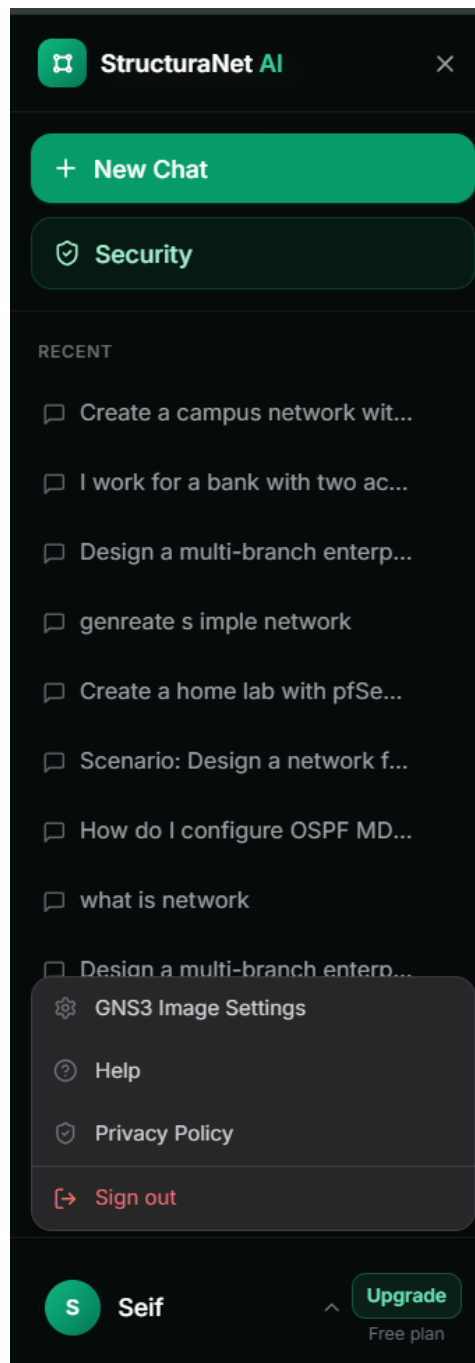


Figure G.1: Chat Interface — Account Properties.

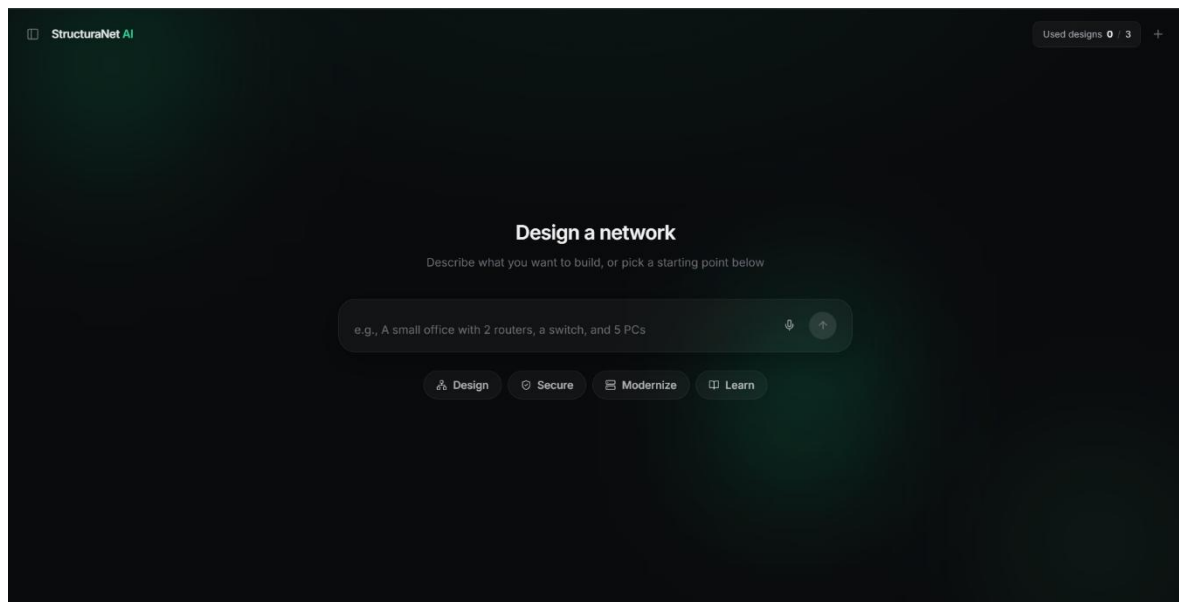


Figure G.2: Chat Interface — Main Workspace.

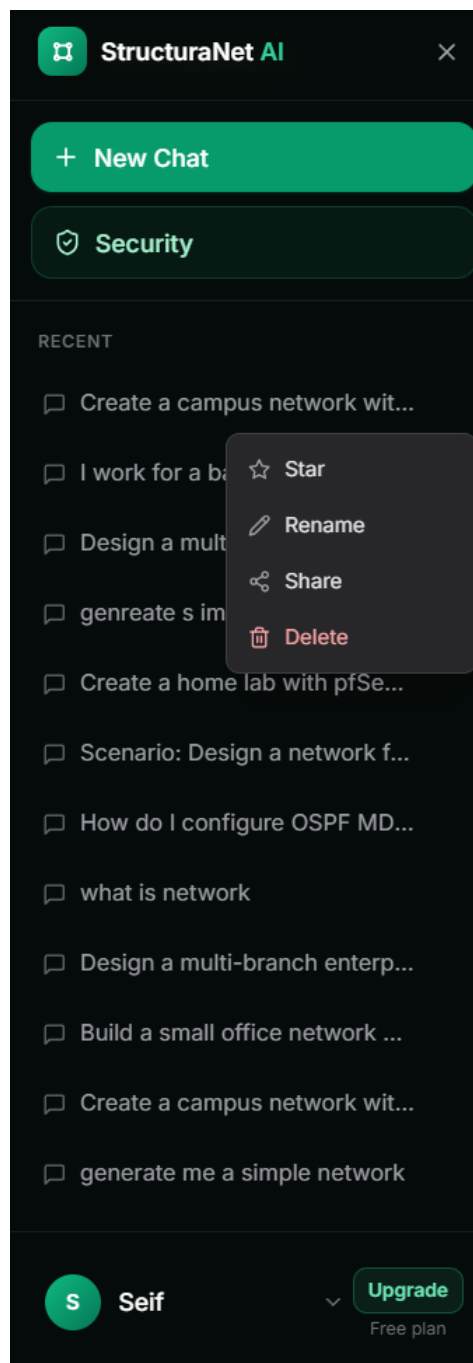


Figure G.3: Chat Interface — History Properties.

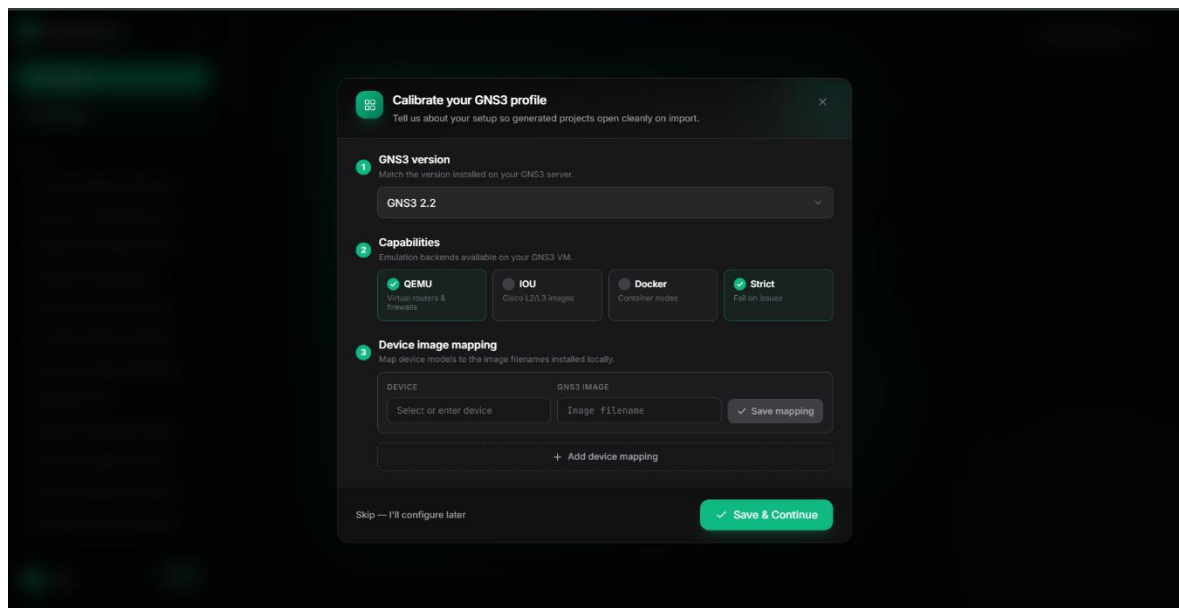


Figure G.4: Chat Interface — Image Specifications.



Figure G.5: Chat Interface — Sidebar.

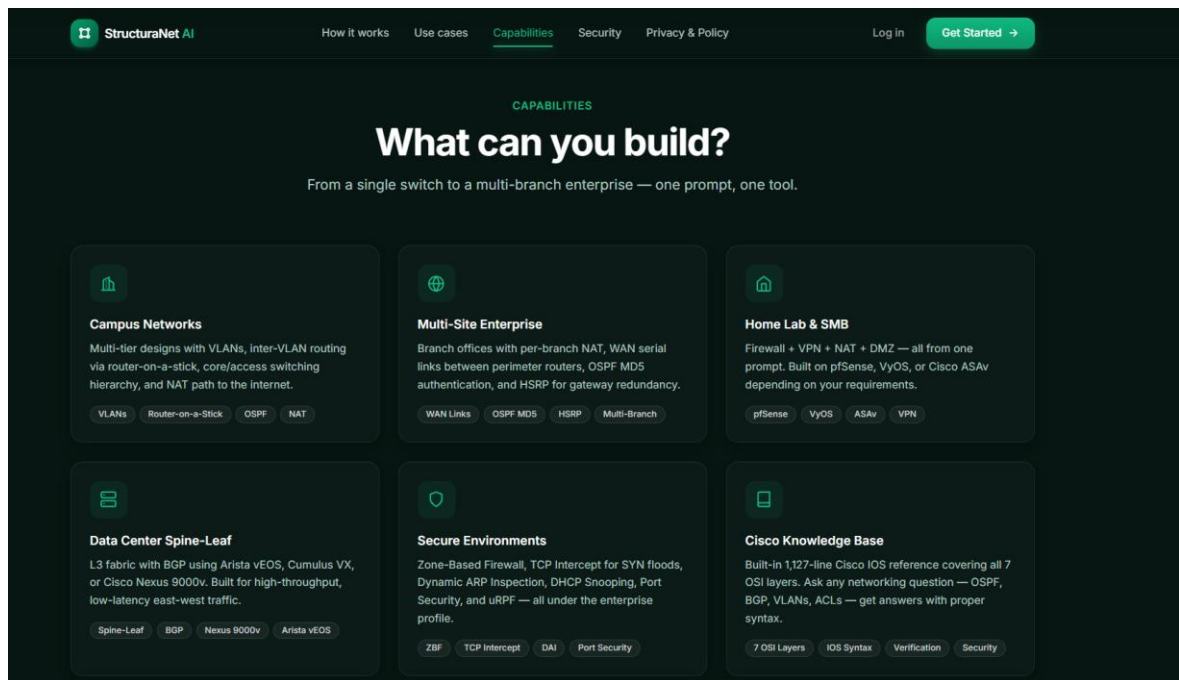


Figure G.6: Landing Interface — Capabilities.

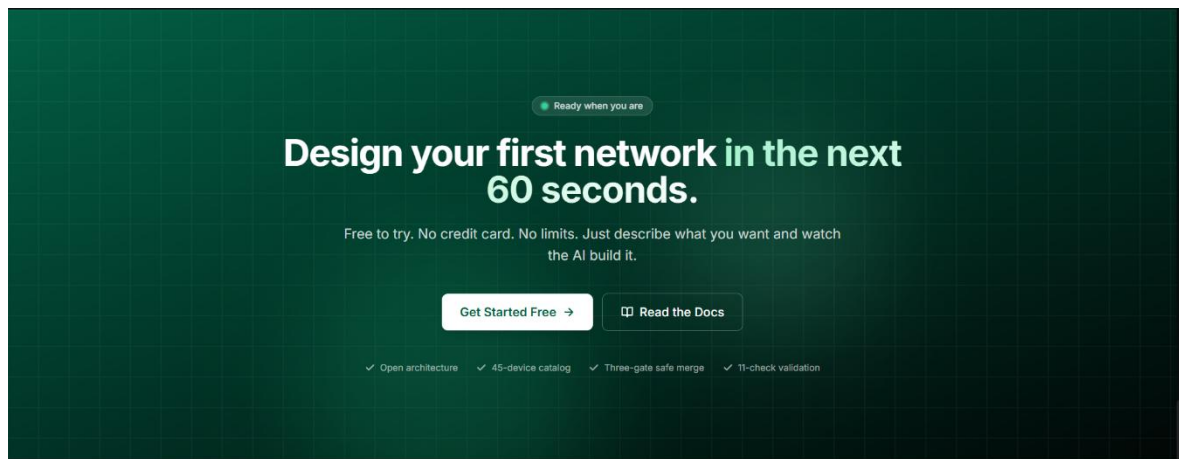


Figure G.7: Landing Interface — Documentation.

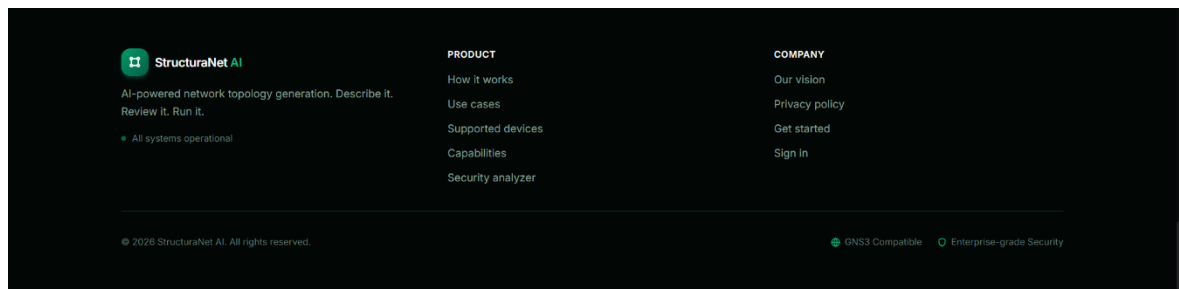


Figure G.8: Landing Interface — Footer.

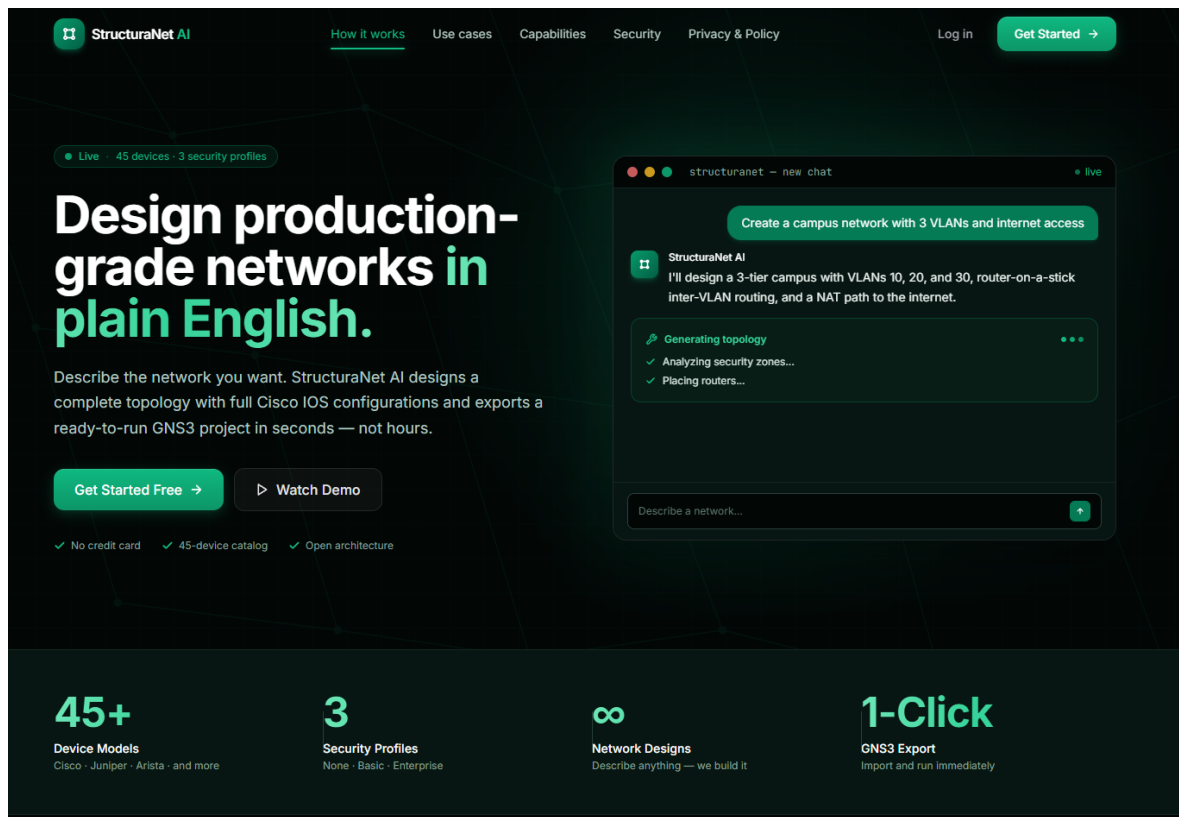


Figure G.9: Landing Interface — Home Page.

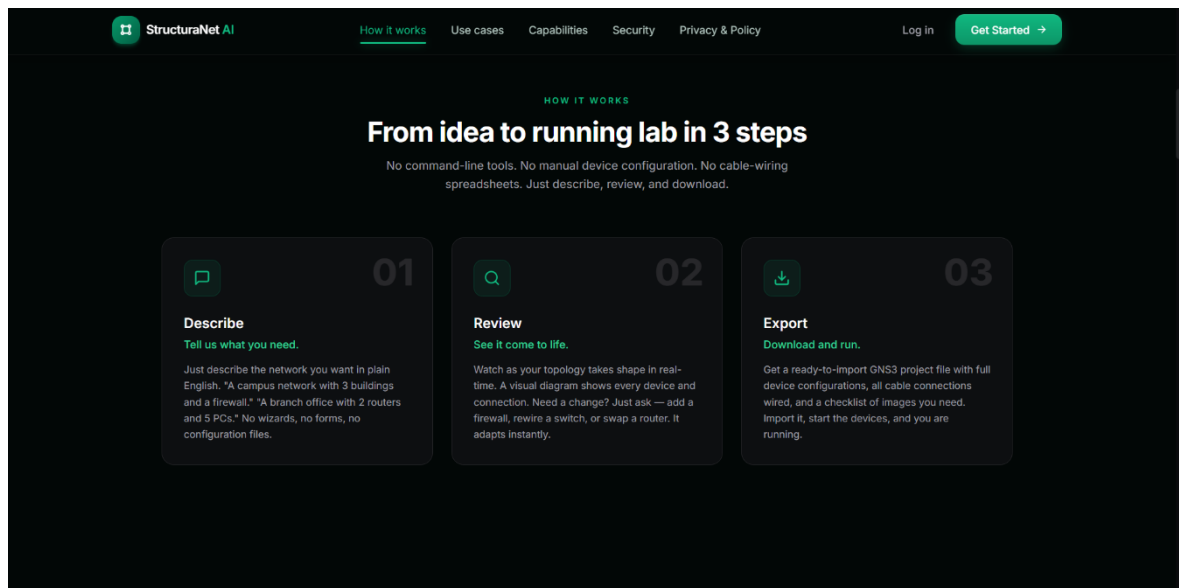


Figure G.10: Landing Interface — How It Works.

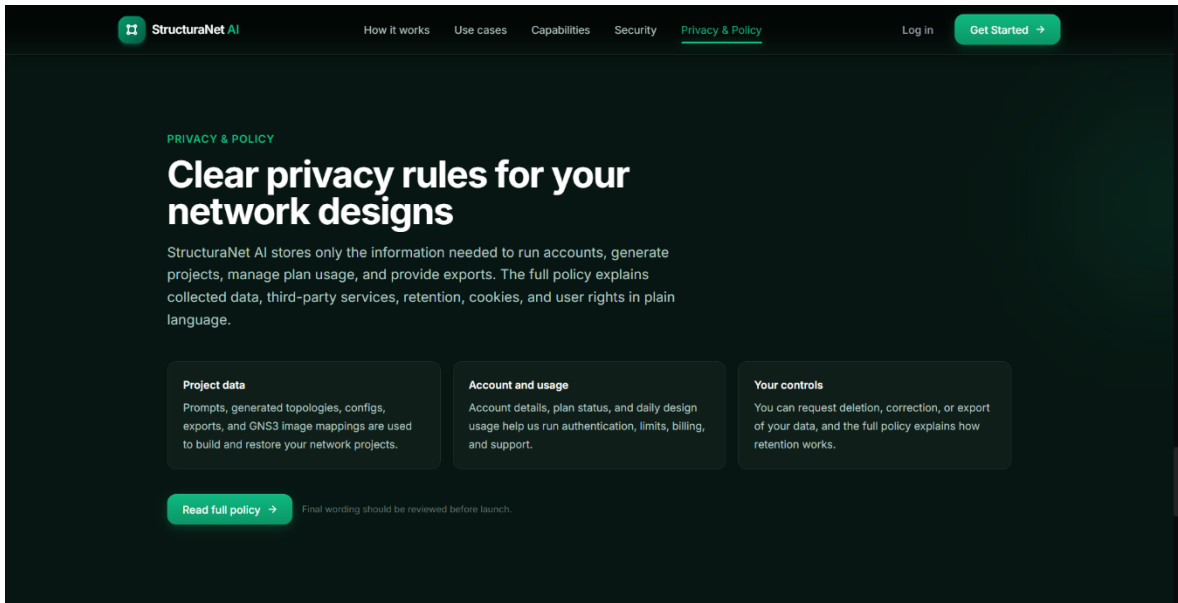


Figure G.11: Landing Interface — Privacy Policy.

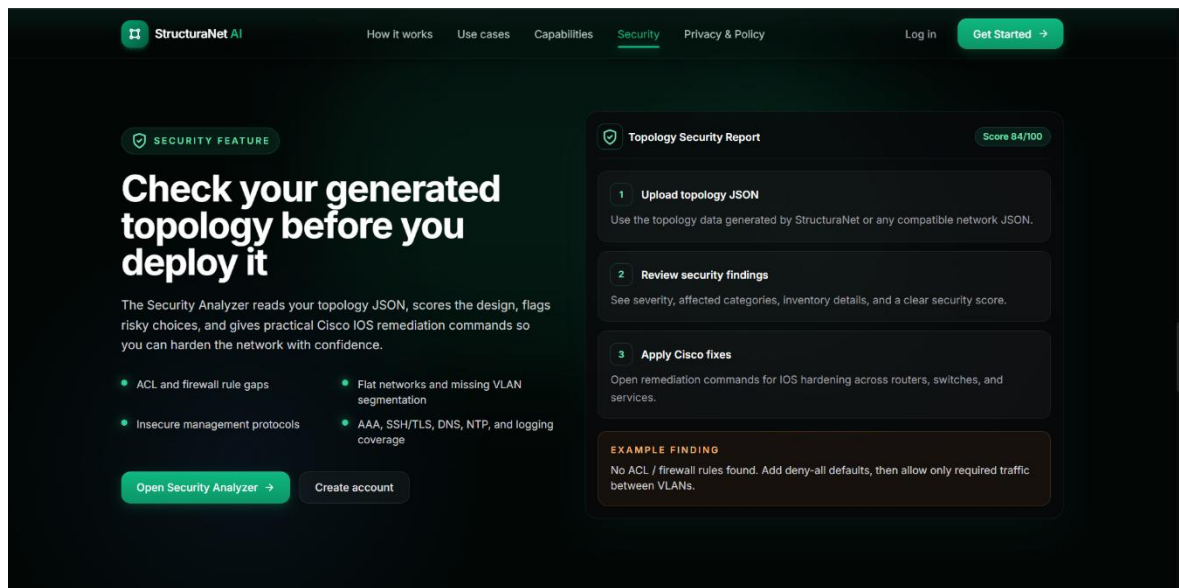


Figure G.12: Landing Interface — Security Information.

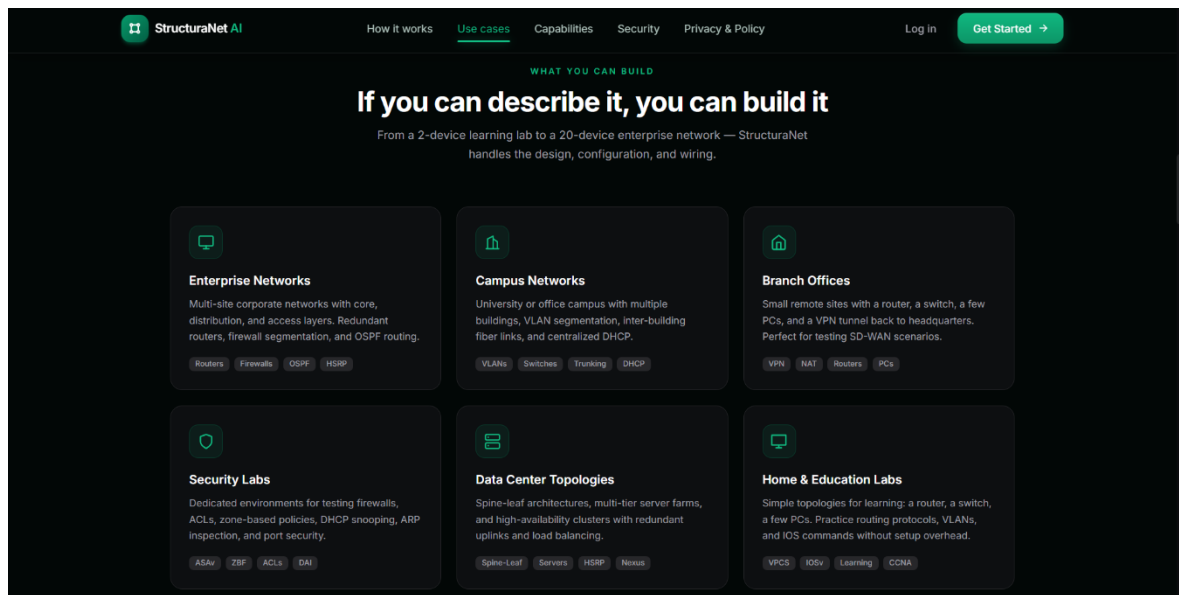


Figure G.13: Landing Interface — Use Cases.

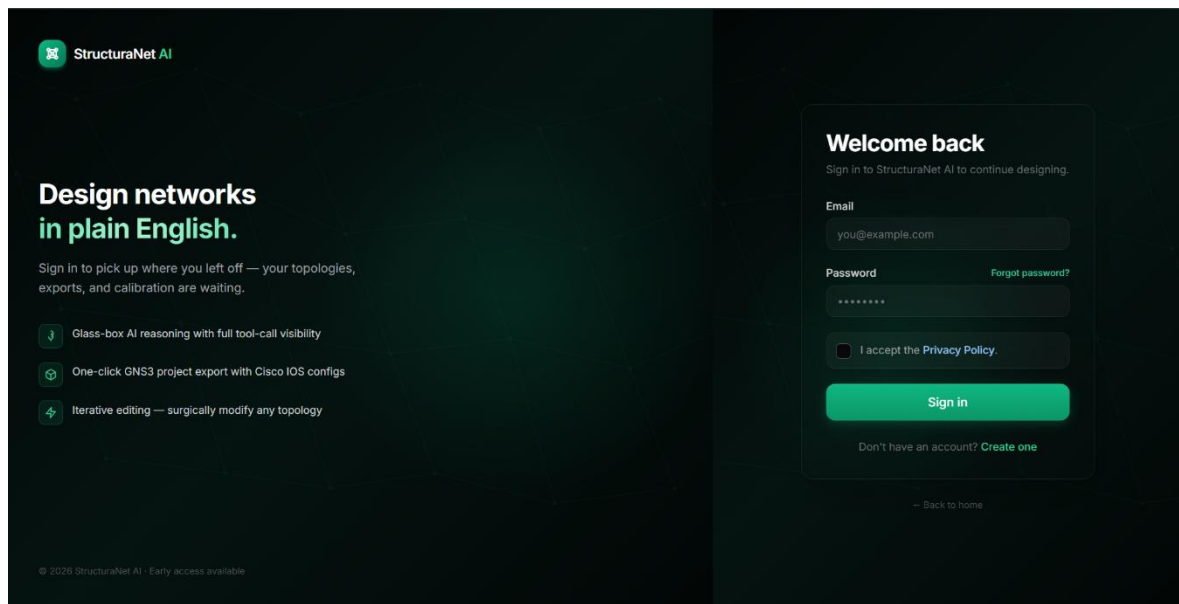


Figure G.14: Authentication Interface — Log In.

StructuraNet AI

Start designing production-grade networks.

Create a free account and get a GNS3-ready topology with full Cisco IOS configurations in seconds.

- No credit card required — free for academic use
- Three security profiles: None, Basic, Enterprise
- Iterative editing through natural-language chat

© 2028 StructuraNet AI - Early access available

Create your account

Start designing networks with AI in seconds.

Name

Email

Password

☒ I accept the [Privacy Policy](#).

Create account

Already have an account? [Sign in](#)

Figure G.15: Authentication Interface — Sign In.

Grauation sandbox

Sandbox

Subscribe to Pro

\$19.00

per month

Apple Pay

link

amazon pay

OR

Contact information

Email

seifmohamedyahya@gmail.com

Payment method

☐

Card

VISA

☐

Cash App Pay

☐

Bank

\$5 back

☐

Save my information for faster checkout

Pay securely at Grauation sandbox and everywhere link is accepted.

Subscribe

By subscribing, you authorize Grauation sandbox to charge you according to the terms until you cancel.

Powered by stripe | Terms | Privacy

Figure G.16: Payment Interface — Secondary Payment View.

←

Grauation sandbox

Sandbox

Plus

\$9.00

Apple Pay

link

amazon pay

OR

Contact information

Email

seifmohamedyahya@gmail.com

Payment method

☐

Card

VISA

MasterCard

Amex

Discover

☐

Cash App Pay

☐

Bank

☐

Klarna

Powered by Link

☐

Save my information for faster checkout

Pay securely at Grauation sandbox and everywhere Link is accepted.

Pay

Powered by stripe

Terms

Privacy

Figure G.17: Payment Interface — Payment Details.

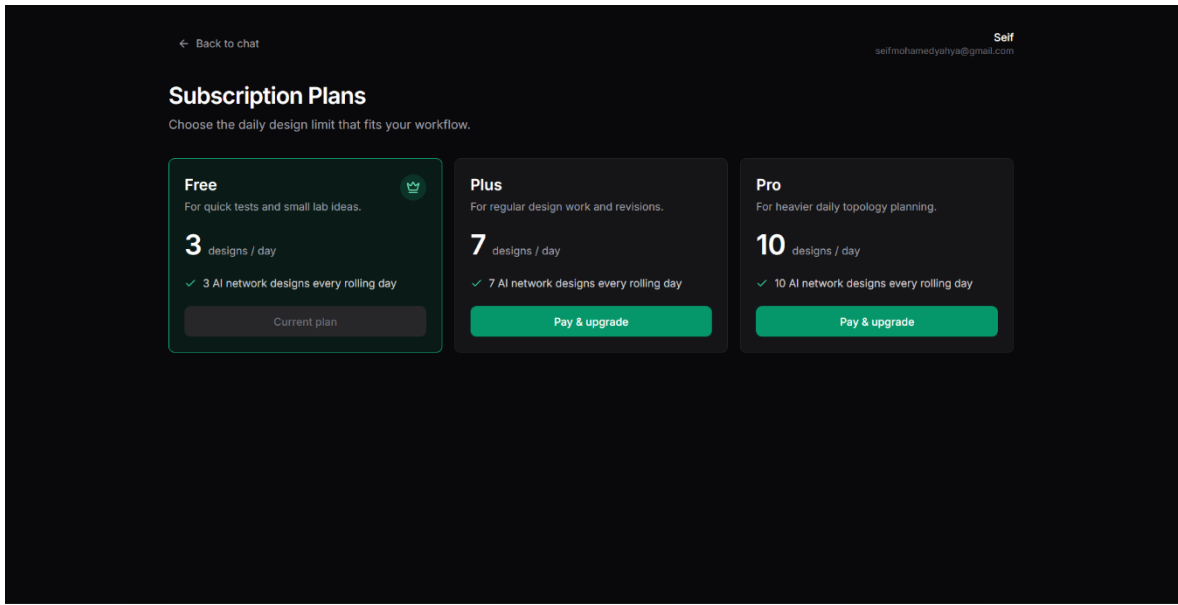


Figure G.18: Payment Interface — Subscription Upgrade.

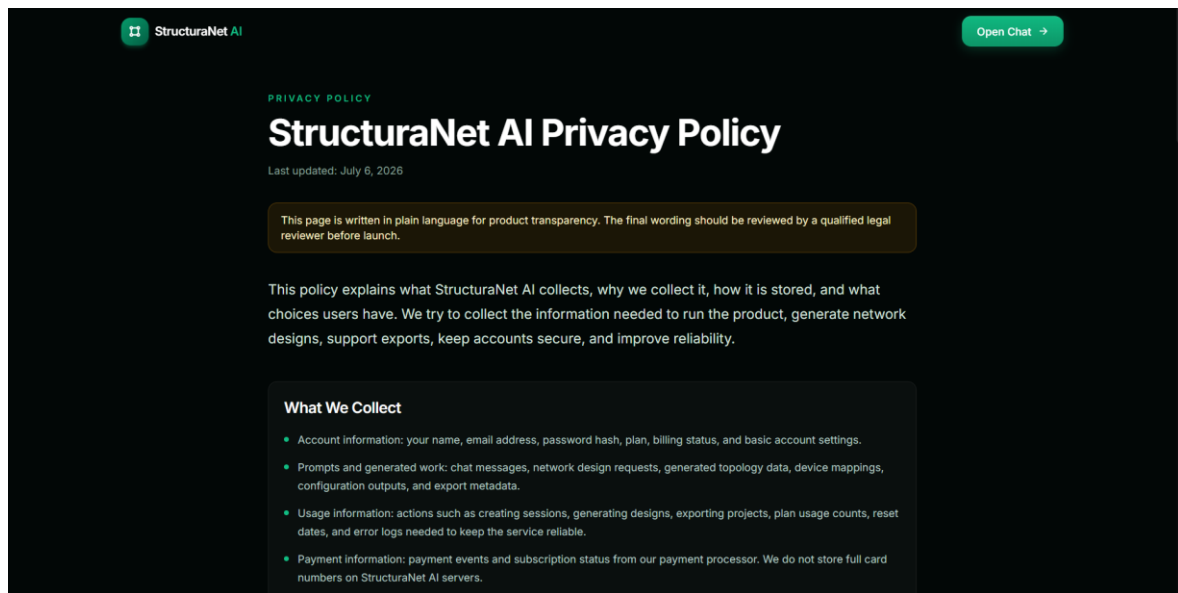


Figure G.19: Privacy Policy Interface — Page 1.

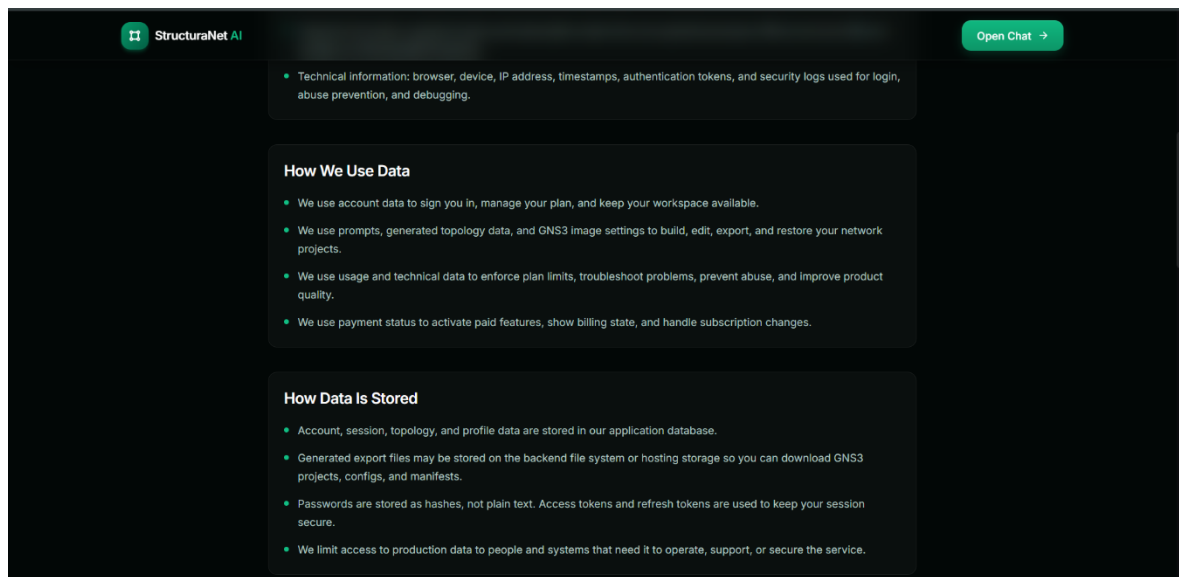


Figure G.20: Privacy Policy Interface — Page 2.

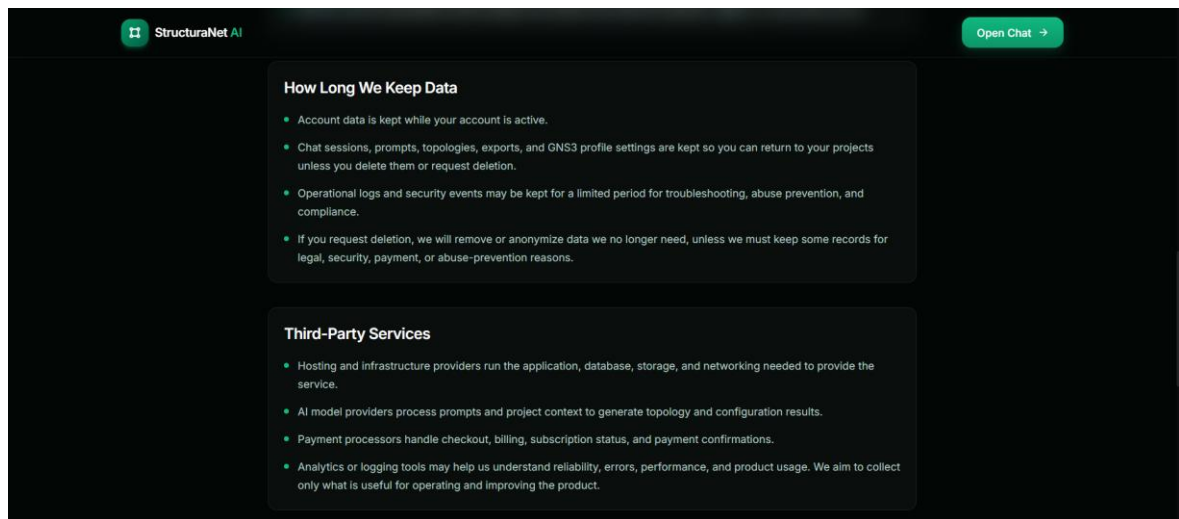


Figure G.21: Privacy Policy Interface — Page 3.

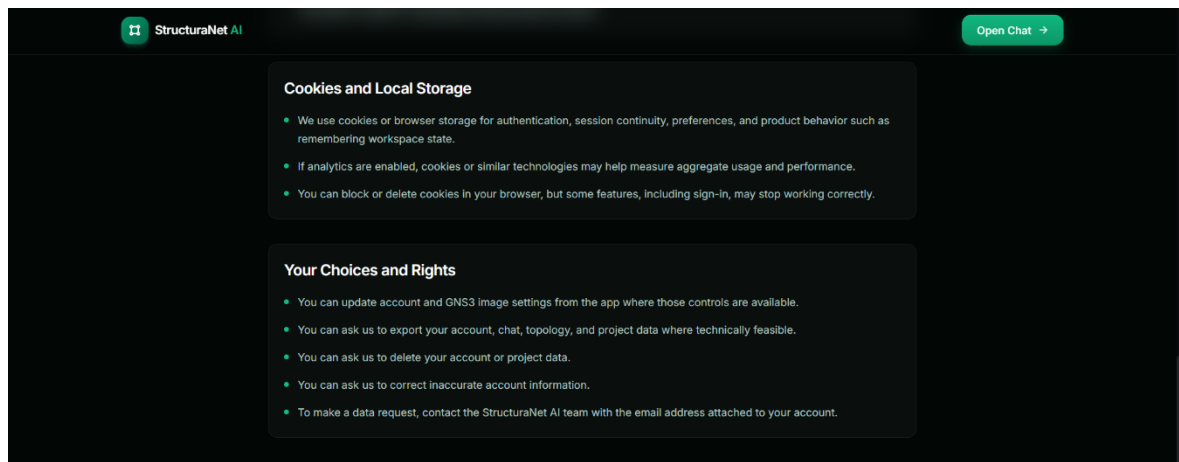


Figure G.22: Privacy Policy Interface — Page 4.

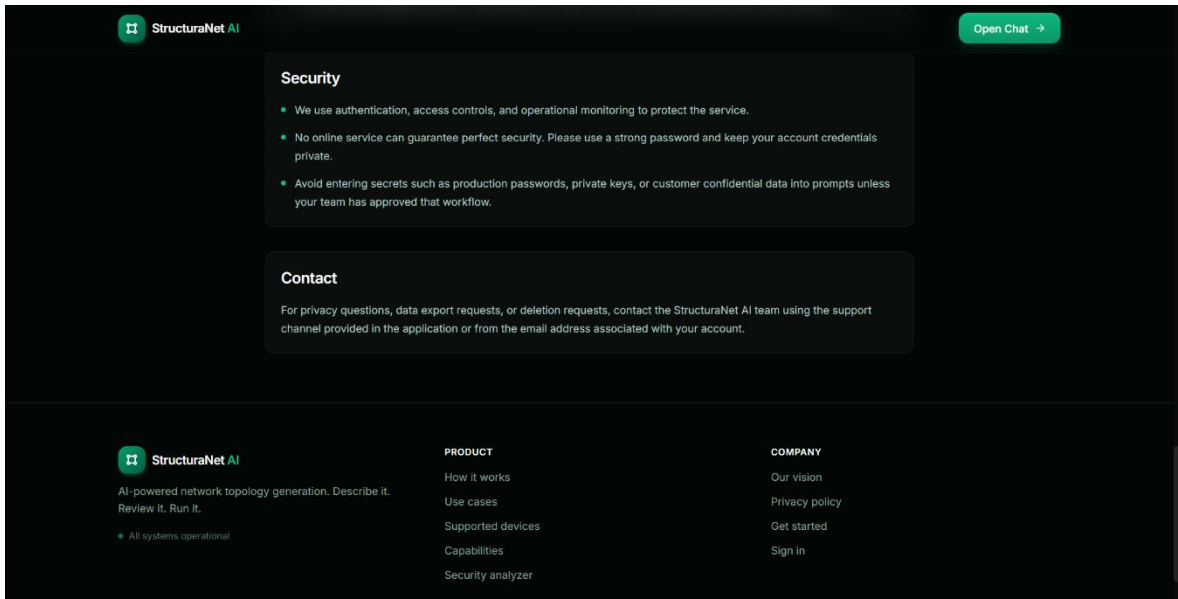


Figure G.23: Privacy Policy Interface — Page 5.

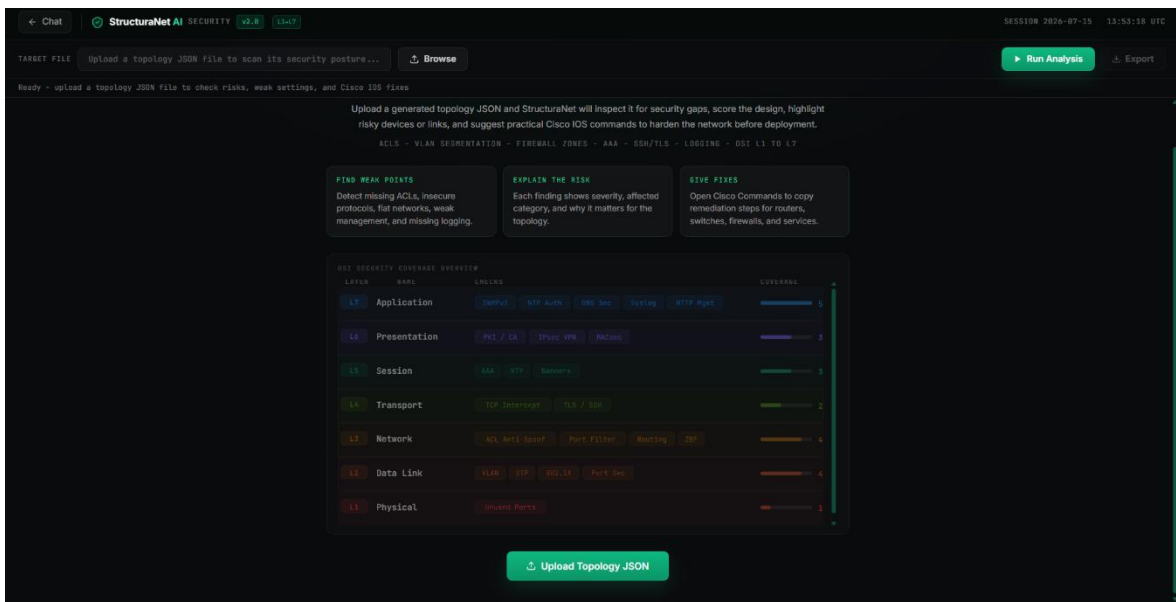


Figure G.24: Security Information Interface.

Appendix H: End-to-End Topology Generation Evidence

This appendix records three representative end-to-end scenarios. Each sequence shows the natural-language request, generated topology, design or configuration view, topology visualization, and downloadable output. Together, these screenshots provide implementation evidence for the workflow and system-testing discussion in Chapter 6.

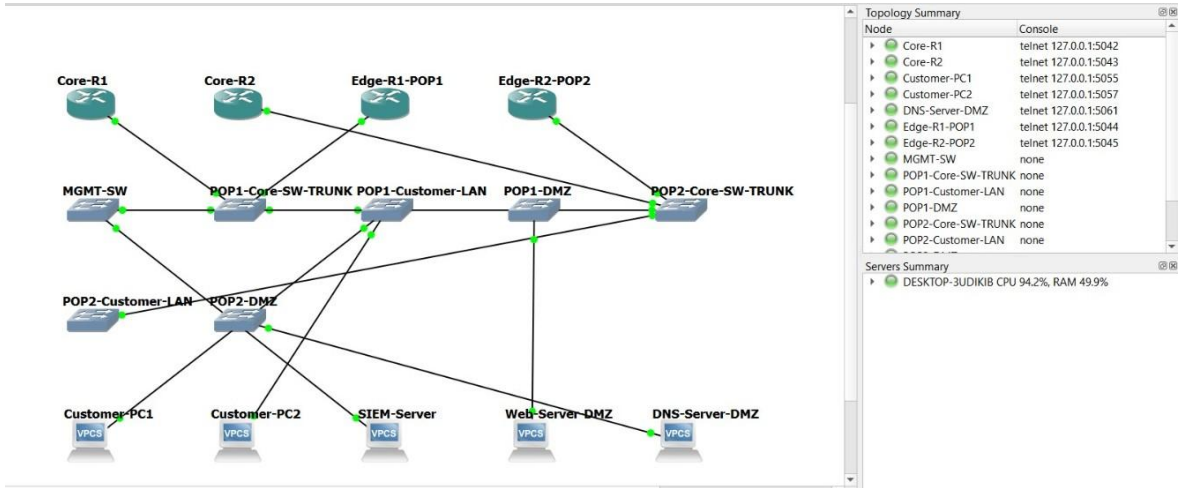


Figure H.1: Test Scenario 1 - Design.

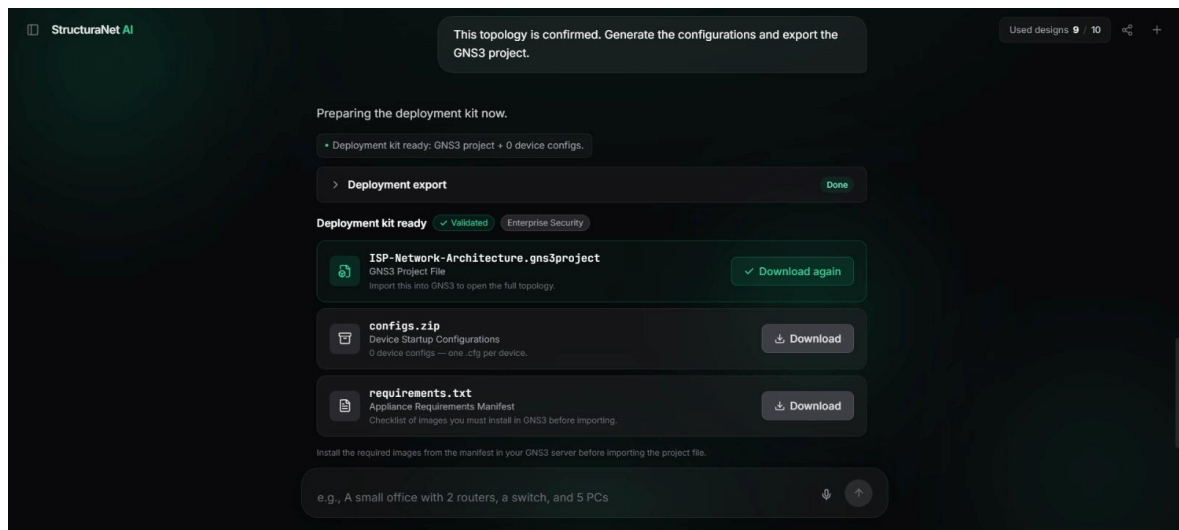


Figure H.2: Test Scenario 1 - Download.

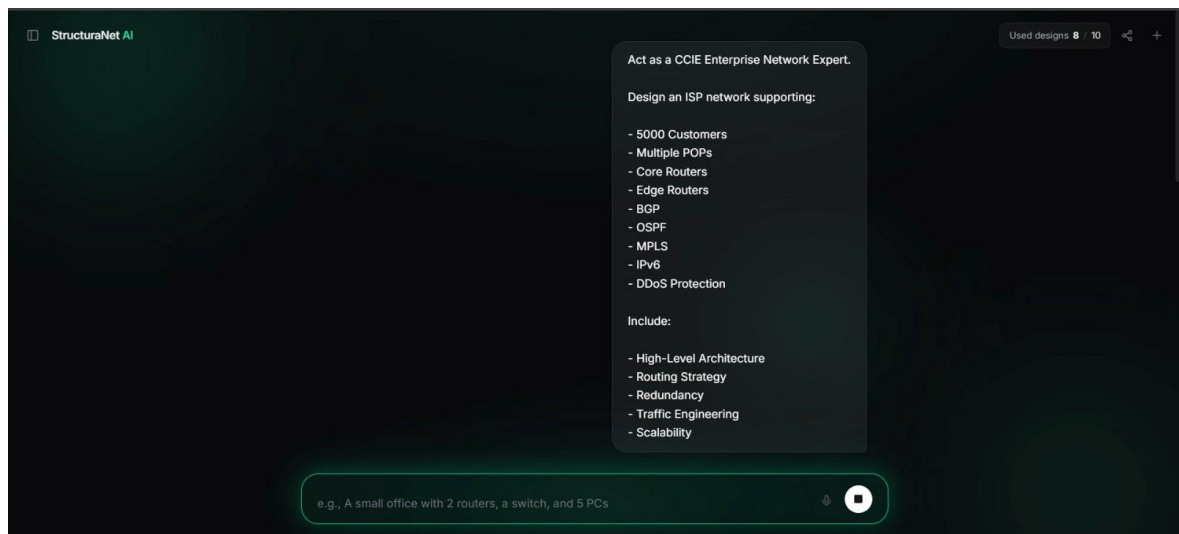


Figure H.3: Test Scenario 1 - Prompt.

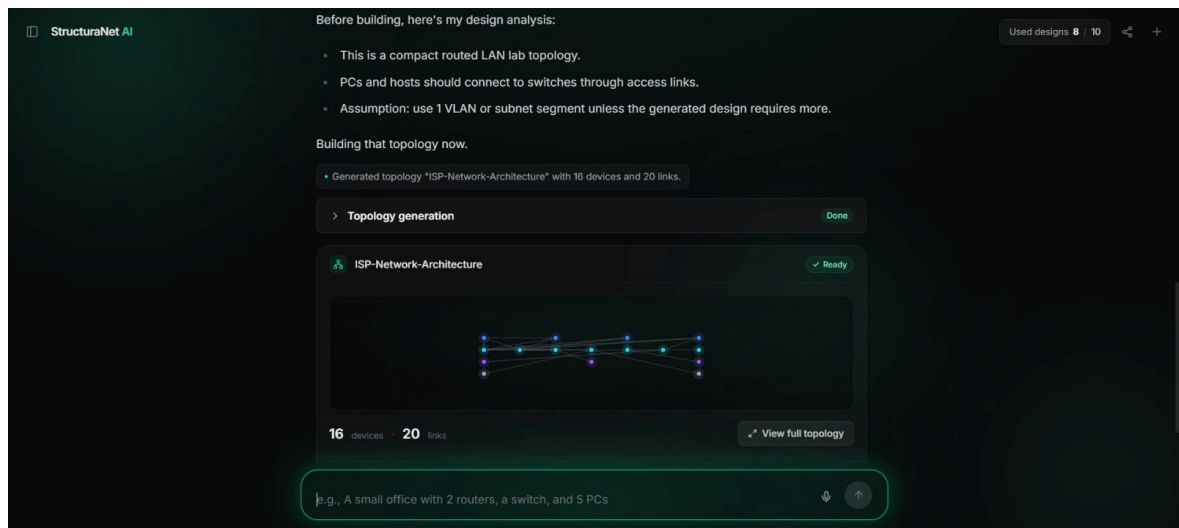


Figure H.4: Test Scenario 1 — Topology Preview.

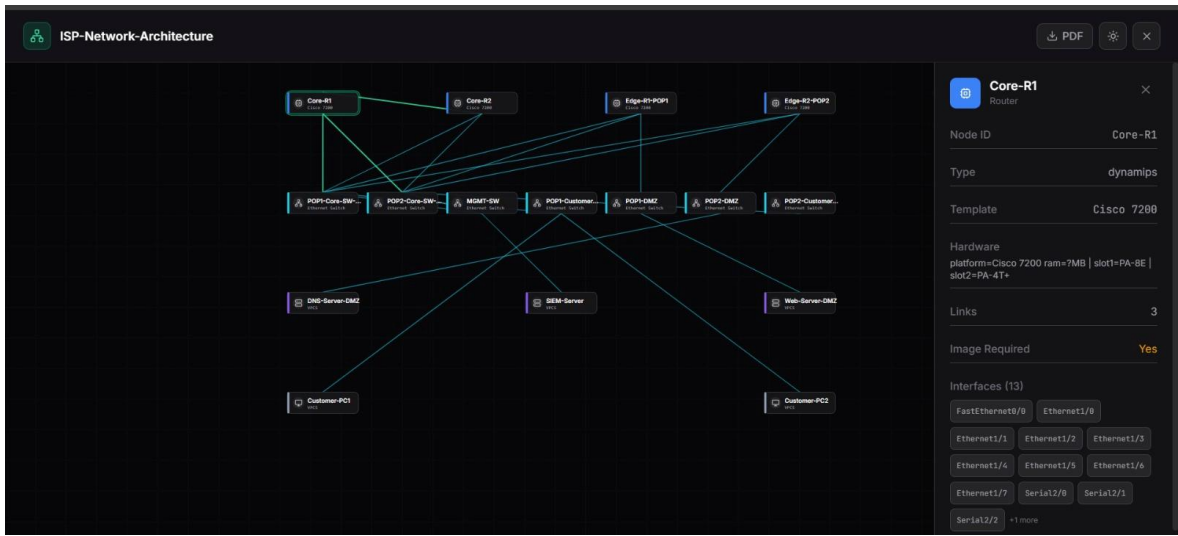


Figure H.5: Test Scenario 1 — Generated Topology.

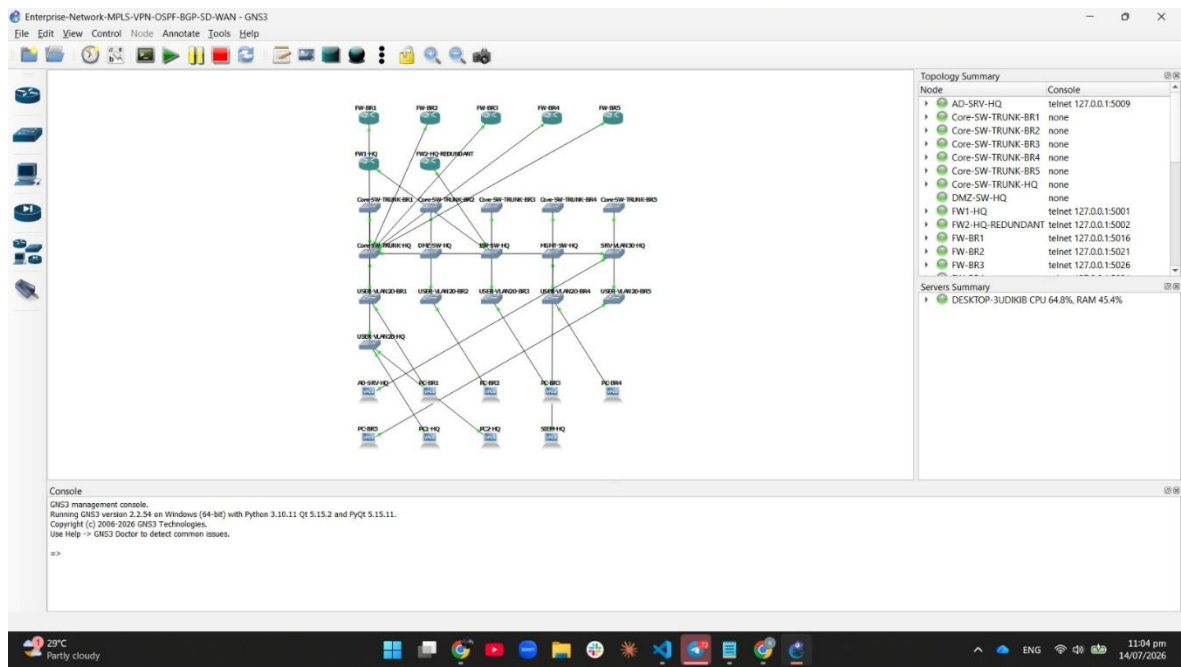


Figure H.6: Test Scenario 2 - Design.

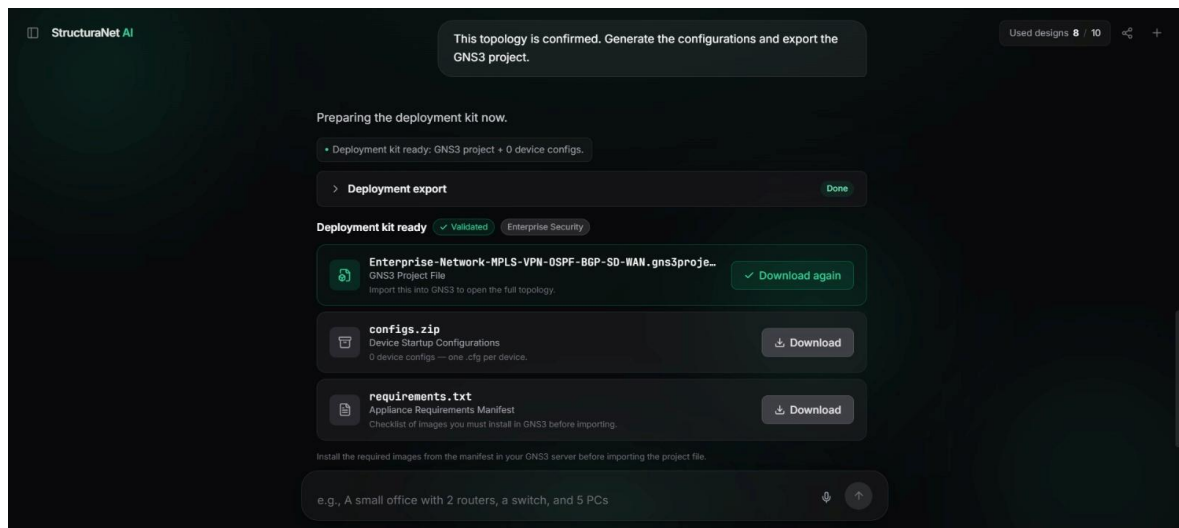


Figure H.7: Test Scenario 2 - Download.

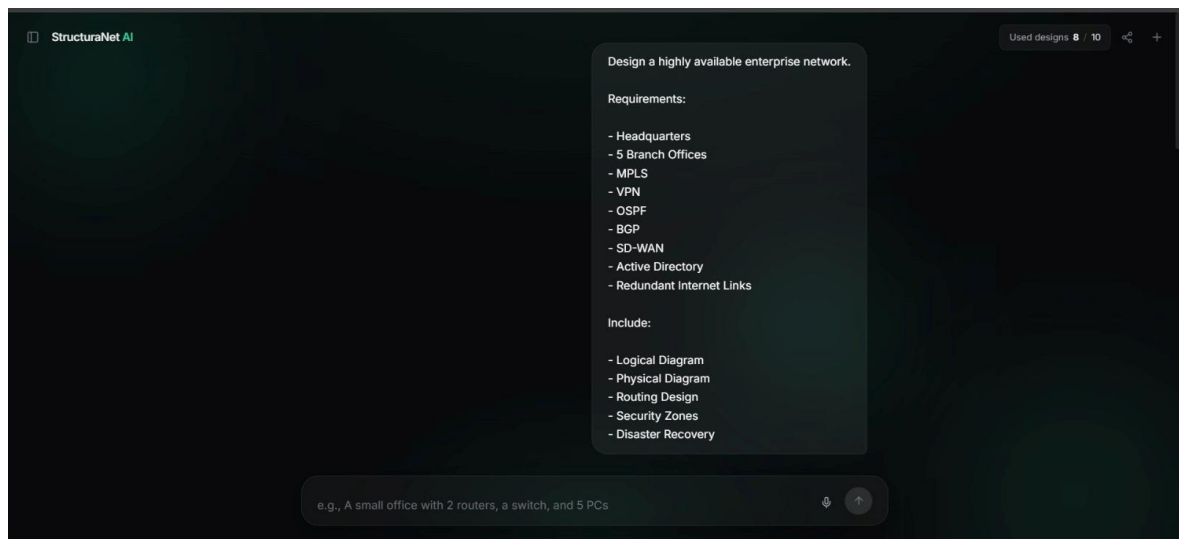


Figure H.8: Test Scenario 2 - Prompt.

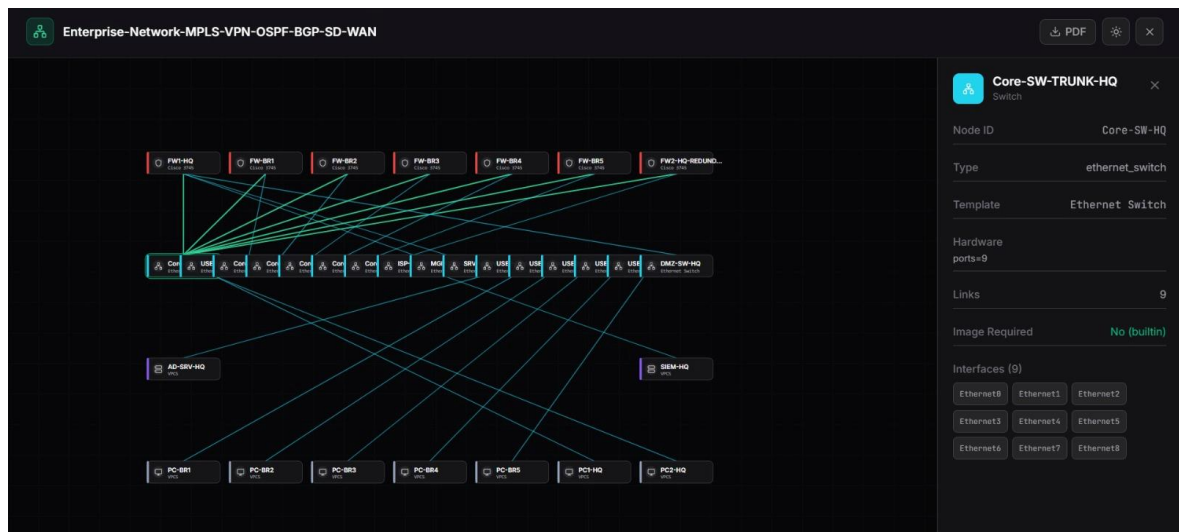


Figure H.9: Test Scenario 2 — Generated Topology.

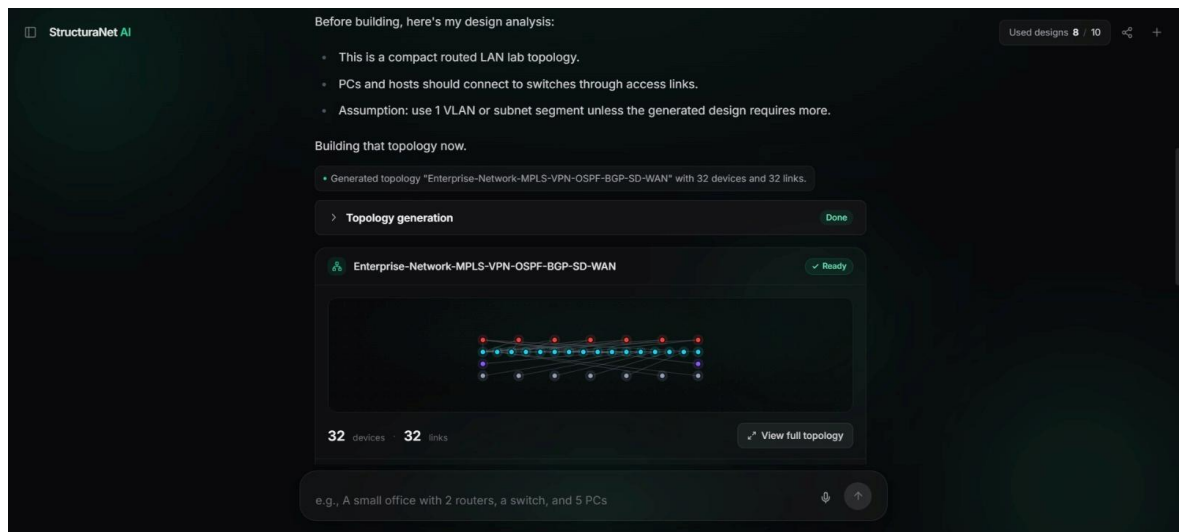


Figure H.10: Test Scenario 2 — Topology View.

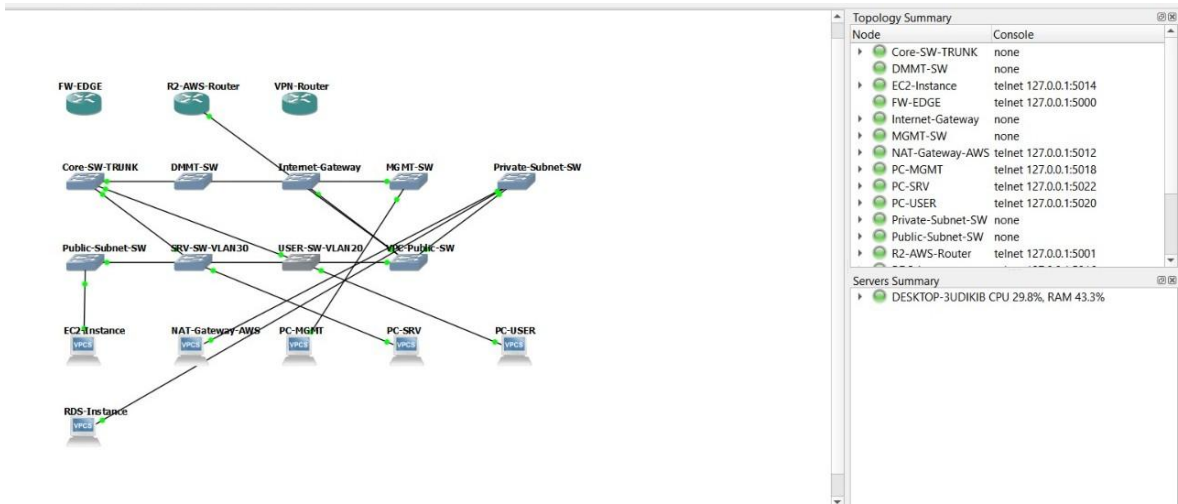


Figure H.11: Test Scenario 3 - Design.

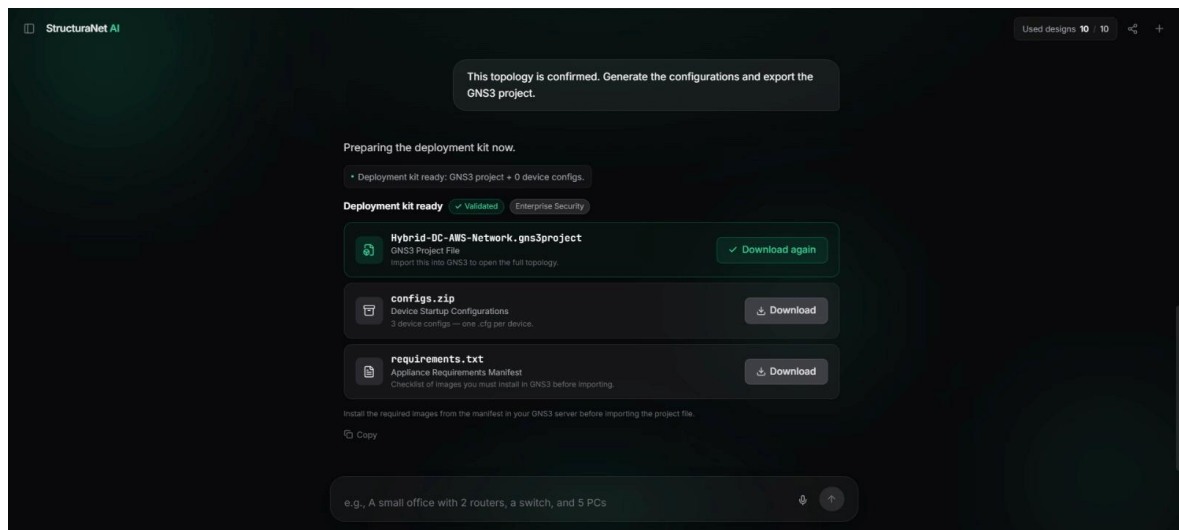


Figure H.12: Test Scenario 3 - Download.

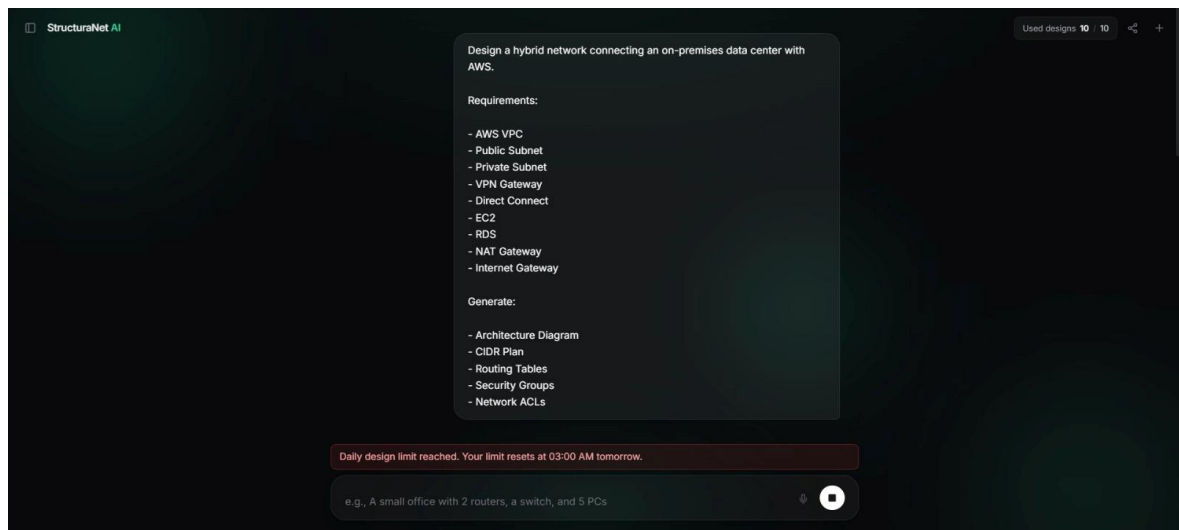


Figure H.13: Test Scenario 3 - Prompt.

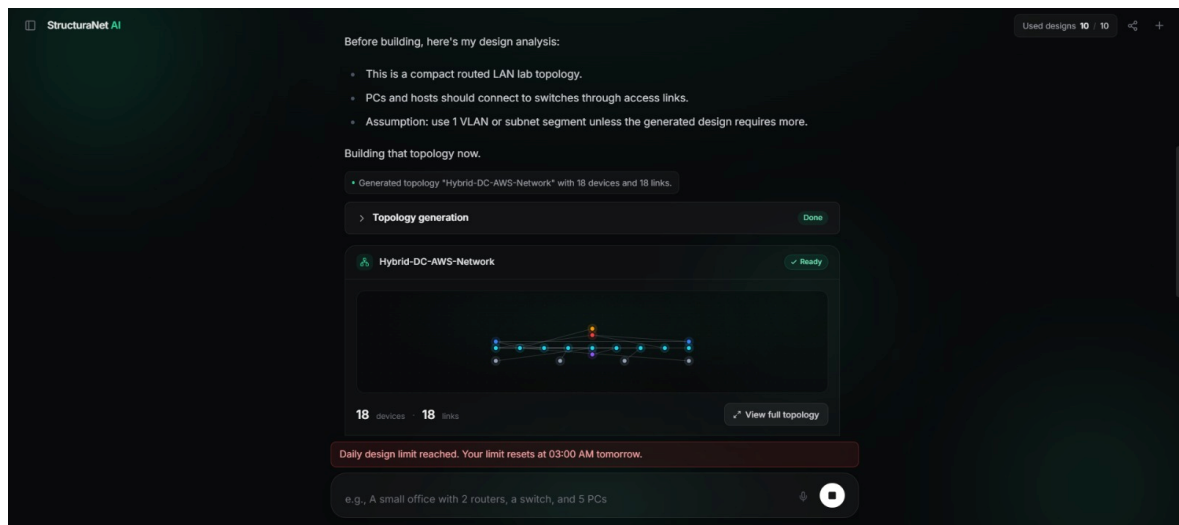


Figure H.14: Test Scenario 3 — Topology Preview.

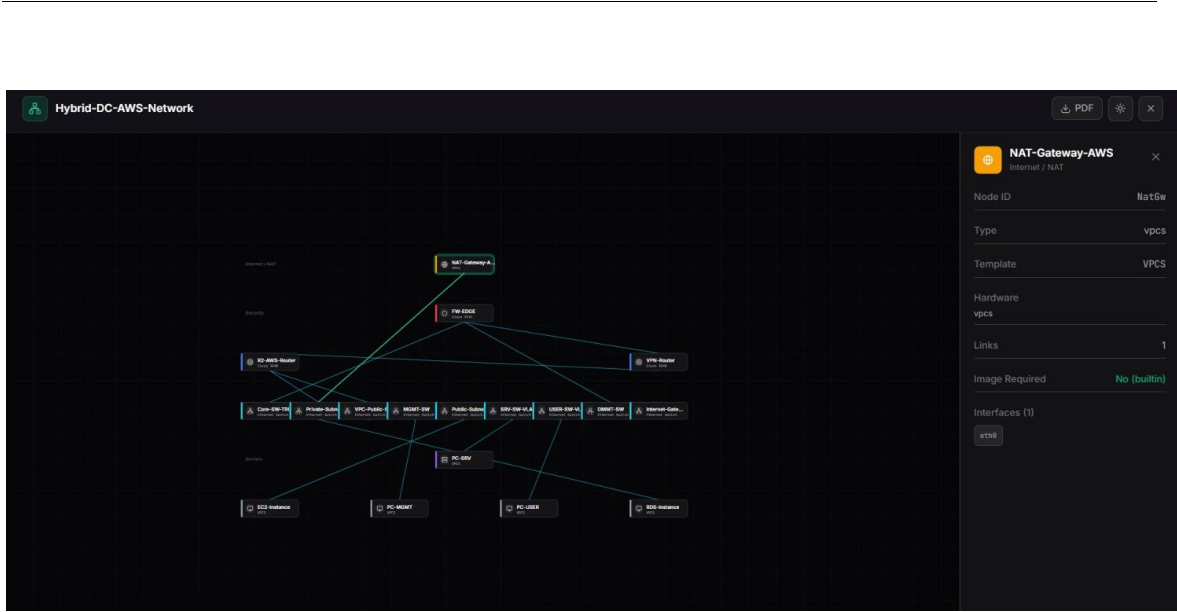


Figure H.15: Test Scenario 3 — Generated Topology.

Appendix I: Security Analyzer Interface Evidence

This appendix presents the implemented security-analysis interface and its output views. The screenshots complement the thesis discussion of configuration verification and security-oriented validation.

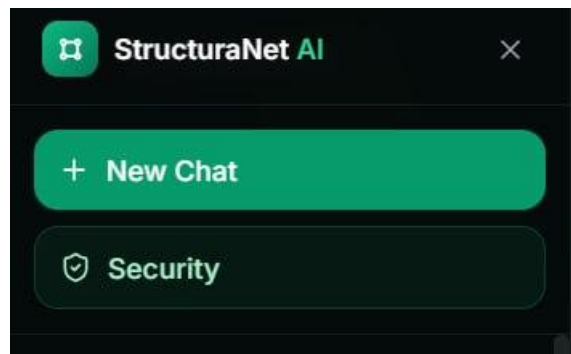


Figure I.1: Security analyzer interface view 1.

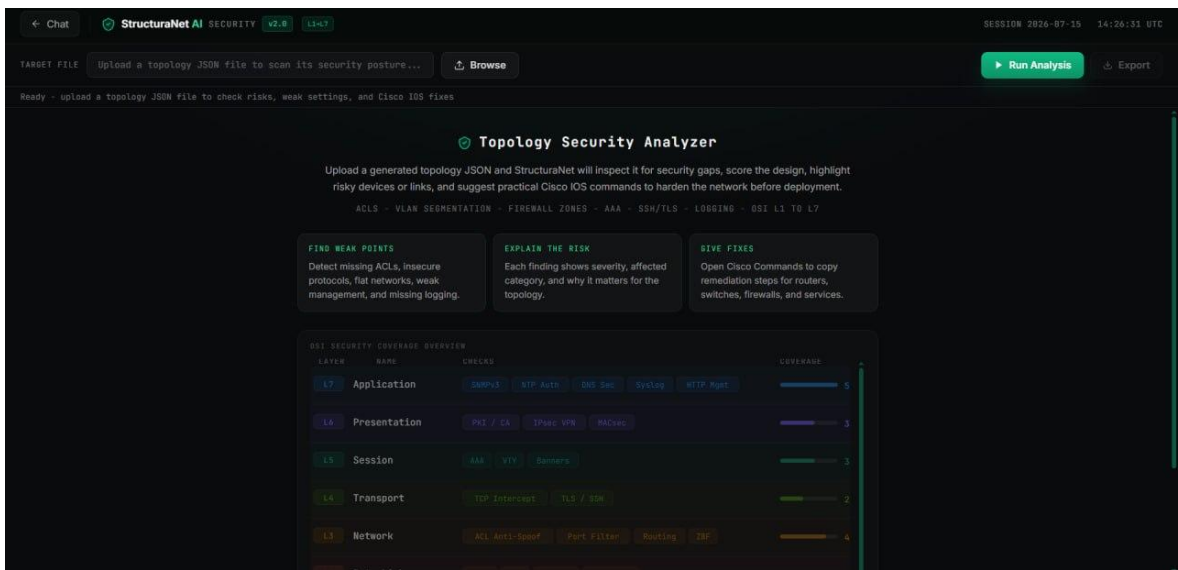


Figure I.2: Security analyzer interface view 2.

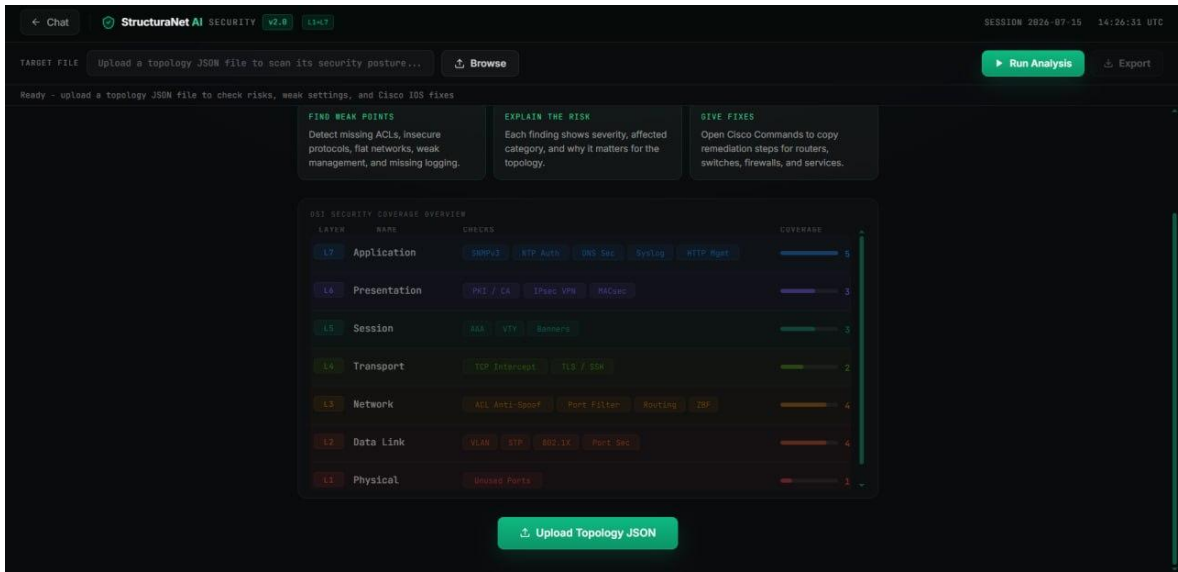


Figure I.3: Security analyzer interface view 3.

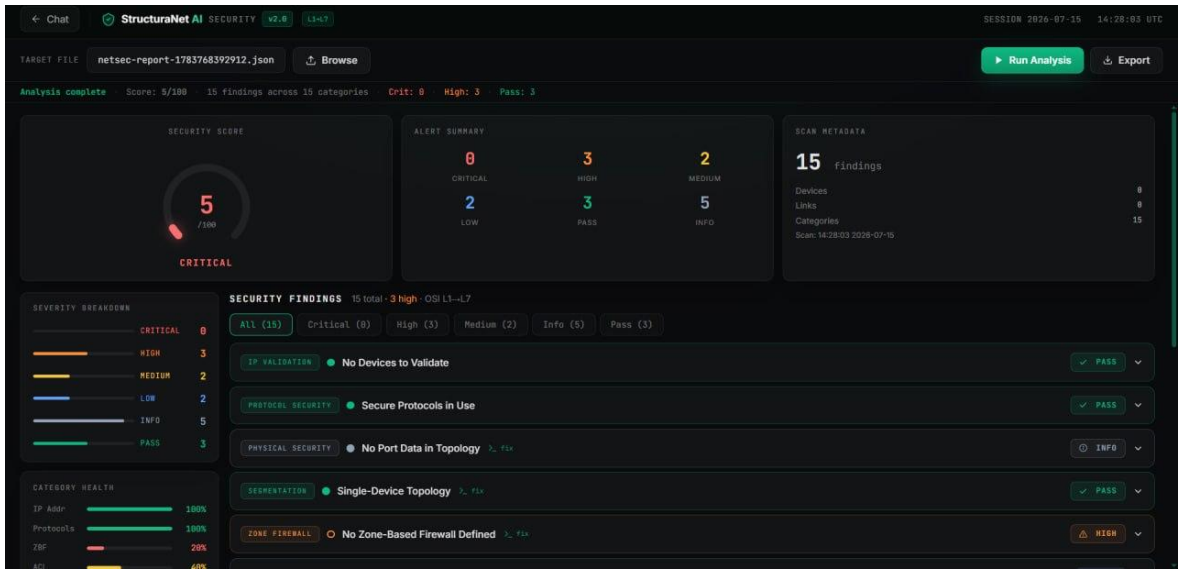


Figure I.4: Security analyzer interface view 4.

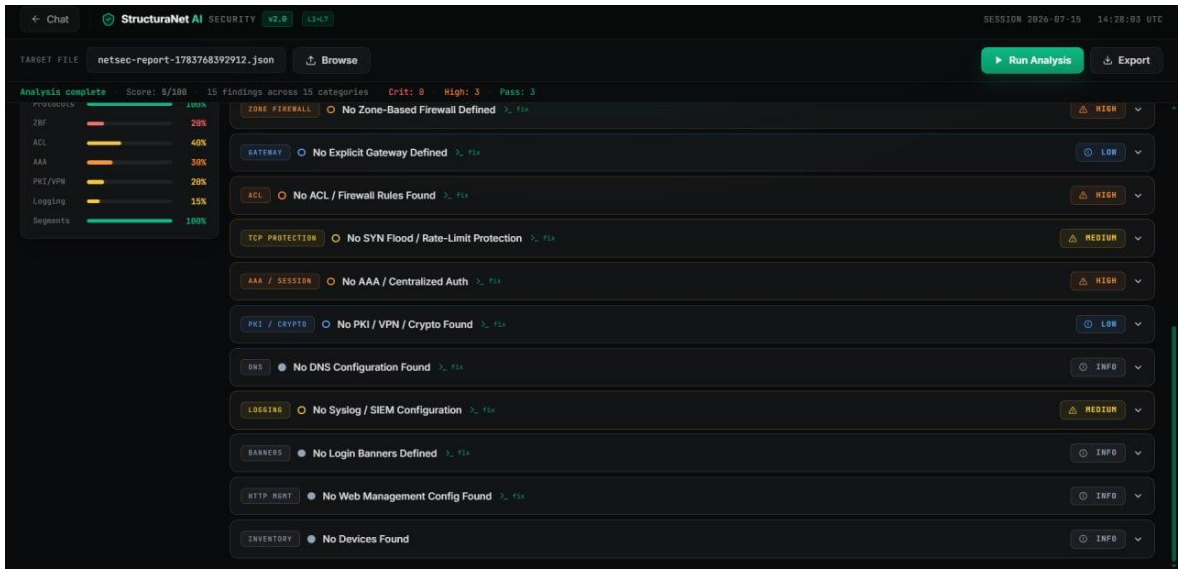


Figure I.5: Security analyzer interface view 5.

Appendix J: Supplementary Strategic Analysis

J.1 SWOT Analysis

A SWOT (Strengths, Weaknesses, Opportunities, Threats) analysis provides a structured framework for evaluating StructuraNet AI's internal capabilities and external environment. This analysis informs the project's strategic direction and identifies areas for improvement and growth.

Table J.1: SWOT Analysis of StructuraNet AI.

Strengths • Novel NL-to-topology approach using LLM function calling • Three-gate safe-merge prevents LLM hardware corruption • Auto-repair pipeline corrects common LLM errors pre-validation • 45-device catalog with deterministic hardware injection • Real-time SSE streaming for transparent AI progress • Three security profile tiers (None, Basic, Enterprise) • Full GNS3 export pipeline with 11-check structural validation	Weaknesses • Output accuracy depends on the underlying LLM's capabilities • Sequential worker processing limits concurrent throughput • Configuration generation limited to Cisco IOS syntax • Topology size constrained by LLM token output window • Requires OpenAI-compatible API key (external dependency) • No support for serial, ATM, or Frame Relay link types • Relatively new project with limited real-world deployment data
Opportunities • Growing demand for AI-assisted network automation in DevOps • Educational market needs for rapid lab topology generation • Extension to multi-vendor support (Juniper, Arista, Huawei) • Integration with cloud provider APIs for hybrid topologies • Potential for Ansible/Terraform playbook export • Subscription-based SaaS model with tiered feature access • Enterprise partnerships for pre-sales network prototyping	Threats • Rapid LLM advancements may lower entry barriers for competitors • Cisco and other vendors may integrate AI natively into their tools • Open-source alternatives could replicate core functionality • API provider pricing changes could affect operational costs • Dependence on GNS3 project file format (changes may break export) • User expectations may exceed what current LLMs can reliably deliver

The SWOT analysis reveals that StructuraNet AI's primary strengths lie in its novel approach to network design automation and its robust engineering (three-gate

safe-merge, auto-repair pipeline, structural validation). The main weaknesses relate to the inherent limitations of LLM-dependent systems and the current single-vendor (Cisco IOS) configuration scope. The opportunities are significant, given the growing demand for AI-assisted engineering tools in both education and enterprise markets. The threats are primarily related to the rapidly evolving AI landscape and the potential for established vendors (Cisco, Juniper) to integrate similar capabilities into their native tools.

J.2 Business Model Canvas

The Business Model Canvas provides a strategic management template for describing, analyzing, and designing StructuraNet AI's business model. The canvas consists of nine building blocks that cover the four main areas of any business: customers, offer, infrastructure, and financial viability. The following canvas outlines the proposed business model for StructuraNet AI as a potential commercial product.

Table J.2: Business Model Canvas for StructuraNet AI.

Key Partners	Key Activities	Value Propositions	Customer Relationships	Customer Segments	Key Resources	Channels	Cost Structure	Revenue Streams
• GNS3 open-source community • OpenRouter / OpenAI (LLM API) • Educational institutions • Cisco certification programs	• LLM prompt engineering • GNS3 export pipeline dev. • Topology validation engine • User experience design	• NL-to-topology in seconds • Iterative conversational design • GNS3-ready export files • Security-hardened configs	• Self-service SaaS platform • Community forums / docs • Interactive chat interface • Academic licensing	• Networking students • University instructors • Network engineers • DevOps teams	• LLM API access • 45-device GNS3 catalog • Cisco knowledge base • Engineering team	• Web application • University partnerships • GitHub repository • Social media / SEO	• LLM API usage costs • Cloud hosting (VPS/DB) • Development & maintenance • Marketing & outreach	• Freemium (free tier + limits) • Premium subscription plans • Academic institution licenses • Enterprise custom contracts

The proposed business model follows a freemium SaaS (Software as a Service) pattern, where a free tier provides basic topology generation with limited complexity, while premium subscription plans unlock advanced features such as enterprise security profiles, larger topology sizes, and priority processing. Academic institution licenses provide bulk access at discounted rates, while enterprise custom contracts offer tailored solutions for organizations with specific requirements. The primary cost drivers are LLM API usage (which scales with the number and complexity of generations) and cloud hosting infrastructure. Revenue generation is supported through the freemium conversion funnel, where users who find value in the free tier are incentivized to upgrade for enhanced capabilities.